AMAZON SDE-I · ONSITE LOOP

The 5-Day Final Plan & Complete Answer Key

Tue 1 Sept · Herndon, VA · 4 × 60 min · LiveCode (typed)

Prep window: Tue 25 Aug → Sat 29 Aug · 32 problems

EVERY ANSWER IN THIS BOOK FOLLOWS ONE ORDER

1. Clarification communication
2. Understanding of the question
3. Brute-force approach in English
4. Data structures used
5. Step-by-step algorithm
6. Time & space complexity with explanation
7. Fully commented brute-force code
8. Limitation of brute force
9. Optimal approach in English
10. Optimal data structures
11. Step-by-step optimal algorithm
12. Fully commented optimal code

Part I — the plan, the story block, the design block. Part II — all 32 problems in the twelve-section format above.

Part I — The 5-Day Plan

Five prep days: Tue 25 Aug through Sat 29 Aug. 30–31 Aug are travel days, not prep days.

Read this first

This plan is organised by **round type**, not by topic — so each day rehearses the shape of an actual interview rather than a chapter of a textbook.

> **One important correction to the round table:** there is no separate behavioural round at Amazon. Every interviewer is assigned 2–3 Leadership Principles and must return evidence, so ownership and failure questions land *inside* your coding rounds — usually in the first 10 minutes. That is why there is a 30-minute story block every day rather than a single behavioural day.

Daily shape — about 3.5 hours

Block	Time	What
Coding	2h 30m	The 6 problems for today, 25-minute timer each
Stories	30 min	Leadership Principle stories (see the story block below)
Design	30 min	One topic, out loud, on paper

If you run short, cut the stretch problems first, then the design block. **Never cut the stories** — with zero prepared stories you are exposed in all four rounds, and 30 minutes a day fixes that.

The three rules

1. **Blank page.** Copilot and IntelliSense off. Plain editor. Paste into LeetCode only when the whole solution is written.
2. **25-minute timer.** Stuck at 8 minutes → read only the *Understanding* and *Optimal approach* sections here, never the code. At 20 minutes → read the code, close the book, rewrite it from scratch.
3. **Four sentences out loud after every problem**, including the easy ones: *"This is a [pattern]. My [structure] holds [what it means]. I [movement rule], because [why that is correct]. Each element is [counted work], so O(…) time, O(…) space."*

Day 1 · Tue 25 Aug

ROUND TYPE A — Graph & tree traversal under constraints. *"Given a graph/tree, find or optimize a path/traversal under constraints."* The single most likely coding round for an SDE-I loop, and where Amazon's own repeat questions cluster.

#	Problem	What to watch
200	Number of Islands ★★	Do it BFS **and** DFS. Ask: may I mutate the grid?
994	Rotting Oranges ★★	Multi-source BFS. Seed every rotten cell before you start.
207	Course Schedule ★	Kahn's topo sort. `[a,b]` means b before a, so the edge is b→a.
127	Word Ladder ★	BFS generating neighbours (26 swaps per position), not pairwise compare.
236	LCA of a Binary Tree ★★	Both subtrees return non-null ⇒ this node. Solve it twice.
102	Level Order Traversal	`for _ in range(len(q))` is the level-size snapshot.
Stretch — only if the six above are done and clean		
417	Pacific Atlantic Water Flow	Invert it: start from each ocean and walk uphill.
863	All Nodes Distance K	Add parent pointers → it is a graph → plain BFS.

> **Say out loud before coding, every time:** shortest path ⇒ BFS, always. Reachability, area, counting ⇒ DFS is fine. Choosing DFS for a shortest-path question is the single most common way to fail this round.

Day 2 · Wed 26 Aug

ROUND TYPE B — Reduce an O(n²) solution. *"Use a hash map / sliding window / two-pointer technique to reduce an O(n²) solution."* Note the phrasing: the interviewer **expects** you to state the brute force first, then improve it. That sequence is the answer, not just the final code.

#	Problem	What to watch
1	Two Sum	5-minute warm-up. Longer than that is a signal.
560	Subarray Sum Equals K ★	Prefix sum + hashmap seeded `{0:1}`. Negatives break sliding window — say so.
3	Longest Substring Without Repeating ★★	The archetype. Should be automatic by now.
424	Longest Repeating Character Replacement	Invalid when `len(window) - maxFreq > k`.
15	3Sum ★	Sort, fix one, two-pointer the rest. Dedupe at BOTH levels.
42	Trapping Rain Water ★★	Two-pointer and monotonic stack. Budget 45 min.
Stretch		
76	Minimum Window Substring ★★	have/need counters — the boss of this pattern.
239	Sliding Window Maximum ★	Monotonic deque of indices.

> **The script this round rewards:** "The brute force is O(n²) — check every pair. The repeated work is the inner scan, so a hashmap of what I have already seen gets me to O(n). Does that sound reasonable before I code it?" Rehearse those three sentences.

Day 3 · Thu 27 Aug

ROUND TYPE C — Intervals, heaps and top-K. *"Solve an interval, heap or top-K problem and explain complexity."* Amazon asks these constantly — Meeting Rooms and Merge Intervals are perennial repeats.

#	Problem	What to watch
56	Merge Intervals ★★	Sort by START. Use `max(end)` or contained intervals break it.
253	Meeting Rooms II ★★	Min-heap of end times, and the sweep-line version. Know both.
215	Kth Largest Element	Min-heap of size k. Yes — min-heap for *largest*.
973	K Closest Points to Origin ★	Skip the sqrt; it is monotonic. Say that.
23	Merge k Sorted Lists ★★	Heap of heads. Index tiebreaker, or Python raises TypeError.
621	Task Scheduler ★	`(maxFreq-1)*(n+1)+numMax`, clamped by `len(tasks)`.
Stretch		
347	Top K Frequent Elements	Bucket sort is the O(n) follow-up.
295	Find Median from Data Stream ★★	Two heaps, kept balanced.

> **Memorise the inversion:** "K largest" ⇒ **min**-heap of size K. "K smallest" ⇒ **max**-heap of size K. And for intervals: **sort by start to merge, sort by end to maximise how many fit.**

Day 4 · Fri 28 Aug

ROUND TYPE D — Justify the paradigm + design-in-code. *"Choose between BFS/DFS, DP, heap or binary search and justify the choice."* This round grades your **reasoning about the choice**, not just the implementation. Expect "why not X?" for every decision.

#	Problem	What to watch
875	Koko Eating Bananas ★	Binary search the ANSWER. Write `feasible()` on the board first.
33	Search in Rotated Sorted Array ★	One half is always sorted — identify which.
322	Coin Change ★	**Greedy is wrong.** Counterexample ready: coins [1,3,4], amount 6.
139	Word Break	DP over prefixes. Why not greedy? A longer match can dead-end.
146	LRU Cache ★★	Hashmap + doubly linked list. The node must store its own key.
138	Copy List with Random Pointer ★★	Hashmap first, then the O(1) interleave follow-up.
Stretch		
1011	Capacity To Ship Packages ★	Literally the same function as Split Array Largest Sum.
380	Insert Delete GetRandom O(1) ★	Array + index map, swap-with-last on delete.

> **The recognition rule for binary search on the answer:** the problem says *"minimum X such that…"* or *"maximum X such that…"*, and you can write an O(n) `feasible(x)` check. When you spot it, say the rule out loud — it is the fastest way to show you recognised the pattern rather than pattern-matched a template.

Day 5 · Sat 29 Aug — Revision + full simulation

Morning (2h 30m) — re-solve cold, blank page, timed. Afternoon (2h) — one full recorded simulation. Evening — logistics, stop by 8pm.

These are the ones Amazon actually repeats. If any takes over 25 minutes, mark it and drill the *four sentences* for it tonight — not the code.

Re-solve cold	Re-solve cold
200. Number of Islands	42. Trapping Rain Water
994. Rotting Oranges	560. Subarray Sum Equals K
236. LCA of a Binary Tree	56. Merge Intervals
127. Word Ladder	253. Meeting Rooms II
3. Longest Substring Without Repeating	23. Merge k Sorted Lists
146. LRU Cache	138. Copy List with Random Pointer

Afternoon — one full recorded simulation

- Pick **2 problems you have not solved** from the Amazon tag. Any medium.
- 60 minutes. Record on your phone. Narrate the entire time.
- Full sequence: clarify → small example → brute force and its complexity → state the optimal → **"does that sound reasonable before I code it?"** → code → dry-run out loud tracking variables → complexity → edge cases.
- Open with one Leadership Principle question, answered out loud, before any coding.
- **Watch the recording.** Note every silence over 20 seconds, and whether you skipped the dry run.

Evening — logistics, then stop by 8pm

- Print 4 resumes; photo ID out; clothes laid out; laptop and charger packed (LiveCode runs on your machine).
- **Confirm the exact building and suite with your recruiter.** Amazon has several Northern Virginia sites and people arrive at the wrong one.
- Two questions per interviewer ready.

The story block — 30 minutes a day

Five stories cover essentially everything you will be asked. Write one per day, about 250 words, in STAR+ form.

Day	Story	Prompt it answers	Leadership Principle
25 Aug	**Ownership**	*"a time you took ownership beyond your assigned responsibility"*	Ownership
26 Aug	**Failure**	*"a disagreement or failure and what you changed afterward"*	Earn Trust / Learn and Be Curious
27 Aug	**Incomplete information**	*"you had incomplete information but still had to deliver"*	Bias for Action
28 Aug	**Most impactful project**	asked in almost every loop	Deliver Results
29 Aug	**Tell me about yourself**	the first 90 seconds of every round	—

> **STAR+:** Situation (2 sentences) · Task (what **you** owned) · **Action (60% of the story)** · Result (**with a number**) · plus what you would do differently.
>
> **The two things that sink these:** saying *"we"* — an interviewer cannot score "we", so every one becomes *"I decided," "I wrote," "I convinced"* — and stories with no ending. "Then it got deprioritised" is a non-answer.
>
> **Rehearse each one out loud, timed. Over 2.5 minutes gets cut.**

The design block — 30 minutes a day, 3 topics only

Do each on **paper**, out loud — the in-person design round uses a whiteboard even though coding is typed.

Day	Topic	What to cover
26 Aug	**Rate limiter**	Token bucket vs sliding window vs fixed window. Where the counter lives in a distributed setting. **The question they will ask: what happens when the counter store is unavailable?** For a rate limiter, fail-*open* is usually right — the limiter must not become the thing that takes the service down.
27 Aug	**Cache**	Read-through vs write-through vs write-back. Eviction (LRU/LFU). TTL. Invalidation. Thundering herd on a cold key. This pairs with the LRU coding problem on 28 Aug — say so if it comes up.
28 Aug	**Notification / messaging service**	Fan-out, queues, retries with exponential backoff **plus jitter**, dead-letter queues, idempotency so a retry does not double-send, per-user preferences and rate limits.

> **The framework, every time:** functional requirements → scale numbers → API signatures → data model → boxes → **one failure mode** → trade-offs stated aloud. The failure-mode answer is what separates candidates at AWS; anyone can draw boxes.

30–31 Aug — travel. Not prep days.

- 45 minutes writing core patterns **by hand on paper**: binary search, sliding window, grid BFS, tree DFS, heap top-K.
- Rehearse your five stories in your head. No notes.
- **No LeetCode, no new problems.** A bad attempt the night before sits in your head and costs more than the practice gains.

- 31 Aug: physically find **12900 Worldgate Dr**. Eat properly, stop by 8pm, sleep 8 hours.

Part II — The 32 Problems

Every problem below is written in the same twelve-section order. That order *is* the answer — an Amazon interviewer scores the sequence you follow, not only the final code you type.

---

200. Number of Islands · Day 1
994. Rotting Oranges · Day 1
207. Course Schedule · Day 1
127. Word Ladder · Day 1
236. Lowest Common Ancestor of a Binary Tree · Day 1
102. Binary Tree Level Order Traversal · Day 1
417. Pacific Atlantic Water Flow · Day 1
863. All Nodes Distance K in Binary Tree · Day 1
1. Two Sum · Day 2
560. Subarray Sum Equals K · Day 2
3. Longest Substring Without Repeating Characters · Day 2
424. Longest Repeating Character Replacement · Day 2
15. 3Sum · Day 2
42. Trapping Rain Water · Day 2
76. Minimum Window Substring · Day 2
239. Sliding Window Maximum · Day 2
56. Merge Intervals · Day 3
253. Meeting Rooms II · Day 3
215. Kth Largest Element in an Array · Day 3
973. K Closest Points to Origin · Day 3
23. Merge k Sorted Lists · Day 3
621. Task Scheduler · Day 3
347. Top K Frequent Elements · Day 3
295. Find Median from Data Stream · Day 3
875. Koko Eating Bananas · Day 4
33. Search in Rotated Sorted Array · Day 4
322. Coin Change · Day 4
139. Word Break · Day 4
146. LRU Cache · Day 4
138. Copy List with Random Pointer · Day 4
1011. Capacity To Ship Packages Within D Days · Day 4
380. Insert Delete GetRandom O(1) · Day 4

200 Number of Islands ★★

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Grid flood fill (DFS / BFS) · Optimal: **O(m·n) time, O(m·n) space**

1 Clarification communication

1. Are the cells characters `'1'` / `'0'` or integers `1` / `0`? LeetCode gives characters, and comparing against the wrong type silently returns 0.
2. **May I modify the input grid?** This is the important one. If yes I mark visited cells in place and use O(1) extra space; if no I keep a separate `visited` set.
3. Is connectivity 4-directional or 8-directional (do diagonals connect)? The default is 4-directional.
4. Can the grid be empty, or a single row/column? I want to confirm the empty grid returns 0 rather than throwing.
5. How large can the grid get? If it can be very large I will prefer BFS over recursive DFS to avoid blowing the call stack.

2 Understanding of the question

I am given a 2-D grid where `'1'` is land and `'0'` is water. An island is a maximal group of land cells connected horizontally or vertically. I must return how many distinct islands there are.

Reframed as a graph problem: each land cell is a vertex, and there is an edge between two land cells that are vertically or horizontally adjacent. The question is then simply **how many connected components does this graph have**. Every counting-connected-components problem has the same shape: iterate over all vertices, and every time you find one you have not visited yet, increment the counter and exhaustively consume its whole component.

3 Brute-force approach in English

Without the idea of "consume the whole component at once", the natural first attempt is iterative label propagation. Give every land cell a unique label to begin with, then repeatedly sweep the entire grid, and on each sweep replace each land cell's label with the smallest label among itself and its four neighbours. Labels leak outward one cell per sweep, so connected cells slowly converge on a shared label. Keep sweeping until a full pass changes nothing, then count how many distinct labels survive.

It is correct because within one connected component the smallest label eventually reaches every cell, and a label can never cross water, so two different components can never merge.

4 Data structures used

- A second 2-D array `label` of the same dimensions, holding one integer per land cell.
- A boolean `changed` flag to know when the sweeps have stabilised.
- A set at the end, to count the distinct surviving labels.

5 Step-by-step algorithm

1. Allocate `label` with the same shape as the grid. Walk the grid once and give each land cell a unique id, for example `r * cols + c`. Water gets a sentinel such as `-1`.
2. Set `changed = True`.
3. While `changed` is True: set `changed = False`, then scan every cell of the grid.
4. For each land cell, look at its four neighbours. If any neighbour is land and carries a smaller label, copy that smaller label into this cell and set `changed = True`.
5. When a whole sweep completes with `changed` still False, the labels have stabilised.
6. Collect all labels of land cells into a set and return the size of that set.

6 Time & space complexity with explanation

Time: O((m·n)²) in the worst case. One sweep costs O(m·n) because it visits every cell and does constant work per cell. The number of sweeps needed is the length of the longest path a label must travel inside a component. A snake-shaped island that winds through the whole grid is roughly m·n cells long, so the smallest label needs on the order of m·n sweeps to reach the far end. Multiply the two: O((m·n)²).

Space: O(m·n) for the `label` grid plus the set of distinct labels at the end.

7 Fully commented brute-force code

```
def numIslands_brute(grid):
    """Label propagation. Correct, but re-sweeps the whole grid until labels settle."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])

    # Step 1: every land cell starts with its OWN unique label; water gets -1.
    label = [[-1] * cols for _ in range(rows)]
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                label[r][c] = r * cols + c        # unique id derived from position

    # Step 2: sweep until a full pass changes nothing.
    changed = True
    while changed:
        changed = False
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] != '1':
                    continue                      # water carries no label
                best = label[r][c]
                # Take the smallest label among me and my 4 neighbours.
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == '1':
                        best = min(best, label[nr][nc])
                if best != label[r][c]:
                    label[r][c] = best            # a smaller label leaked in
                    changed = True                # so another sweep is required

    # Step 3: one label survives per connected component.
    distinct = set()
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':
                distinct.add(label[r][c])
    return len(distinct)
```

8 Limitation of brute force

The waste is that the algorithm has no memory of what it already resolved. Each sweep re-examines every cell in the grid, including whole islands that stabilised many sweeps ago, just to discover that nothing changed. Information moves only one cell per pass, so a long winding island forces an enormous number of nearly-useless passes.

On a 1000 × 1000 grid that is up to 10^{12} cell visits, which will not finish. The fix is to stop letting information trickle: when I land on an unvisited land cell, I should walk its entire component immediately in one traversal, mark it, and never look at it again.

9 Optimal approach in English

Scan the grid once. The moment I hit a land cell that has not been visited, I know I have found a brand-new island that no previous scan has touched, because any earlier island would already have marked this cell. So I increment the counter by one and then flood fill outward from that cell, marking every land cell reachable from it as visited. The outer scan then continues and simply skips over everything the flood fill consumed.

The flood fill can be DFS (recursion, or an explicit stack) or BFS (a queue). Both visit exactly the same set of cells and cost the same. I lead with DFS because it is shorter to write, and I mention BFS as the version to use when the grid is large enough that recursion depth could overflow the stack.

10 Optimal data structures

- The **grid itself, mutated**, as the visited marker: setting a consumed cell to `'0'` means I need no extra structure at all. If the interviewer says the input is read-only, I swap in a `visited` set of `(r, c)` tuples, which costs O(m·n) space instead.
- For the DFS version, the **call stack**; for the BFS version, a **deque** used as a FIFO queue.
- A single integer counter.

11 Step-by-step optimal algorithm

1. Handle the empty grid up front and return 0.
2. Loop over every cell `(r, c)` of the grid.
3. If the cell is not `'1'`, skip it: it is either water or part of an island already counted.
4. Otherwise increment `count` by one, because this cell begins a new island.
5. Flood fill from `(r, c)`: visit the cell, immediately set it to `'0'` so it is never revisited, then recurse (or enqueue) into its four neighbours, ignoring anything out of bounds or not equal to `'1'`.
6. In the BFS variant, mark a neighbour as visited **at the moment you enqueue it**, not when you dequeue it. Marking on dequeue lets the same cell be pushed several times before it is first popped.
7. When the flood fill returns, the whole island has been erased. Continue the outer scan.
8. Return `count`.

12 Fully commented optimal code

```
def numIslands(grid):
    """DFS flood fill. Each land cell is visited once and then erased."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])

    def sink(r, c):
        # Guard first: off the grid, or water / already-consumed land -> stop.
        if not (0 <= r < rows and 0 <= c < cols) or grid[r][c] != '1':
            return
        grid[r][c] = '0'                  # mark visited by mutating the input
        sink(r + 1, c)                    # down
        sink(r - 1, c)                    # up
        sink(r, c + 1)                    # right
        sink(r, c - 1)                    # left

    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == '1':         # unvisited land => a brand-new island
                count += 1
                sink(r, c)                # consume the entire island now
    return count


from collections import deque

def numIslandsBFS(grid):
    """BFS flood fill. Same complexity; use it when the grid is huge, because
    it keeps the frontier on the heap instead of on the call stack."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    count = 0
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] != '1':
                continue
            count += 1
            q = deque([(r, c)])
            grid[r][c] = '0'              # mark the seed immediately
            while q:
                x, y = q.popleft()
                for dx, dy in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nx, ny = x + dx, y + dy
                    if 0 <= nx < rows and 0 <= ny < cols and grid[nx][ny] == '1':
                        grid[nx][ny] = '0'   # *** mark ON ENQUEUE, not on dequeue ***
                        q.append((nx, ny))   # otherwise duplicates pile up in q
    return count
```

+ **Optimal complexity, and what to say out loud**

Time: O(m·n). The outer loop touches each cell once. Across all flood fills each land cell is pushed and popped at most once, because it is set to `'0'` the first time it is reached. Every cell therefore does O(1) work in total.

Space: O(m·n) worst case. For DFS that is the recursion depth when the whole grid is one snaking island; for BFS it is the maximum queue size. If I may not mutate the grid, add O(m·n) for the `visited` set.

Say this out loud: "This is counting connected components. My grid doubles as the visited marker. I erase each island as I count it, because that guarantees I never count the same component twice. Each cell is visited once, so O(m·n) time."

The one gotcha: in BFS, mark on enqueue. **The follow-up:** union-find also solves it in near-O(m·n), and is the right answer if the grid streams in cell by cell, or if the follow-up is Number of Islands II (islands added one at a time).

994 Rotting Oranges ★★ Amazon favourite

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Multi-source BFS, level by level · Optimal: **O(m·n) time, O(m·n) space**

1 Clarification communication

1. To confirm the encoding: `0` is empty, `1` is fresh, `2` is rotten.
2. Rot spreads to the four orthogonal neighbours only, not diagonally, and it spreads to **all** neighbours simultaneously each minute?
3. If there are no fresh oranges at all, should I return 0? I want to confirm it is 0 and not −1, because that edge case is what most wrong submissions get wrong.
4. If some fresh orange is unreachable, I return −1 for the whole grid?
5. May I modify the grid in place to mark newly rotten cells?

2 Understanding of the question

Every minute, any fresh orange orthogonally adjacent to a rotten one becomes rotten. I must return the number of minutes until no fresh orange remains, or −1 if that never happens.

The key reframing: this is a **shortest-path** question in disguise. The answer is the maximum, over all fresh oranges, of the distance from that orange to its nearest rotten orange. Shortest path on an unweighted grid means BFS. The twist is that there are many starting points, not one, so it is a **multi-source** BFS: I put every rotten orange into the queue before the search begins, which is exactly equivalent to adding a virtual super-source connected to all of them at distance zero. Each BFS level is then one minute of simulated time.

3 Brute-force approach in English

Simulate the clock literally. For each minute, scan the entire grid and build a list of every fresh orange that has a rotten neighbour right now. After the scan finishes, turn all of those to rotten together. Repeat, counting minutes, until a full scan finds nothing new to rot. Then scan once more: if any fresh orange remains it is unreachable, so return −1; otherwise return the minute count.

The one detail that must be right is that the newly-rotten cells are collected first and committed afterwards. If I flip cells to rotten during the scan, the rot can cascade several steps within a single minute and the answer comes out too small.

4 Data structures used

- A list `to_rot` holding the coordinates that will turn rotten at the end of the current minute.
- The grid itself, mutated after each minute.
- An integer minute counter.

5 Step-by-step algorithm

1. Set `minutes = 0`.
2. Loop forever: clear `to_rot`, then scan every cell of the grid.
3. For each cell holding a `1`, check its four neighbours. If any neighbour holds a `2`, append this cell's coordinates to `to_rot`.
4. If `to_rot` is empty, the process has stalled: break out of the loop.
5. Otherwise set every coordinate in `to_rot` to `2` and increment `minutes` by one. Committing after the scan is what keeps the rot to exactly one step per minute.
6. After the loop, scan the grid one final time. If any `1` remains, return −1.
7. Otherwise return `minutes`. Note that a grid with no fresh oranges never enters a productive iteration, so it naturally returns 0.

6 Time & space complexity with explanation

Time: O((m·n)²) in the worst case. One minute of simulation costs a full grid scan, O(m·n). The number of minutes is the largest distance from a fresh orange to the nearest rotten one, which for a snaking corridor of oranges is on the order of m·n. Multiplying gives O((m·n)²).

Space: O(m·n) for the `to_rot` list, which in the first minute can hold a constant fraction of the grid.

7 Fully commented brute-force code

```
def orangesRotting_brute(grid):
    """Literal minute-by-minute simulation with a full grid scan per minute."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])
    minutes = 0

    while True:
        to_rot = []
        # Scan the WHOLE grid to find what rots this minute.
        for r in range(rows):
            for c in range(cols):
                if grid[r][c] != 1:
                    continue
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nr, nc = r + dr, c + dc
                    if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 2:
                        to_rot.append((r, c))
                        break
        if not to_rot:
            break                           # nothing changed -> the process stalled

        # *** Commit AFTER the scan. Flipping cells during the scan would let
        # the rot cascade several steps inside a single minute. ***
        for r, c in to_rot:
            grid[r][c] = 2
        minutes += 1

    # Anything still fresh was unreachable.
    for row in grid:
        if 1 in row:
            return -1
    return minutes
```

8 Limitation of brute force

Every minute I rescan the entire grid even though only the cells next to last minute's newly-rotten oranges can possibly change. In a 1000 × 1000 grid with a long corridor, that is a million cells examined per minute for perhaps a million minutes.

The realisation is that I already know exactly which cells rotted last minute; those are the only useful frontier. Keeping that frontier explicitly in a queue removes the scan entirely, and a queue processed one level at a time is just BFS.

9 Optimal approach in English

Do a single BFS whose queue is seeded with **every** rotten orange at once, and count the fresh oranges while seeding. Then process the queue one full level at a time: pop exactly the number of items currently in the queue, rot each of their fresh neighbours, push those neighbours, and decrement the fresh counter for each. One completed level equals one minute.

Because BFS expands in order of distance, the level at which a fresh orange is reached is precisely its distance to the nearest rotten orange, which is when it actually rots. When the queue empties, if the fresh counter is zero every orange was reached and the level count is the answer; if it is positive, those oranges are in a region no rot can enter, so I return −1.

10 Optimal data structures

- A **deque** as a FIFO queue holding the coordinates of the current rot frontier.
- An integer `fresh` counting remaining fresh oranges, which gives an O(1) termination test and removes the need for a final full scan.
- The grid itself, mutated to `2`, doubling as the visited marker.

11 Step-by-step optimal algorithm

1. Scan the grid once. Push every cell containing `2` into the queue, and count every cell containing `1` into `fresh`.
2. If `fresh == 0`, return 0 immediately. There is nothing to rot, so the answer is zero minutes, never −1.
3. Set `minutes = 0`.
4. While the queue is non-empty **and** `fresh > 0`: increment `minutes`, then snapshot `len(q)` and pop exactly that many cells, which is precisely the previous minute's frontier.
5. For each popped cell, look at its four neighbours. For a neighbour that is in bounds and equal to `1`, set it to `2`, decrement `fresh`, and push it.
6. The `and fresh` in the loop condition matters: after the last orange rots the queue still holds that final frontier, and looping again would increment `minutes` one time too many for a minute in which nothing happens.
7. After the loop, return `minutes` if `fresh == 0`, otherwise −1.

12 Fully commented optimal code

```
from collections import deque

def orangesRotting(grid):
    """Multi-source BFS. One BFS level == one minute of wall-clock time."""
    if not grid or not grid[0]:
        return 0
    rows, cols = len(grid), len(grid[0])

    q = deque()
    fresh = 0
    # Seed the queue with EVERY rotten orange before the search starts.
    # This is equivalent to a virtual super-source at distance 0 from them all.
    for r in range(rows):
        for c in range(cols):
            if grid[r][c] == 2:
                q.append((r, c))
            elif grid[r][c] == 1:
                fresh += 1

    if fresh == 0:
        return 0          # *** nothing to rot -> 0 minutes, NOT -1 ***

    minutes = 0
    # "and fresh" stops us counting one extra minute after the last orange rots.
    while q and fresh:
        minutes += 1
        for _ in range(len(q)):        # snapshot the level size FIRST
            r, c = q.popleft()
            for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                nr, nc = r + dr, c + dc
                if 0 <= nr < rows and 0 <= nc < cols and grid[nr][nc] == 1:
                    grid[nr][nc] = 2   # rot it now == mark visited on enqueue
                    fresh -= 1
                    q.append((nr, nc))

    # Anything still fresh sits in a region the rot could never reach.
    return minutes if fresh == 0 else -1
```

+ **Optimal complexity, and what to say out loud**

Time: O(m·n). The seeding scan is O(m·n). Each cell is enqueued at most once, because it is flipped to `2` the instant it is queued, and each dequeue does constant work over four neighbours.

Space: O(m·n). In the worst case the initial frontier is most of the grid.

Say this out loud: "Shortest time to reach every cell from any of several starting points is multi-source BFS. I seed all rotten oranges before starting, and I process the queue one level at a time so a level equals a minute. Each cell enters the queue once, so O(m·n)."

The two gotchas that decide this problem: the `fresh == 0` early return, which must be 0 and not −1; and `while q and fresh`, which prevents an off-by-one extra minute. Both are worth stating before you write them.

207 Course Schedule ★

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Cycle detection / topological sort (Kahn) · Optimal: **O(V + E) time, O(V + E) space**

1 Clarification communication

1. To pin the edge direction: does `[a, b]` mean "to take course a you must first take b"? Then the dependency edge runs **b → a**. Getting this backwards is the single most common bug here, so I want to confirm it explicitly.
2. Can the prerequisite list contain duplicate pairs, or a self-loop like `[1, 1]`? A self-loop is an immediate cycle.
3. Is the graph guaranteed connected? I will assume not, and handle a forest.
4. Do I only need the boolean, or will you also want the actual ordering? Kahn's algorithm gives me the order for free, so I will build it in a way that extends to Course Schedule II.

2 Understanding of the question

There are `numCourses` courses labelled `0` to `numCourses - 1`, and a list of prerequisite pairs. I must decide whether it is possible to finish all courses.

Modelled as a directed graph with courses as vertices and "must come before" as edges, the schedule is feasible exactly when the graph is a **DAG** — it has no directed cycle. A cycle means a set of courses each waiting on another, which can never start. So the entire question is: **does this directed graph contain a cycle?**

3 Brute-force approach in English

Check each course independently by brute-force path exploration. For a given course, walk every directed path leaving it, following prerequisites forward, and see whether any path ever returns to that same course. If it does, that course sits on a cycle and the answer is False. Repeat for all courses; if no course is ever reachable from itself, the answer is True.

Crucially this version keeps no memoisation across explorations and no global visited set. It only tracks the nodes on the current path, which is enough to avoid looping forever, but means the same subgraph is re-explored along every path that reaches it.

4 Data structures used

- An **adjacency list**, a dict mapping each course to the list of courses that depend on it.
- A `path` set holding the nodes on the current recursion stack, used to stop infinite descent.
- The call stack for the recursion.

5 Step-by-step algorithm

1. Build the adjacency list: for each pair `[course, prereq]`, append `course` to `adj[prereq]`, giving the edge `prereq → course`.
2. Define `reaches(node, start, path)` returning True if a directed path from `node` gets back to `start`.
3. Inside it, iterate over every neighbour of `node`. If the neighbour equals `start`, a cycle through `start` exists, so return True.
4. If the neighbour is already in `path`, skip it: continuing would loop forever, and any cycle it belongs to will be discovered when that node is itself used as `start`.
5. Otherwise add the neighbour to `path`, recurse into it, and remove it again on the way out so other branches can still explore it.
6. Call `reaches(c, c, {c})` for every course `c`. If any call returns True, return False from the outer function.
7. If no course reaches itself, return True.

6 Time & space complexity with explanation

Time: O(V · (V + E)) at best and exponential in the worst case. Each of the V starting courses launches its own exploration, and because there is no memoisation across branches, a graph shaped like a chain of diamonds forces the same downstream subgraph to be re-walked once per distinct path reaching it, and the number of distinct paths can double with each diamond. That is the exponential blow-up.

Space: O(V + E) for the adjacency list, plus O(V) for the recursion depth and the `path` set.

7 Fully commented brute-force code

```
from collections import defaultdict

def canFinish_brute(numCourses, prerequisites):
    """For every course, ask 'can I get back to myself?'. No memoisation at all,
    so shared subgraphs are re-explored along every path that reaches them."""
    adj = defaultdict(list)
    for course, prereq in prerequisites:
        adj[prereq].append(course)        # edge prereq -> course

    def reaches(node, start, path):
        """True if some directed path from `node` returns to `start`."""
        for nxt in adj[node]:
            if nxt == start:
                return True               # closed the loop back to start
            if nxt in path:
                continue                  # already on this path; do not recurse forever
            path.add(nxt)
            if reaches(nxt, start, path):
                return True
            path.remove(nxt)              # backtrack: other branches may still use it
        return False

    for c in range(numCourses):
        if reaches(c, c, {c}):            # c lies on a cycle
            return False
    return True
```

8 Limitation of brute force

Two separate wastes. First, the answer is a single global property — "is there a cycle anywhere?" — yet I ask it V times, once per course. Second, within a single exploration nothing is remembered, so a vertex reachable by many different paths is expanded many times, which is where the exponential behaviour comes from.

The insight that fixes both: a course is takeable as soon as all of its prerequisites are done. If I peel off courses that currently have zero outstanding prerequisites and remove their outgoing edges, a DAG collapses completely. Anything left over is, by definition, waiting on itself — a cycle. That turns V explorations into one linear pass.

9 Optimal approach in English

Use Kahn's algorithm, which is topological sort by indegree peeling. The indegree of a course is the number of prerequisites it is still waiting on. Any course with indegree 0 can be taken right now, so seed a queue with all of them. Pop a course, count it as completed, and for every course that depended on it decrement that course's indegree; if a decrement brings an indegree to 0, that course has just become takeable, so push it.

When the queue empties, compare the number of completed courses to `numCourses`. If they match, every course was scheduled and the graph is a DAG. If some courses are left over, none of them ever reached indegree 0, which means each is waiting on another leftover: that is exactly a cycle.

As an alternative I would mention DFS with three colours — white unvisited, grey on the current stack, black fully finished — where meeting a grey node means a back edge and therefore a cycle. Same complexity; Kahn is easier to explain and gives the ordering directly for the Course Schedule II follow-up.

10 Optimal data structures

- An **adjacency list** `adj`, mapping prerequisite → list of dependent courses.
- An **indegree array** of length `numCourses`, the count of unmet prerequisites per course. This is the structure that replaces all the repeated searching.
- A **deque** as a FIFO queue of currently-takeable courses. A stack would work equally well here, since any topological order is acceptable.
- An integer counter of completed courses.

11 Step-by-step optimal algorithm

1. Create `adj` as an empty adjacency list and `indeg` as an array of `numCourses` zeros.
2. For each pair `[course, prereq]`: append `course` to `adj[prereq]`, and increment `indeg[course]` by one.
3. Initialise the queue with every course `i` whose `indeg[i]` is 0. These need no prerequisites and can be taken immediately.
4. Set `done = 0`.
5. While the queue is non-empty: pop a course, increment `done`.
6. For each dependent `nei` in `adj[course]`, decrement `indeg[nei]`. If it reaches 0, all of that course's prerequisites are now satisfied, so push it.
7. After the loop, return `done == numCourses`.
8. For Course Schedule II, append each popped course to an output list; that list is a valid ordering, and if `done < numCourses` return an empty list instead.

12 Fully commented optimal code

```
from collections import defaultdict, deque

def canFinish(numCourses, prerequisites):
    """Kahn's topological sort. If we cannot schedule every course, the leftovers
    form a cycle."""
    adj = defaultdict(list)
    indeg = [0] * numCourses

    for course, prereq in prerequisites:
        adj[prereq].append(course)        # *** direction: prereq -> course ***
        indeg[course] += 1                # `course` waits on one more thing

    # Everything with no outstanding prerequisites can be taken right now.
    q = deque(i for i in range(numCourses) if indeg[i] == 0)

    done = 0
    while q:
        node = q.popleft()
        done += 1                         # this course is now completed
        for nei in adj[node]:
            indeg[nei] -= 1               # one prerequisite satisfied
            if indeg[nei] == 0:           # all satisfied -> it just became takeable
                q.append(nei)

    # Leftover courses never hit indegree 0 => each waits on another leftover => cycle.
    return done == numCourses
```

+ Optimal complexity, and what to say out loud

Time: O(V + E). Building the graph touches every edge once. Each vertex is enqueued and dequeued at most once, and each edge is relaxed exactly once, when its source vertex is popped.

Space: O(V + E) for the adjacency list and indegree array, plus O(V) for the queue.

Say this out loud: "Feasibility of a dependency schedule is the same as asking whether the directed graph is acyclic. My indegree array holds how many prerequisites each course is still waiting on. I pop courses whose count has reached zero, because those are exactly the ones that are now takeable. Every vertex and every edge is handled once, so O(V + E)."

The gotcha: the edge direction. `[a, b]` means b before a, so the edge is b → a and the indegree belongs to a. Say this while you type it.

127 Word Ladder ★★ Amazon favourite

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: BFS on an implicit graph · Optimal: **O(N · L · 26) time, O(N · L) space**

1 Clarification communication

1. Do all words have the same length? And are they lowercase English letters only? That decides whether generating neighbours over a 26-letter alphabet is valid.
2. Does the returned length count the words in the sequence including `beginWord` and `endWord`? So a direct one-letter change gives 2, not 1.
3. Does `beginWord` need to be in `wordList`? Standard statement says no, but `endWord` must be, otherwise the answer is 0.
4. Roughly how many words and how long are they? If N is large and L is small, generating neighbours beats comparing pairs; if L were enormous the trade-off flips.
5. Do you want just the length, or the actual sequence? The sequence variant (Word Ladder II) needs parent tracking, so I want to know before I choose the structure.

2 Understanding of the question

I am given a start word, an end word, and a dictionary. Each step may change exactly one letter, and every intermediate word must exist in the dictionary. I must return the number of words in the shortest such sequence, or 0 if none exists.

This is a graph problem where the vertices are dictionary words and an edge joins two words differing in exactly one position. The graph is **implicit**: nobody hands me the adjacency list, I have to produce neighbours on demand. And because every edge costs the same one step, shortest path on an unweighted graph means **BFS** — not DFS, and not Dijkstra. The whole design problem is how to produce a word's neighbours cheaply.

3 Brute-force approach in English

Explore transformation sequences exhaustively with DFS. From the current word, try every dictionary word that differs from it in exactly one position and has not been used in the current chain, recurse into it, and when the end word is reached record the chain length. Take the minimum over every chain found.

This is correct because it literally enumerates every valid ladder, but it explores whole families of long detours before it ever finds a short route, and it can revisit the same word through a different prefix over and over.

4 Data structures used

- The word list, scanned linearly each time neighbours are needed.
- A `used` set holding the words on the current chain, to keep the recursion finite.
- The call stack, and a running best-length integer.

5 Step-by-step algorithm

1. Write a helper `differs_by_one(a, b)` that walks both words in parallel, counts positions where the characters differ, and returns True only if the count is exactly 1. This is O(L).
2. Define `dfs(word, depth)`. If `word == endWord`, update the global best with `depth` and return.
3. Otherwise loop over every word `w` in the dictionary.
4. Skip `w` if it is already in `used`, or if `differs_by_one(word, w)` is False.
5. Add `w` to `used`, call `dfs(w, depth + 1)`, then remove `w` from `used` so a different branch can still use it.
6. Start with `dfs(beginWord, 1)` and `used = {beginWord}`.
7. Return the best length found, or 0 if the end word was never reached.

6 Time & space complexity with explanation

Time: O(N! · L) in the worst case. At the first level up to N words are candidates, at the second up to N−1, and so on, because backtracking removes words from `used` again. Each candidacy test costs O(L). In a densely connected dictionary this is factorial behaviour.

Space: O(N · L) — recursion depth up to N frames, each holding a word.

7 Fully commented brute-force code

```
def ladderLength_brute(beginWord, endWord, wordList):
    """Exhaustive DFS over every valid ladder. Correct, catastrophically slow."""
    words = list(wordList)
    if endWord not in words:
        return 0

    def differs_by_one(a, b):
        """True iff a and b differ in exactly one position. O(L)."""
        diff = 0
        for x, y in zip(a, b):
            if x != y:
                diff += 1
                if diff > 1:
                    return False
        return diff == 1

    best = [float('inf')]
    used = {beginWord}

    def dfs(word, depth):
        if word == endWord:
            best[0] = min(best[0], depth)
            return
        if depth >= best[0]:
            return                          # this branch can no longer win
        for w in words:                     # O(N) scan, O(L) test => O(N*L) per node
            if w in used or not differs_by_one(word, w):
                continue
            used.add(w)
            dfs(w, depth + 1)
            used.remove(w)                  # backtrack -> the same word is reusable

    dfs(beginWord, 1)
    return 0 if best[0] == float('inf') else best[0]
```

8 Limitation of brute force

Two independent problems. First, **DFS is the wrong search order for a shortest-path question**: it dives down one arbitrary branch to the end before considering the alternative one step away, so it must explore essentially everything before it can be sure it found the minimum. BFS finds the shortest path the first time it touches the target, and can stop.

Second, **finding neighbours by scanning the dictionary is O(N·L) per word**. There are only L × 26 possible one-letter variants of a word regardless of how big the dictionary is, so I can generate those directly and test each for membership in a set in O(1). For a large dictionary of short words that is dramatically cheaper.

9 Optimal approach in English

Run BFS from `beginWord`, with the queue holding `(word, steps_so_far)`. Because BFS visits vertices in nondecreasing distance order, the first time `endWord` is popped, the recorded step count is the shortest possible, and I return immediately.

To generate the neighbours of a word without touching the dictionary, I take each of its L positions in turn and substitute each of the 26 lowercase letters, giving L × 26 candidate strings. A candidate is a real neighbour precisely when it is in the dictionary **set**, which is an O(L) hash lookup rather than an O(N·L) scan.

The visited set must be updated **when a word is enqueued**, not when it is dequeued. If I wait until dequeue, the same word can be pushed by several nodes on the same level and gets expanded several times.

10 Optimal data structures

- A **hash set** of the dictionary, converting the neighbour test from an O(N·L) scan into an O(L) lookup. This is the single change that makes the algorithm fast.
- A **deque** as a FIFO queue of `(word, steps)` pairs. Storing the step count in the queue avoids needing a separate distance map.
- A `seen` **set** of already-enqueued words, which prevents both cycles and duplicate work.

11 Step-by-step optimal algorithm

1. Convert `wordList` to a set. If `endWord` is not in it, return 0 straight away, since no valid ladder can end there.
2. Initialise the queue with `(beginWord, 1)` — 1 because the count includes the start word — and `seen = {beginWord}`.
3. While the queue is non-empty, pop `(word, steps)` from the front.
4. If `word == endWord`, return `steps`. BFS guarantees this is the minimum.
5. Otherwise, for each index `i` in the word and each letter `c` from `'a'` to `'z'`, form `cand = word[:i] + c + word[i+1:]`.
6. If `cand` is in the dictionary set and not in `seen`, add it to `seen` **now**, and push `(cand, steps + 1)`.
7. If the queue drains without finding `endWord`, return 0: no ladder exists.

12 Fully commented optimal code

```
from collections import deque

def ladderLength(beginWord, endWord, wordList):
    """BFS over an implicit graph. Neighbours are GENERATED (L*26 of them),
    not found by scanning the dictionary."""
    words = set(wordList)                # O(1) membership instead of an O(N) scan
    if endWord not in words:
        return 0                         # unreachable by definition

    q = deque([(beginWord, 1)])          # 1: the count includes beginWord itself
    seen = {beginWord}

    while q:
        word, steps = q.popleft()
        if word == endWord:
            return steps                 # BFS => the first arrival is the shortest

        # Generate every one-letter mutation: L positions x 26 letters.
        for i in range(len(word)):
            for c in 'abcdefghijklmnopqrstuvwxyz':
                cand = word[:i] + c + word[i + 1:]
                if cand in words and cand not in seen:
                    seen.add(cand)       # *** mark ON ENQUEUE ***
                    q.append((cand, steps + 1))

    return 0                             # queue drained, endWord never reached
```

+ **Optimal complexity, and what to say out loud**

Time: O(N · L²), often written O(N · L · 26). Each of the N words is dequeued at most once. For each, I build L × 26 candidates, and building one candidate string plus hashing it is O(L). So per word it is O(26 · L · L), and 26 is a constant.

Space: O(N · L) for the dictionary set, the seen set and the queue.

Say this out loud: "Shortest transformation means unweighted shortest path, so BFS, not DFS. The graph is implicit, so my real design decision is how to enumerate neighbours: L × 26 generated candidates tested against a hash set, rather than comparing against all N words."

The follow-up they want: bidirectional BFS. Search forward from `beginWord` and backward from `endWord` at the same time, always expanding the smaller frontier, and stop when the two frontiers touch. Since a frontier grows roughly like b^d, two searches of depth d/2 cost about 2·b^(d/2) instead of b^d — a large constant-factor win. Mentioning this unprompted is what distinguishes a strong answer here.

236 Lowest Common Ancestor of a Binary Tree ★★ Amazon favourite

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Post-order DFS returning information upward · Optimal: **O(n) time, O(h) space**

1 Clarification communication

1. Are `p` and `q` **guaranteed to exist** in the tree? The standard problem guarantees it, and my clean solution relies on that guarantee. If they might be absent I need a different return protocol, which I will describe.
2. Is a node allowed to be its own ancestor? The usual definition says yes, so if `p` is an ancestor of `q` the answer is `p`.
3. Are the node values unique, and am I comparing by identity or by value? I will compare by identity with `is`, which is safe even with duplicate values.
4. Is this a plain binary tree or a BST? A BST admits a simpler O(h) solution using value comparisons; I am assuming a plain binary tree.
5. Do the nodes have parent pointers? If they did, this reduces to finding the intersection of two linked lists.

2 Understanding of the question

Given the root of a binary tree and two nodes `p` and `q`, I must return their lowest common ancestor: the deepest node that has both `p` and `q` somewhere in its subtree, where a node counts as a descendant of itself.

The structural fact that drives everything: for the LCA node, `p` and `q` lie in **different** subtrees — one on the left, one on the right — unless one of them *is* the LCA. Above the LCA both targets are on the same side; below it, at most one target is present. So the LCA is exactly the unique node where the two targets "split". That makes it a bottom-up question, which means post-order DFS: solve both children first, then decide about the current node.

3 Brute-force approach in English

Directly encode the definition. Write a helper `contains(node, target)` that reports whether a target lives anywhere in a given subtree. Then walk every node of the tree and ask: does my left subtree, my right subtree, or I myself hold both `p` and `q`? Every node that contains both is a common ancestor. Among all of them, the answer is the deepest, so track depth and keep the deepest one seen.

It is obviously correct because it is a literal translation of the definition, but the `contains` calls repeat enormous amounts of work.

4 Data structures used

- The call stack for two nested recursions, the outer walk and the inner containment check.
- Two variables holding the best node found so far and its depth.

5 Step-by-step algorithm

1. Write `contains(node, target)`: return False for a null node, True if `node is target`, otherwise the OR of the recursive calls on the two children. This is O(n).
2. Write `walk(node, depth)` that visits every node in the tree.
3. At each node, evaluate `contains(node, p) and contains(node, q)`.
4. If both are true, this node is a common ancestor; if its `depth` exceeds the best depth seen so far, record it as the new answer.
5. Recurse into both children with `depth + 1`.
6. Call `walk(root, 0)` and return the recorded node.

6 Time & space complexity with explanation

Time: O(n²). The outer walk visits n nodes, and at each one the two `contains` calls each cost O(size of that subtree), which is O(n) near the root. Summed over a balanced tree this is O(n log n); over a degenerate, list-shaped tree it is $O(n^2)$, since the subtree sizes are n, n−1, n−2 and so on.

Space: O(h) for the recursion, where h is the height — O(log n) balanced, O(n) degenerate.

7 Fully commented brute-force code

```
def lowestCommonAncestor_brute(root, p, q):
    """Literal definition: find every node containing both targets, keep the deepest."""

    def contains(node, target):
        """Does `target` appear anywhere in this subtree? O(size of subtree)."""
        if not node:
            return False
        if node is target:
            return True
        return contains(node.left, target) or contains(node.right, target)

    best = [None, -1]                    # [node, depth of that node]

    def walk(node, depth):
        if not node:
            return
        # Two full subtree scans at EVERY node -- this is the quadratic part.
        if contains(node, p) and contains(node, q):
            if depth > best[1]:          # deeper common ancestor wins
                best[0], best[1] = node, depth
        walk(node.left, depth + 1)
        walk(node.right, depth + 1)

    walk(root, 0)
    return best[0]
```

8 Limitation of brute force

Every `contains` call re-descends a subtree that the outer walk is about to descend anyway, and a node near the root is scanned once for itself and again as part of each of its ancestors' checks. The same subtree is traversed over and over to answer the same question.

The realisation: a single post-order traversal can compute, for every node at once, whether its subtree contains a target — because that answer is just the OR of what its children report plus whether the node is itself a target. If each recursive call *returns* that information upward, one traversal replaces all of the nested scans.

9 Optimal approach in English

One post-order DFS in which each call returns a single node, under this contract: *return the LCA if it is inside this subtree; otherwise return `p` or `q` if exactly one of them is here; otherwise return null.*

The base case does the finding: if the current node is null, or is `p`, or is `q`, return it as is. Otherwise recurse into both children and inspect the two results. If **both** are non-null, then one target was found on the left and the other on the right, so the two targets split exactly here and the current node is the LCA. If only one side is non-null, both targets live on that side (or only one target exists at all), so I pass that result straight up. If both are null, this subtree is irrelevant and I return null.

The elegant part is that the case "`p` is an ancestor of `q`" needs no special handling: the recursion stops at `p` and returns it, and since `q` lies below `p` no other branch returns anything, so `p` propagates to the top — which is the correct answer.

10 Optimal data structures

- No auxiliary data structure at all. The **call stack** carries the state, and the **return value** is the channel that carries information from the leaves back to the root.
- That is the whole point of the optimisation: replacing an explicit search with a well-chosen return contract.

11 Step-by-step optimal algorithm

1. If `root` is null, return null — nothing here.
2. If `root is p` or `root is q`, return `root`. Do not keep searching below: if the other target is beneath this one, this node is already the answer.
3. Recurse on the left child, storing the result in `left`.
4. Recurse on the right child, storing the result in `right`.
5. If `left` and `right` are both non-null, the two targets were found on opposite sides, so `root` is the LCA. Return `root`.
6. Otherwise return whichever of `left` and `right` is non-null, or null if neither is. In Python that is exactly `left or right`.
7. Note the order: the two recursive calls happen before any decision. That is what makes it post-order, and post-order is what lets information flow upward.

12 Fully commented optimal code

```
def lowestCommonAncestor(root, p, q):
    """Post-order DFS. Contract for the return value:
         - the LCA, if it is inside this subtree; else
         - p or q, if exactly one of them is in this subtree; else
         - None.
    """
    # Base case: nothing here, or we just landed on one of the targets.
    # Returning immediately also handles 'p is an ancestor of q' for free.
    if not root or root is p or root is q:
        return root

    left = lowestCommonAncestor(root.left, p, q)
    right = lowestCommonAncestor(root.right, p, q)

    if left and right:
        return root       # p and q were found on OPPOSITE sides -> they split here

    # Exactly one side (or neither) reported something: pass it straight up.
    return left or right
```

+ Optimal complexity, and what to say out loud

Time: O(n). Each node is visited exactly once and does O(1) work beyond its two recursive calls.

Space: O(h) for the recursion stack — O(log n) for a balanced tree, O(n) for a degenerate one. There is no auxiliary structure.

Say this out loud: "This is post-order DFS with information flowing upward. Each call returns the LCA if it is in that subtree, otherwise whichever target it found. When both children return something non-null, the targets split at me, so I am the LCA. Each node is visited once, so O(n) time and O(h) stack."

The follow-up to have ready: *what if `p` or `q` might not be in the tree?* This version would wrongly return `p` when `q` is absent, because it cannot distinguish "found both" from "found one". The fix is to keep two boolean flags set at the moment each target is matched, run the same recursion to completion without early-returning, and only accept the computed node if both flags ended up true.

102 Binary Tree Level Order Traversal

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: BFS with a level-size snapshot · Optimal: **O(n) time, O(w) space**

1 Clarification communication

1. Should the output be a list of lists, one inner list per level, top to bottom? And an empty tree returns `[]` rather than `[[]]`?
2. Within a level, must the order be strictly left to right? That determines that I need a FIFO queue rather than a stack.
3. Should I output node values or node references? Values, I assume.
4. Can the tree be very deep or very wide? That affects whether recursion is safe, and it decides the real space bound.

2 Understanding of the question

Given the root of a binary tree, return its node values level by level: the root's level first, then its children, and so on, with each level as its own list and each level ordered left to right.

Visiting nodes in nondecreasing depth order is exactly what BFS does. The only extra requirement beyond ordinary BFS is knowing **where one level ends and the next begins**, so that the output can be grouped. That single requirement is the whole problem, and the technique that solves it — snapshotting the queue length before draining a level — is reused in Rotting Oranges, All Nodes Distance K, zigzag traversal, right-side view, and every other level-aware BFS.

3 Brute-force approach in English

Attack one level at a time, independently. First compute the height of the tree. Then, for each depth `d` from 0 up to height minus 1, traverse the **entire tree from the root** and collect the values of nodes that happen to sit at depth `d`, appending the resulting list to the output.

It is correct because a DFS that carries a depth counter and appends only when the counter equals `d` visits the level's nodes left to right. But it re-walks the whole tree once per level.

4 Data structures used

- The call stack, for the height computation and for one full traversal per level.
- One list per level, plus the output list of lists.

5 Step-by-step algorithm

1. Compute the height: an empty node has height 0, otherwise it is 1 plus the larger of the two children's heights.
2. For `d` from 0 to `height - 1`, create an empty `level` list.
3. Run a full DFS from the root carrying the current depth.
4. If the current depth equals `d`, append this node's value to `level` and stop descending, since nothing below can be at depth `d`.
5. Otherwise recurse left then right, in that order, so the level comes out left to right.
6. Append `level` to the output.
7. Return the output.

6 Time & space complexity with explanation

Time: O(n · h). Each of the h levels triggers a traversal that can touch up to n nodes. For a balanced tree h is log n, giving O(n log n); for a degenerate list-shaped tree h is n, giving **O(n²)**.

Space: O(h) for the recursion, plus O(n) for the output, which is required by the problem and is not really overhead.

7 Fully commented brute-force code

```
def levelOrder_brute(root):
    """One full traversal per level. Correct, but re-walks the tree h times."""
    def height(node):
        if not node:
            return 0
        return 1 + max(height(node.left), height(node.right))

    def collect(node, depth, target, out):
        """Append values of nodes sitting exactly at depth == target."""
        if not node:
            return
        if depth == target:
            out.append(node.val)
            return                          # nothing deeper can be at `target`
        collect(node.left,  depth + 1, target, out)   # left first =>
        collect(node.right, depth + 1, target, out)   # left-to-right order

    result = []
    for d in range(height(root)):
        level = []
        collect(root, 0, d, level)        # re-walks from the root EVERY time
        result.append(level)
    return result
```

8 Limitation of brute force

To reach level 5 the traversal must pass through levels 0 to 4 again, and it already did that on the previous five passes. All the upper levels are re-walked once for every level below them, which is where the O(n · h) comes from. On a skewed tree that degrades to quadratic.

The fix is to keep the frontier instead of recomputing it. Once I have finished level `d`, the nodes of level `d + 1` are precisely the children I just saw — I should hold them in a queue rather than throw them away and rediscover them from the root.

9 Optimal approach in English

Standard BFS with one addition. Put the root in a queue. Then, at the top of each outer iteration, **record the current queue length** — at that instant the queue holds exactly the nodes of one level and nothing else. Pop precisely that many nodes, collecting their values into a fresh `level` list and pushing their children to the back of the queue.

When the snapshot count is exhausted, the level is complete, and everything now in the queue belongs to the next level. Append the `level` list to the output and repeat until the queue is empty. Because a node's children are always pushed behind every node of the current level, the FIFO ordering keeps each level in left-to-right order automatically.

The reason for taking the snapshot **before** the inner loop is that the queue grows during the loop; testing against a live `len(q)` would spill the next level into the current one.

10 Optimal data structures

- A **deque** used strictly as a FIFO queue. FIFO is what preserves left-to-right order within a level; a stack would reverse and interleave levels.
- A **level-size integer snapshot** — the whole trick, and it needs no extra memory beyond one variable. The alternative of pushing `(node, depth)` pairs also works but stores an integer per node for no benefit.
- The output list of lists.

11 Step-by-step optimal algorithm

1. If `root` is null, return an empty list.
2. Initialise the queue with the root, and the output as an empty list.
3. While the queue is non-empty: create an empty `level` list.
4. Take `n = len(q)` **now**, before popping anything. Those n nodes are exactly the current level.
5. Repeat n times: pop the front node, append its value to `level`, and push its left child then its right child, skipping nulls. Left before right is what preserves the required order.
6. After the n pops, append `level` to the output. The queue now holds exactly the next level.
7. When the queue empties, return the output.

12 Fully commented optimal code

```
from collections import deque

def levelOrder(root):
    """BFS. `for _ in range(len(q))` snapshots the level size and is the only
    idea in this problem -- every level-aware BFS reuses it."""
    if not root:
        return []

    out = []
    q = deque([root])

    while q:
        level = []
        # *** snapshot the size BEFORE popping: the queue grows inside this loop ***
        for _ in range(len(q)):
            node = q.popleft()
            level.append(node.val)
            if node.left:                # left pushed first ->
                q.append(node.left)
            if node.right:               # -> preserves left-to-right order
                q.append(node.right)
        out.append(level)                # queue now holds exactly the next level

    return out
```

+ Optimal complexity, and what to say out loud

Time: O(n). Every node is enqueued once and dequeued once, doing O(1) work each time.

Space: O(w) where w is the maximum width of the tree, since the queue never holds more than one level plus part of the next. For a complete binary tree the bottom level holds about n/2 nodes, so this is O(n) in the worst case. The output itself is a further O(n), but that is the required result rather than working space.

Say this out loud: "BFS, with the queue length snapshotted at the start of each iteration so I know exactly where the level boundary is. Each node enters and leaves the queue once, so O(n) time, and the queue is bounded by the widest level."

The variants this unlocks: zigzag order (reverse alternate `level` lists, or use a deque and alternate the append end); right-side view (take the last value of each level); bottom-up order (reverse the output at the end); and average per level. All of them are the same loop with one line changed.

417 Pacific Atlantic Water Flow ★ (stretch)

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Reverse multi-source DFS from the boundary · Optimal: **O(m·n) time, O(m·n) space**

1 Clarification communication

1. Water flows from a cell to a neighbour whose height is **less than or equal to** the current cell — so equal heights do allow flow in both directions?
2. The Pacific touches the top and left edges; the Atlantic touches the bottom and right edges. A corner cell such as `(0, 0)` therefore already touches the Pacific before moving at all.
3. Should the output be a list of `[row, col]` pairs, in any order?
4. Can the grid be 1 × 1, or a single row? In a single row, every cell touches both the top and bottom edges, so all of them qualify.

2 Understanding of the question

Each cell of the grid has a height. Water flows from a cell to any orthogonal neighbour of equal or lower height. The Pacific borders the top and left edges, the Atlantic the bottom and right edges. I must return every cell from which water can reach **both** oceans.

Stated forwards, that is: for each cell, does some non-increasing path lead off the top or left edge, and does some non-increasing path lead off the bottom or right edge? The critical observation is that this is a reachability question, and reachability questions can be run **backwards**. Instead of asking which cells drain into an ocean, I ask which cells an ocean can climb up to, which turns thousands of single-source searches into two multi-source ones.

3 Brute-force approach in English

Take the definition literally. For every cell in the grid, run a fresh DFS that only ever steps to neighbours of equal or lower height, and record whether that search ever touches the top or left edge, and whether it ever touches the bottom or right edge. If both flags come back true, add the cell to the answer.

Each search needs its own visited set, because a cell that was visited during the search starting at some other cell tells me nothing about this one — the searches are independent.

4 Data structures used

- A fresh `visited` set for **each** of the m·n searches.
- Two booleans per search, for Pacific and Atlantic contact.
- The call stack for the DFS.

5 Step-by-step algorithm

1. For each cell `(r, c)` of the grid, create an empty `visited` set and two flags set to False.
2. Run a DFS from `(r, c)`. On entering a cell, if its row is 0 or its column is 0, set the Pacific flag; if its row is the last row or its column is the last column, set the Atlantic flag.
3. Add the cell to `visited`, then for each of the four neighbours that is in bounds, unvisited, and of height less than or equal to the current cell, recurse into it.
4. When the DFS finishes, if both flags are true, append `[r, c]` to the answer.
5. Return the answer list.

6 Time & space complexity with explanation

Time: O((m·n)²). There are m·n starting cells, and a single downhill DFS can in the worst case — a plateau where every height is identical — visit every one of the m·n cells.

Space: O(m·n) for one search's visited set and recursion depth. The set is discarded and rebuilt for the next cell, which is precisely the waste.

7 Fully commented brute-force code

```
def pacificAtlantic_brute(heights):
    """From every cell, flow downhill and see which oceans we reach. O((mn)^2)."""
    if not heights or not heights[0]:
        return []
    rows, cols = len(heights), len(heights[0])
    result = []

    for sr in range(rows):
        for sc in range(cols):
            visited = set()               # a FRESH set for every start cell
            flags = [False, False]        # [reached Pacific, reached Atlantic]

            def dfs(r, c):
                if (r, c) in visited:
                    return
                visited.add((r, c))
                if r == 0 or c == 0:
                    flags[0] = True       # top or left edge -> Pacific
                if r == rows - 1 or c == cols - 1:
                    flags[1] = True       # bottom or right edge -> Atlantic
                for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
                    nr, nc = r + dr, c + dc
                    # water only flows to an equal or LOWER neighbour
                    if (0 <= nr < rows and 0 <= nc < cols
                            and heights[nr][nc] <= heights[r][c]):
                        dfs(nr, nc)

            dfs(sr, sc)
            if flags[0] and flags[1]:
                result.append([sr, sc])
    return result
```

8 Limitation of brute force

The m·n searches overlap almost completely: if cell A drains through cell B, then A's search re-walks the whole downstream region that B's search already covered, and neither can reuse the other's visited set because each search is anchored to a different origin.

The inversion removes the redundancy entirely. "Cell X can reach the Pacific" is the same statement as "the Pacific can reach cell X by walking uphill". There are only two oceans, so instead of m·n forward searches I need exactly two reverse searches, each seeded from a whole border and each visiting any cell at most once.

9 Optimal approach in English

Run two multi-source searches, one per ocean, in the reverse direction. For the Pacific, seed the search at every cell on the top row and every cell on the left column; for the Atlantic, seed at the bottom row and right column. From a seed, move to a neighbour only if that neighbour's height is **greater than or equal to** the current height — that is, walk uphill, which is exactly the reverse of the direction water flows.

Each search marks every cell it can reach in its own set. A cell in the Pacific set is a cell from which water can flow down to the Pacific; likewise for the Atlantic. The answer is the **intersection** of the two sets.

The reversal is the entire idea, and it is one of the most reusable tricks in grid problems. Number of Enclaves, Surrounded Regions and Walls and Gates all use the same move: when a property is defined by reaching a boundary, start at the boundary.

10 Optimal data structures

- Two **hash sets** of `(r, c)` tuples, `pac` and `atl`, each doubling as that search's visited marker and as its result. Using the set for both jobs is what keeps each search linear.
- The call stack for the DFS. BFS with a deque works identically and avoids deep recursion on a large grid.
- Python's set intersection operator `&` for the final combine.

11 Step-by-step optimal algorithm

1. Create two empty sets, `pac` and `atl`.
2. Write `dfs(r, c, seen, prev_height)`: return immediately if `(r, c)` is out of bounds or already in `seen`, or if `heights[r][c] < prev_height`, since water could not have flowed down from here to where we came from.
3. Otherwise add `(r, c)` to `seen` and recurse into all four neighbours, passing this cell's height as the new `prev_height`.
4. Seed the Pacific: for every column `c`, call `dfs(0, c, pac, 0)`. For every row `r`, call `dfs(r, 0, pac, 0)`. Passing 0 as the initial previous height means the seed cell always passes the height test.
5. Seed the Atlantic: for every column `c`, call `dfs(rows - 1, c, atl, 0)`. For every row `r`, call `dfs(r, cols - 1, atl, 0)`.
6. Return `[list(cell) for cell in pac & atl]`.

12 Fully commented optimal code

```
def pacificAtlantic(heights):
    """Invert the question: instead of flowing DOWN from every cell, climb UP
    from each ocean. Two multi-source searches instead of m*n single-source ones."""
    if not heights or not heights[0]:
        return []
    rows, cols = len(heights), len(heights[0])
    pac, atl = set(), set()               # each set is both 'visited' and 'result'

    def dfs(r, c, seen, prev_height):
        if not (0 <= r < rows and 0 <= c < cols) or (r, c) in seen:
            return
        if heights[r][c] < prev_height:   # cannot climb DOWN when running in reverse
            return
        seen.add((r, c))
        for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
            dfs(r + dr, c + dc, seen, heights[r][c])

    for c in range(cols):
        dfs(0, c, pac, 0)                 # top row    -> Pacific
        dfs(rows - 1, c, atl, 0)          # bottom row -> Atlantic
    for r in range(rows):
        dfs(r, 0, pac, 0)                 # left col   -> Pacific
        dfs(r, cols - 1, atl, 0)          # right col  -> Atlantic

    # Cells that drain to BOTH oceans are exactly the intersection.
    return [list(cell) for cell in pac & atl]
```

+ Optimal complexity, and what to say out loud

Time: O(m·n). Each of the two searches adds a cell to its set the first time it reaches it and never revisits it, so each search is linear in the grid, and there are two searches. The final intersection is linear in the set sizes.

Space: O(m·n) for the two sets and the recursion depth.

Say this out loud: "Rather than simulating flow from every cell, I invert it: a cell reaches an ocean exactly when the ocean reaches it walking uphill. That turns m·n searches into two multi-source searches, and the answer is the intersection of the two reachable sets. O(m·n) time and space."

The transferable pattern: whenever a grid property is defined by "can get to the border", start the search at the border and reverse the movement rule.

863 All Nodes Distance K in Binary Tree ★ (stretch)

Day 1 · Tue 25 Aug · Round Type A (graph & tree traversal) · Pattern: Tree → undirected graph, then BFS · Optimal: **O(n) time, O(n) space**

1 Clarification communication

1. Distance means the number of edges on the path between two nodes, travelling either up or down the tree?
2. If `k == 0`, the answer is just the target node itself?
3. Are node values unique? The signature gives me the target as a node reference, so I will compare by identity, which is safe regardless.
4. Should the output be values or nodes, and does the order matter? I will return values in any order.
5. If fewer nodes exist at distance k than expected, I return whatever exists, possibly an empty list?

2 Understanding of the question

Given the root of a binary tree, a specific target node, and an integer k, I must return the values of all nodes exactly k edges away from the target — in **any** direction: below the target, above it, or across through a sibling subtree.

The difficulty is entirely structural. A tree's pointers only run downward, so from the target I can descend but I cannot climb, and yet many of the answers are up and then back down. Once I add the ability to move from a node to its parent, the tree becomes an **undirected graph** with the same node set, and "all nodes at distance exactly k" is then the plainest possible BFS: expand k levels from the target and read off the frontier.

"Convert the tree into a graph by wiring parent pointers" is one of the highest-value reusable tricks in tree interviews.

3 Brute-force approach in English

Compute the distance from the target to every node independently, using only downward traversals. For a pair of nodes, the distance is `depth(a) + depth(b) − 2 · depth(lca(a, b))`, which is the standard path-through-the-ancestor formula. So: walk the whole tree; for each node, find the LCA of it and the target, compute the three depths with further traversals, evaluate the formula, and keep the node if the result equals k.

It is correct because it is the definition of tree distance, but every one of those helper calls is itself a full traversal.

4 Data structures used

- The call stack for several nested recursions: the outer walk, an LCA search, and a depth lookup per node.
- The output list.

5 Step-by-step algorithm

1. Write `depth(root, node)` that returns the number of edges from the root down to `node`, or −1 if not found. This is O(n).
2. Write `lca(root, a, b)`, the standard post-order LCA. This is also O(n).
3. Walk every node `x` in the tree.
4. For each `x`, compute `l = lca(root, x, target)`, then `d = depth(root, x) + depth(root, target) − 2 * depth(root, l)`.
5. If `d == k`, append `x.val` to the output.
6. Return the output.

6 Time & space complexity with explanation

Time: O(n²). The outer walk visits n nodes and each iteration performs a constant number of O(n) traversals for the LCA and the depths.

Space: O(h) for the recursion stack, plus the output.

7 Fully commented brute-force code

```
def distanceK_brute(root, target, k):
    """distance(a, b) = depth(a) + depth(b) - 2 * depth(lca(a, b)).
    Correct, but every helper is a full O(n) traversal."""

    def depth(node, want, d=0):
        """Edges from `node` down to `want`, or -1 if `want` is not below `node`."""
        if not node:
            return -1
        if node is want:
            return d
        left = depth(node.left, want, d + 1)
        if left != -1:
            return left
        return depth(node.right, want, d + 1)

    def lca(node, a, b):
        if not node or node is a or node is b:
            return node
        L, R = lca(node.left, a, b), lca(node.right, a, b)
        return node if (L and R) else (L or R)

    out = []
    d_target = depth(root, target)

    def walk(node):
        if not node:
            return
        anc = lca(root, node, target)                # O(n)
        d = depth(root, node) + d_target - 2 * depth(root, anc)   # two more O(n) calls
        if d == k:
            out.append(node.val)
        walk(node.left)
        walk(node.right)

    walk(root)
    return out
```

8 Limitation of brute force

Every node triggers a fresh LCA search and fresh depth computations from the root, all of which re-traverse the same upper parts of the tree over and over. The formula is also doing far more than the question needs: I do not want the distance to *every* node, only the set of nodes at one specific distance, and BFS produces exactly that set as a by-product of expanding.

The real obstacle is only that I cannot walk upward. Fixing that once, with a single traversal that records each node's parent, removes the need for all the LCA and depth machinery.

9 Optimal approach in English

Two phases. **Phase one:** run one DFS over the tree recording, for every node, a pointer to its parent in a hash map. The tree now behaves as an undirected graph in which each node has up to three neighbours: `left`, `right`, and `parent`.

Phase two: run a plain BFS starting from the target node, with a `seen` set to stop the search from bouncing back into the node it just came from. Process the queue one full level at a time, tracking the current distance. When the distance counter reaches k, everything currently in the queue is exactly k edges from the target, so return those values.

If the queue drains before the distance reaches k, no node is that far away and the answer is an empty list.

10 Optimal data structures

- A **hash map** from node to parent node, which is what makes movement bidirectional. This is the structure that turns an impossible traversal into a routine one.
- A **deque** as the BFS queue, drained one level at a time with the `len(q)` snapshot from problem 102.
- A `seen` **set** of nodes, essential here because the graph now contains cycles: a node and its parent point at each other.

11 Step-by-step optimal algorithm

1. Create an empty `parent` map. Run `wire(node, par)` from the root with `par = None`: record `parent[node] = par`, then recurse into `node.left` and `node.right` with `node` as the parent.
2. Initialise `seen = {target}` and the queue holding just `target`, and set `dist = 0`.
3. While the queue is non-empty: if `dist == k`, return the values of every node currently in the queue — they are all exactly k edges away.
4. Otherwise snapshot `len(q)` and pop that many nodes, which is the current level.
5. For each popped node, consider its three neighbours: `node.left`, `node.right`, and `parent[node]`. For each that is non-null and not in `seen`, add it to `seen` and push it.
6. After the level completes, increment `dist` and loop.
7. If the queue drains, return an empty list.

12 Fully commented optimal code

```
from collections import deque

def distanceK(root, target, k):
    """Give every node a parent pointer -> the tree becomes an undirected graph
    -> 'all nodes at distance k' is a plain BFS stopped after k levels."""

    parent = {}

    def wire(node, par):
        """One DFS to record each node's parent."""
        if not node:
            return
        parent[node] = par
        wire(node.left, node)
        wire(node.right, node)

    wire(root, None)

    seen = {target}                       # required: the graph now has cycles
    q = deque([target])
    dist = 0

    while q:
        if dist == k:
            # Everything still queued sits exactly k edges from the target.
            return [n.val for n in q]
        for _ in range(len(q)):           # drain exactly one level
            node = q.popleft()
            # three neighbours now, not two: left, right, and UP
            for nei in (node.left, node.right, parent[node]):
                if nei and nei not in seen:
                    seen.add(nei)         # mark on enqueue
                    q.append(nei)
        dist += 1

    return []                             # no node is that far from the target
```

+ **Optimal complexity, and what to say out loud**

Time: O(n). The wiring DFS visits each node once. The BFS enqueues each node at most once, and each dequeue inspects three neighbours, which is constant work.

Space: O(n) for the parent map and the seen set, plus O(h) for the wiring recursion.

Say this out loud: "The obstacle is that tree edges only point downward, but the answer includes nodes above the target. So I spend one pass adding parent pointers, which converts the tree into an undirected graph, and then it is an ordinary BFS from the target stopped after k levels. O(n) time and space."

The gotcha: the `seen` set is not optional here as it was in a pure downward traversal. Once parents are wired, every edge is bidirectional, so without `seen` the search immediately oscillates between a node and its parent forever.

1 Two Sum

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Hash map of complements · Optimal: **O(n) time, O(n) space**

1 Clarification communication

1. Is there **exactly one** valid answer guaranteed, or could there be several? If several, do you want any one of them or all of them?
2. May I use the same element twice? The standard statement says no, and that is what forces the ordering of the lookup and the insert in my code.
3. Do you want the **indices** or the values? Indices, which rules out sorting the array in place unless I also carry the original positions.
4. Can the array contain negatives, zeros or duplicates? All three are fine for a hash map, but duplicates are what break a careless implementation.
5. What should I return if no pair exists — an empty list?

2 Understanding of the question

Given an array of integers and a target, return the indices of the two numbers that add up to the target.

The reframing that makes it trivial: for a fixed element `x`, there is exactly one value that could pair with it, namely `target - x`. So the question is not "which two elements work together" — a pairing question, quadratic by nature — but "for this `x`, **have I already seen** its complement?" That is a membership question, and membership questions are what hash maps answer in O(1). This is the smallest possible instance of the pattern that this whole round is about: replace a repeated inner scan with a lookup.

3 Brute-force approach in English

Try every pair. For each index `i`, walk every index `j` strictly greater than `i` and test whether `nums[i] + nums[j]` equals the target. Return `[i, j]` on the first hit.

Starting `j` at `i + 1` rather than 0 does two useful things at once: it prevents pairing an element with itself, and it avoids examining each unordered pair twice.

4 Data structures used

- None. Two index variables and the input array. This is the baseline against which the optimisation is measured, and it is worth saying out loud that it uses O(1) extra space, because that is the one thing the brute force is better at.

5 Step-by-step algorithm

1. Loop `i` from 0 to `n - 1`.
2. For each `i`, loop `j` from `i + 1` to `n - 1`.
3. If `nums[i] + nums[j] == target`, return `[i, j]`.
4. If both loops complete without a match, return an empty list.

6 Time & space complexity with explanation

Time: O(n²). The inner loop runs n−1 times for the first `i`, n−2 for the second, and so on, which sums to n(n−1)/2 pair checks — quadratic. Each check is O(1).

Space: O(1). Only two loop counters; nothing is stored.

7 Fully commented brute-force code

```
def twoSum_brute(nums, target):
    """Check every unordered pair exactly once."""
    n = len(nums)
    for i in range(n):
        for j in range(i + 1, n):         # j > i: no self-pairing, no repeated pairs
            if nums[i] + nums[j] == target:
                return [i, j]
    return []
```

8 Limitation of brute force

For each `i`, the inner loop re-scans the tail of the array asking the same question every time: "is the value `target - nums[i]` present?" That is a membership query being answered by linear search, over and over, on data that never changes.

At n = 100,000 this is about 5×10^9 comparisons, which will not run in time. Any membership query answered by scanning is the signal to reach for a hash map.

9 Optimal approach in English

One pass. Keep a dictionary mapping each value already seen to the index where it was seen. For the current element `x` at index `i`, compute `target - x` and check whether that complement is already in the dictionary. If it is, I have found the pair immediately and can return the stored index together with `i`. If it is not, record `x` with its own index and move on.

The ordering matters and is worth narrating: I **look up before I insert**. Looking up first means the dictionary only ever contains elements strictly before `i`, which is exactly what guarantees I never pair an element with itself. With `nums = [3, 3]` and `target = 6`, the look-up-first order finds index 0 paired with index 1, while insert-first would match the single element at index 0 against itself.

Because the complement of the complement is the original number, one pass is enough: whichever of the two partners appears second will find the first one waiting in the map.

10 Optimal data structures

- A **hash map** from value to index. Value-to-index rather than a plain set, because the problem asks for indices — a set would tell me the pair exists but not where it is.
- If duplicates exist and only one answer is needed, keeping the *first* index of a repeated value is fine; the look-up-first ordering means the second occurrence finds the first.

11 Step-by-step optimal algorithm

1. Create an empty dictionary `seen`, mapping value → index.
2. Iterate over the array with both index `i` and value `x`.
3. Compute `need = target - x`.
4. If `need` is a key in `seen`, return `[seen[need], i]`. The stored index is strictly smaller than `i`, so the pair is ordered and distinct.
5. Otherwise set `seen[x] = i`. **This must come after the lookup**, never before.
6. If the loop ends with no match, return an empty list.

12 Fully commented optimal code

```
def twoSum(nums, target):
    """One pass, hash map of complements."""
    seen = {}                               # value -> index where we saw it
    for i, x in enumerate(nums):
        need = target - x                   # the ONLY value that can pair with x
        if need in seen:                    # *** look up BEFORE inserting x ***
            return [seen[need], i]          # seen[need] < i, so the pair is distinct
        seen[x] = i
    return []
```

+ **Optimal complexity, and what to say out loud**

Time: O(n). A single pass, with one O(1) average-case hash lookup and one O(1) insert per element. Worst-case hashing is O(n) per operation under adversarial collisions, but that does not occur with ordinary integer keys.

Space: O(n). The map may hold up to n−1 entries before the answer is found. This is the trade being made: O(n) memory buys an O(n) reduction in time.

Say this out loud: "The brute force checks every pair, O(n²). The repeated work is scanning for the complement, and a complement lookup is a membership query, so a hash map makes it O(1) and the whole algorithm O(n). I look up before inserting so an element can never pair with itself."

The gotcha: the lookup-then-insert ordering, demonstrated with `[3, 3]`. **The follow-up:** if the array were sorted and only values were needed, two pointers give O(n) time with O(1) space — but here the indices are required, so sorting would destroy the answer unless I first paired every value with its original index.

560 Subarray Sum Equals K ★★

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Prefix sum + hash map of counts · Optimal: **O(n) time, O(n) space**

1 Clarification communication

1. Are the values allowed to be **negative**? This is the decisive question. With negatives, a sliding window is invalid, because extending the window no longer moves the sum monotonically. I want to confirm this before I choose an approach.
2. Do I count the **number** of subarrays, or return the actual subarrays? Just the count.
3. Must subarrays be contiguous and non-empty? Yes on both, I assume.
4. Can `k` be zero or negative? Both are fine for the prefix-sum method.
5. Can subarrays overlap? Yes — every distinct index range counts separately.

2 Understanding of the question

Given an integer array and an integer `k`, count how many **contiguous** subarrays sum exactly to `k`.

The key algebraic step: let `prefix[i]` be the sum of the first `i` elements. Then the sum of the subarray from index `i` to index `j` inclusive is `prefix[j+1] - prefix[i]`. Setting that equal to `k` and rearranging gives `prefix[i] = prefix[j+1] - k`.

That rearrangement converts the problem completely. Instead of asking "which ranges sum to k", which is quadratic in nature, I fix the right end `j` and ask "**how many earlier prefix sums equal `prefix[j+1] - k`?**" — a counting-membership question a hash map answers in O(1). It is Two Sum, applied to prefix sums, counting matches instead of stopping at the first.

3 Brute-force approach in English

Enumerate every contiguous subarray and test it. Fix the start index `i`, then extend the end index `j` rightward, maintaining a running total so each extension costs O(1) rather than re-summing the range. Every time the running total equals `k`, increment the counter.

Keeping the running sum is what makes this O(n²) rather than the fully naive O(n³) version that re-adds the whole range for each pair of endpoints. It is worth mentioning both in the interview: it shows you already removed one layer of waste before proposing the real optimisation.

4 Data structures used

- A single running-sum integer and a counter. No auxiliary structure.
- Space O(1), which is the brute force's only advantage over the optimal solution.

5 Step-by-step algorithm

1. Set `count = 0`.
2. Loop `i` from 0 to `n - 1`. This is the start of the subarray.
3. Set `running = 0`.
4. Loop `j` from `i` to `n - 1`, adding `nums[j]` to `running`. After the addition, `running` is the sum of the subarray from `i` to `j`.
5. If `running == k`, increment `count`. Do **not** break: with negative numbers a longer subarray from the same start can also hit `k`.
6. Return `count`.

6 Time & space complexity with explanation

Time: O(n²). There are n(n+1)/2 contiguous subarrays and the running sum makes each one O(1) to evaluate.

Space: O(1). Only two integers.

7 Fully commented brute-force code

```
def subarraySum_brute(nums, k):
    """Every contiguous subarray, with a running sum so each extension is O(1)."""
    n = len(nums)
    count = 0
    for i in range(n):                    # start of the subarray
        running = 0
        for j in range(i, n):             # extend the end rightward
            running += nums[j]            # sum of nums[i..j], updated in O(1)
            if running == k:
                count += 1                # do NOT break: negatives allow more hits
    return count
```

8 Limitation of brute force

The two loops recompute overlapping information constantly: the sum of `nums[2..7]` is recomputed from scratch when the start moves to 3, even though it differs from `nums[3..7]` by a single element. Across the array, the same prefix arithmetic is repeated O(n) times.

It is also worth stating explicitly why the usual O(n²)-to-O(n) trick, the sliding window, does **not** apply. A sliding window relies on the window sum increasing when you extend right and decreasing when you shrink left; with negative numbers neither holds, so there is no valid rule for when to move `left`. Prefix sums plus a hash map work regardless of sign, which is exactly why they are the right tool here.

9 Optimal approach in English

Sweep left to right maintaining the running prefix sum. Also maintain a dictionary counting **how many times each prefix-sum value has occurred so far**. At each index, the number of subarrays ending here that sum to `k` is precisely the number of earlier positions whose prefix sum equals `running - k`, so I add that stored count to the answer. Then I record the current prefix sum in the dictionary and continue.

The dictionary must start containing `{0: 1}`. That entry represents the empty prefix — the sum before any element is consumed — and it is what allows a subarray that starts at index 0 to be counted. Without it, `nums = [3]` with `k = 3` returns 0 instead of 1. Forgetting this single line is the classic bug in this problem, so I would say it aloud as I type it.

As with Two Sum, the lookup happens before the insert, so a subarray is never matched against its own endpoint, which would produce a spurious zero-length hit when `k = 0`.

10 Optimal data structures

- A **hash map from prefix-sum value to a count of occurrences**. A count, not a single index, because the same prefix sum can occur many times and each occurrence is a distinct valid subarray start. Using a set here is the second most common bug.
- Seeded with `{0: 1}` for the empty prefix.
- Two integers: the running sum and the total count.

11 Step-by-step optimal algorithm

1. Create `count` as a dictionary of prefix sum → occurrences, initialised with `count[0] = 1` for the empty prefix.
2. Set `running = 0` and `total = 0`.
3. For each value `x` in the array, add `x` to `running`, so `running` is now the prefix sum through the current index.
4. Add `count[running - k]` to `total`. Every earlier position holding that prefix sum marks the start of a subarray ending here that sums to `k`.
5. Then increment `count[running]` by one. **After** the lookup, always.
6. Return `total`.

12 Fully commented optimal code

```
from collections import defaultdict

def subarraySum(nums, k):
    """sum(i..j) == prefix[j] - prefix[i-1], so for each j count the earlier
    prefixes equal to prefix[j] - k."""
    count = defaultdict(int)
    count[0] = 1                          # *** the empty prefix: without this,
                                          # subarrays starting at index 0 are missed ***
    running = 0
    total = 0
    for x in nums:
        running += x                      # prefix sum through the current index
        total += count[running - k]       # look up BEFORE inserting
        count[running] += 1               # register this prefix for future indices
    return total
```

+ Optimal complexity, and what to say out loud

Time: O(n). One pass, with one O(1) average hash lookup and one O(1) increment per element.

Space: O(n). In the worst case every prefix sum is distinct and the map holds n+1 entries.

Say this out loud: "Because the sum of a range is a difference of two prefix sums, I rewrite the condition as prefix[j] − prefix[i] = k. Fixing the right endpoint turns it into counting how many earlier prefixes equal prefix[j] − k, and a hash map of prefix counts answers that in O(1). One pass, O(n) time, O(n) space."

The three things to say unprompted: `count[0] = 1` and why; the map stores counts, not indices, because a prefix sum can repeat; and sliding window does **not** work here because negative values break the monotonicity that a window depends on. That last point is often the real thing being tested.

3 Longest Substring Without Repeating Characters ★★★

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Sliding window with a last-seen map · Optimal: **O(n) time, O(min(n, charset)) space**

1 Clarification communication

1. Do you want the **length**, or the substring itself? Length, I assume — if you want the substring I just record the start index too.
2. Substring means contiguous, as opposed to subsequence?
3. What is the character set — lowercase letters, ASCII, or full Unicode? It does not change the algorithm, only the space bound.
4. Is the empty string a valid input? It should return 0.
5. Are spaces and symbols ordinary characters here? I will assume yes, nothing is excluded.

2 Understanding of the question

Given a string, find the length of the longest contiguous substring containing no repeated character.

This is the archetype of the sliding-window pattern, and every window problem has the same three parts. **The window** is a contiguous range `[left, right]`. **The validity condition** is "no character appears twice inside the window". **The movement rule** is: extend `right` unconditionally, and whenever the window becomes invalid, advance `left` until it is valid again. Because a window that is valid stays valid when you shrink it from the left, and validity is only ever broken by the newly added right-hand character, this single pass is enough — you never need to move `left` backwards.

3 Brute-force approach in English

Test every substring for distinctness. For each start index `i`, extend the end index `j` rightward, keeping a set of the characters seen since `i`. The moment `s[j]` is already in that set, the substring starting at `i` cannot be extended any further, so record the length `j - i` and break to the next start. Track the maximum across all starts.

The set makes each extension O(1). The fully naive alternative — slicing out each substring and checking `len(set(sub)) == len(sub)` — is O(n³), and mentioning that you already avoided it is a good sign.

4 Data structures used

- One **set** of characters, rebuilt from scratch for each start index. Rebuilding it is precisely the waste.
- An integer holding the best length so far.

5 Step-by-step algorithm

1. Set `best = 0`.
2. Loop `i` from 0 to `n - 1`, the start of the candidate substring.
3. Create an empty set `seen`.
4. Loop `j` from `i` to `n - 1`. If `s[j]` is already in `seen`, break: no longer substring starting at `i` can be distinct.
5. Otherwise add `s[j]` to `seen` and update `best` with `j - i + 1`.
6. Return `best`.

6 Time & space complexity with explanation

Time: O(n²). There are n start indices, and for each the inner loop can run to the end of the string. Set operations are O(1), so the total is quadratic. The slicing version would be O(n³).

Space: O(min(n, c)) where c is the alphabet size, for the set.

7 Fully commented brute-force code

```
def lengthOfLongestSubstring_brute(s):
    """Every start index, extended until a duplicate appears."""
    n = len(s)
    best = 0
    for i in range(n):
        seen = set()                    # rebuilt from scratch for every i
        for j in range(i, n):
            if s[j] in seen:
                break                   # no longer substring from i can be distinct
            seen.add(s[j])
            best = max(best, j - i + 1)
    return best
```

8 Limitation of brute force

When the substring starting at `i` fails at position `j`, the algorithm throws away everything it learned and restarts at `i + 1` from scratch, re-reading nearly the same characters. But the failure told me something precise: the only reason `[i, j]` is invalid is the earlier copy of `s[j]`, so **every** start index at or before that earlier copy is also doomed. Restarting at `i + 1` ignores that, when I could jump `left` past the duplicate in one move.

That is the whole optimisation: replace "restart the left end at `i + 1`" with "advance the left end only as far as the duplicate forces", which lets `left` move monotonically forward across the entire run and never backwards.

9 Optimal approach in English

Keep a window `[left, right]` and a dictionary `last` recording the most recent index at which each character was seen. Move `right` across the string one character at a time. If the incoming character `ch` has been seen before **and its last occurrence is at or after** `left` — that is, it is actually inside the current window — then the window is invalid, and I jump `left` to `last[ch] + 1`, which is the nearest position that excludes the duplicate. Then I update `last[ch]` to `right` and record the window length.

The condition `last[ch] >= left` is the subtle part and the thing interviewers probe. Without it, a stale occurrence from far earlier in the string — already outside the window — would drag `left` *backwards*, silently producing lengths that are too large. With the check, `left` is monotonically non-decreasing, which is also why the algorithm is linear.

I would also mention the count-map variant, which is closer to the universal window template: increment a counter for the incoming character and, while that counter exceeds 1, shrink from the left. It is slightly slower in the worst case but generalises directly to problems 424 and 76, so knowing both is worth the extra minute.

10 Optimal data structures

- A **hash map from character to its last-seen index**. Storing the index rather than a boolean is what allows `left` to jump straight past the duplicate instead of creeping one step at a time.
- Two integer pointers, `left` and `right`, and a best-length integer.
- For the variant: a **count map** of characters currently inside the window.

11 Step-by-step optimal algorithm

1. Create an empty dictionary `last`, and set `left = 0` and `best = 0`.
2. Iterate `right` over the string, letting `ch = s[right]`.
3. If `ch` is in `last` **and** `last[ch] >= left`, the duplicate is inside the current window, so set `left = last[ch] + 1`, which is the smallest left endpoint that excludes it.
4. Set `last[ch] = right`, recording this as the most recent occurrence. This happens whether or not the window moved.
5. Update `best` with `right - left + 1`, the current window length.
6. Return `best`.
7. Note that `left` never decreases, so `right` and `left` each traverse the string once.

12 Fully commented optimal code

```
def lengthOfLongestSubstring(s):
    """Sliding window; `last` lets the left edge JUMP past a duplicate."""
    last = {}                           # character -> index of its last occurrence
    left = 0
    best = 0
    for right, ch in enumerate(s):
        # *** `last[ch] >= left` is the crux: without it, a stale occurrence
        # from OUTSIDE the window would drag `left` backwards. ***
        if ch in last and last[ch] >= left:
            left = last[ch] + 1         # smallest left that excludes the duplicate
        last[ch] = right                # always refresh the last-seen index
        best = max(best, right - left + 1)
    return best


from collections import defaultdict

def lengthOfLongestSubstring_v2(s):
    """Count-map variant: closer to the universal window template, and the one
    that generalises to problems 424 and 76."""
    count = defaultdict(int)
    left = 0
    best = 0
    for right, ch in enumerate(s):
        count[ch] += 1
        while count[ch] > 1:            # window INVALID: shrink until it is not
            count[s[left]] -= 1
            left += 1
        best = max(best, right - left + 1)
    return best
```

+ Optimal complexity, and what to say out loud

Time: O(n). `right` advances n times, and `left` only ever moves forward, so across the whole run the two pointers perform at most 2n moves. Every dictionary operation is O(1).

Space: O(min(n, c)) where c is the alphabet size — the map holds at most one entry per distinct character, so it is bounded by 26 for lowercase letters or 128 for ASCII.

Say this out loud: "Sliding window. My map holds the last index of each character. The window is invalid exactly when the incoming character already sits inside it, and then I jump left past its previous occurrence, because every start at or before that point is also invalid. Left never moves backwards, so each pointer crosses the string once: O(n)."

The gotcha to volunteer: the `last[ch] >= left` guard, with `"abba"` as the demonstration — at the final `a`, the stale index 0 is outside the window and must be ignored, or `left` snaps back to 1 and the answer becomes 3 instead of 2.

424 Longest Repeating Character Replacement ★

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Sliding window with a frequency map · Optimal: **O(n) time, O(26) space**

1 Clarification communication

1. Am I replacing **any** character with **any** other uppercase letter, at most `k` times total within the chosen substring?
2. Is the alphabet restricted to uppercase A–Z? That is what makes the space O(26) rather than O(n).
3. Can `k` be 0? Then the answer is the longest run of a single repeated character, which is a good sanity test.
4. Can `k` exceed the string length? Then the whole string is achievable and the answer is `len(s)`.
5. Do you want the length only, not the resulting string?

2 Understanding of the question

Given a string of uppercase letters and an integer `k`, I may change at most `k` characters to any other letter. I must return the length of the longest substring that can be made to consist of a single repeated character.

The reframing that turns this into a window problem: for a fixed substring, the best possible play is obviously to keep whichever character is already most frequent and convert everything else. So the number of changes required is `window_length - count_of_most_frequent_character`. The window is therefore **valid** exactly when that quantity is at most `k`, and I want the longest valid window. Same three-part window structure as problem 3, with a different validity predicate.

3 Brute-force approach in English

Check every substring against the validity rule directly. For each start index `i`, extend the end index `j` rightward while maintaining a frequency count of the characters in `[i, j]` and the running maximum of those counts. After each extension, evaluate whether `(j - i + 1) - max_frequency <= k`. If it holds, the substring is achievable and I update the best length; if it does not, no longer substring from this same start can help either, so I break to the next start.

That break is legitimate because extending the window by one character increases the length by one but can increase the maximum frequency by at most one, so the number of required changes never decreases as the window grows from a fixed start.

4 Data structures used

- A **frequency map** of 26 letters, rebuilt for each start index.
- An integer for the maximum frequency within the current substring, and one for the best answer so far.

5 Step-by-step algorithm

1. Set `best = 0`.
2. Loop `i` from 0 to `n - 1`.
3. Reset an empty frequency map and `max_freq = 0`.
4. Loop `j` from `i` to `n - 1`. Increment the count for `s[j]` and update `max_freq` with it.
5. Compute `window = j - i + 1` and `changes_needed = window - max_freq`.
6. If `changes_needed <= k`, update `best` with `window`. Otherwise break out to the next start index.
7. Return `best`.

6 Time & space complexity with explanation

Time: O(n²). n start indices, each extending up to n positions, with O(1) work per extension because the frequency map is updated incrementally rather than recomputed. Recomputing the counts for each substring instead would make it O(n² · 26).

Space: O(26) = O(1). One fixed-size frequency map, discarded and rebuilt per start.

7 Fully commented brute-force code

```
def characterReplacement_brute(s, k):
    """Every start index, extended while the window stays achievable."""
    from collections import defaultdict
    n = len(s)
    best = 0
    for i in range(n):
        count = defaultdict(int)          # rebuilt for every start index
        max_freq = 0
        for j in range(i, n):
            count[s[j]] += 1
            max_freq = max(max_freq, count[s[j]])
            window = j - i + 1
            # keep the most common char, change all the others
            if window - max_freq <= k:
                best = max(best, window)
            else:
                break                     # from this start, longer is only worse
    return best
```

8 Limitation of brute force

When the window from `i` becomes unachievable, the algorithm discards the entire frequency map and rebuilds it from `i + 1`, re-reading almost the same characters. But moving the start forward by one only removes a single character from the window — the counts for everything else are unchanged and were already correct.

So instead of restarting, I should keep the same map and simply decrement the count of the character leaving on the left. That makes `left` move forward monotonically across the whole string, and both pointers together do O(n) work rather than O(n²).

9 Optimal approach in English

Maintain one window and one frequency map for the entire pass. Extend `right`, incrementing the count of the incoming character and updating `max_freq`. If the window is now invalid — `(right - left + 1) - max_freq > k` — advance `left` by one, decrementing the count of the outgoing character. Record the best window length at each step.

Now the famous subtlety, which interviewers reliably probe: `max_freq` **is never decreased**, even after characters leave the window, so it can become stale and overstate the true maximum frequency. This is deliberate and it is safe. A stale, too-large `max_freq` makes the validity test *easier* to pass, so the window can never be wrongly shrunk; the worst that happens is the window fails to grow for a while. And crucially, `best` can only ever be updated by a window that is at least as long as the previous best, and to beat the previous best the window must at some point have contained a genuinely higher character frequency. So the recorded answer is always achievable, and the true optimum is always found.

I would present the `while` form rather than the `if` form. Both give the correct answer here, but `while` is the honest sliding-window template and it keeps the invariant "the window is always valid at the bottom of the loop", which is easier to defend.

10 Optimal data structures

- A **frequency map over the 26 uppercase letters**, maintained incrementally across the entire pass rather than per start index. This is the single change from the brute force.
- Two pointers, `left` and `right`.
- Two integers: `max_freq`, intentionally never decremented, and `best`.

11 Step-by-step optimal algorithm

1. Create an empty frequency map. Set `left = 0`, `best = 0`, `max_freq = 0`.
2. Iterate `right` over the string with `ch = s[right]`.
3. Increment `count[ch]`, then update `max_freq = max(max_freq, count[ch])`.
4. While `(right - left + 1) - max_freq > k`, the window needs more than `k` replacements, so decrement `count[s[left]]` and advance `left` by one.
5. Update `best` with `right - left + 1`, which is now guaranteed to be a valid window length.
6. Return `best`.
7. Do not decrement `max_freq` when a character leaves. Be ready to explain why that is safe rather than a bug.

12 Fully commented optimal code

```
from collections import defaultdict

def characterReplacement(s, k):
    """A window is achievable when (length - most_frequent_count) <= k, because
    you keep the most frequent character and replace everything else."""
    count = defaultdict(int)
    left = 0
    best = 0
    max_freq = 0                          # deliberately NEVER decreased -- see below

    for right, ch in enumerate(s):
        count[ch] += 1
        max_freq = max(max_freq, count[ch])

        # INVALID: this window would need more than k replacements.
        while (right - left + 1) - max_freq > k:
            count[s[left]] -= 1
            left += 1

        best = max(best, right - left + 1)

    return best

# Why a stale max_freq is safe, not a bug:
#   A too-large max_freq only makes the validity test EASIER to pass, so the
#   window is never shrunk when it should not be. And `best` can only grow when
#   the window is genuinely longer than before, which requires a real character
#   frequency at least that high to have occurred. So the answer stays correct.
```

+ Optimal complexity, and what to say out loud

Time: O(n). `right` advances n times; `left` only moves forward and therefore advances at most n times in total. Every map operation is O(1).

Space: O(26) = O(1). The frequency map holds at most one entry per uppercase letter, independent of the input size.

Say this out loud: "Sliding window. My map holds character frequencies inside the window. The window is valid when its length minus the highest frequency is at most k, because that difference is exactly how many characters I would have to replace. Both pointers move forward only, so O(n) time and O(26) space."

Have the `max_freq` defence ready before they ask — volunteering it is one of the highest-signal things you can do on this problem, because it demonstrates you understand why the code is correct rather than having memorised it.

15 3Sum ★

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Sort + two pointers, with dedup at two levels · Optimal: **O(n²) time, O(1) extra space**

1 Clarification communication

1. The output should be **unique triplets by value**, not by index? That is the entire difficulty of this problem, so I want it confirmed.
2. May I **sort the input**? Sorting is what makes the two-pointer scan possible, and it destroys the original indices — but since the answer is values, not indices, that is fine.
3. Can the array contain duplicates, and can the same value be used more than once if it appears more than once? For example, does `[0, 0, 0]` yield `[[0,0,0]]`?
4. Does the order of the triplets in the output matter, or the order within a triplet?
5. Is the target always zero, or should I write it for a general target so it extends to 3Sum Closest and 4Sum?

2 Understanding of the question

Given an integer array, return all unique triplets `[a, b, c]` with `a + b + c == 0`, where no triplet may be a duplicate of another by value.

There are two distinct sub-problems here, and conflating them is why this question has such a high failure rate. The first is **finding** the triplets efficiently. The second is **deduplicating** the results. Sorting solves both at once: once sorted, fixing the first element reduces the rest to Two Sum on a sorted array, which two pointers solve in linear time and constant space; and equal values become adjacent, so duplicates can be skipped by comparing against the immediate neighbour. But the skipping has to happen at **both** levels — the fixed element and the pair — and forgetting the second is the classic bug.

3 Brute-force approach in English

Three nested loops over all index triples `i < j < l`, testing whether the three values sum to zero. To handle uniqueness, sort each qualifying triplet's values and put the resulting tuple into a set, so that different index combinations producing the same three values collapse to one entry. At the end, convert the set back into a list of lists.

The index ordering `i < j < l` guarantees each element is used at most once per triplet and that each combination is examined only once.

4 Data structures used

- A **set of sorted value tuples**, used solely for deduplication. Tuples because they are hashable; sorted so that `(-1, 0, 1)` and `(0, -1, 1)` collapse to one key.
- Three loop indices.

5 Step-by-step algorithm

1. Create an empty set `found`.
2. Loop `i` from 0 to `n - 3`.
3. Loop `j` from `i + 1` to `n - 2`.
4. Loop `l` from `j + 1` to `n - 1`.
5. If `nums[i] + nums[j] + nums[l] == 0`, build the tuple `tuple(sorted((nums[i], nums[j], nums[l])))` and add it to `found`. Sorting inside the tuple is what makes equal-valued triplets hash to the same key.
6. Return `[list(t) for t in found]`.

6 Time & space complexity with explanation

Time: O(n³). Three nested loops give roughly $n^3/6$ triples, each tested in O(1) plus an O(3 log 3) = O(1) sort of three elements.

Space: O(number of unique triplets) for the set, which can itself be $O(n^2)$ in the worst case.

At n = 3000 this is on the order of 4.5×10^9 iterations, far past any time limit.

7 Fully commented brute-force code

```
def threeSum_brute(nums):
    """All index triples i < j < l, deduplicated through a set of sorted tuples."""
    n = len(nums)
    found = set()
    for i in range(n - 2):
        for j in range(i + 1, n - 1):
            for l in range(j + 1, n):
                if nums[i] + nums[j] + nums[l] == 0:
                    # sorted() so (-1,0,1) and (0,-1,1) collapse to one key
                    found.add(tuple(sorted((nums[i], nums[j], nums[l]))))
    return [list(t) for t in found]
```

8 Limitation of brute force

The innermost loop is a linear search for the value `-(nums[i] + nums[j])`, repeated for every pair, over unsorted data that reveals nothing about where that value might be. That is the same inefficiency as Two Sum, one level deeper.

The deduplication is also being handled after the fact, by hashing every result. That works but it means doing all the wasted work first and cleaning up afterwards, and it costs O(n²) memory in the worst case.

Sorting fixes both problems at once. On a sorted array the inner search becomes a two-pointer sweep that is linear rather than quadratic, and duplicates become adjacent so they can be skipped *before* the work is done rather than filtered out afterwards.

9 Optimal approach in English

Sort the array first. Then fix the leftmost element at index `i` and solve Two Sum on the sorted remainder with two pointers: `lo` just after `i`, `hi` at the end. Compute the sum of the three. If it is too small, the only way to increase it is to move `lo` right; if it is too large, move `hi` left; if it is exactly zero, record the triplet and move both inward.

Two dedup rules, and both are required. At the **outer** level, if `nums[i]` equals `nums[i-1]` then every triplet using this value as the fixed element was already generated, so skip it. At the **inner** level, after recording a hit, advance `lo` past any run of equal values, otherwise the identical triplet is emitted once per duplicate.

One more early exit worth including: because the array is sorted, once `nums[i] > 0` every remaining element is positive and no three of them can sum to zero, so the loop can break entirely.

This structure generalises cleanly: 4Sum is the same idea with one more fixed level, and 3Sum Closest is the same sweep tracking the minimum absolute difference instead of exact equality.

10 Optimal data structures

- The **sorted array itself** — sorting is the enabling step, providing both the ordering that makes two pointers valid and the adjacency that makes dedup a neighbour comparison.
- Two integer pointers, `lo` and `hi`. No hash set is needed, which is why the extra space is O(1) rather than O(n²).
- The output list.

11 Step-by-step optimal algorithm

1. Sort `nums` ascending.
2. Loop `i` from 0 to `n - 3`.
3. If `nums[i] > 0`, break. The array is sorted, so all remaining values are positive and cannot sum to zero.
4. If `i > 0` and `nums[i] == nums[i - 1]`, continue — this is the **outer** dedup, skipping a fixed value already fully processed.
5. Set `lo = i + 1` and `hi = n - 1`.
6. While `lo < hi`, compute `s = nums[i] + nums[lo] + nums[hi]`.
7. If `s < 0`, increment `lo` to increase the sum. If `s > 0`, decrement `hi` to decrease it.
8. If `s == 0`, append the triplet, then increment `lo` once and, while `lo < hi` and `nums[lo] == nums[lo - 1]`, keep incrementing — this is the **inner** dedup. Then decrement `hi` once.
9. Return the result list.

12 Fully commented optimal code

```
def threeSum(nums):
    """Sort, fix the first element, two-pointer the rest. Dedup at BOTH levels."""
    nums.sort()
    res = []
    n = len(nums)

    for i in range(n - 2):
        if nums[i] > 0:
            break                          # sorted: everything left is positive
        if i > 0 and nums[i] == nums[i - 1]:
            continue                       # DEDUP LEVEL 1: the fixed element

        lo, hi = i + 1, n - 1
        while lo < hi:
            s = nums[i] + nums[lo] + nums[hi]
            if s < 0:
                lo += 1                    # too small -> need a bigger value
            elif s > 0:
                hi -= 1                    # too big -> need a smaller value
            else:
                res.append([nums[i], nums[lo], nums[hi]])
                lo += 1
                # DEDUP LEVEL 2: skip the run of equal values at `lo`,
                # otherwise the same triplet is emitted once per duplicate.
                while lo < hi and nums[lo] == nums[lo - 1]:
                    lo += 1
                hi -= 1

    return res
```

+ Optimal complexity, and what to say out loud

Time: O(n²). The sort is O(n log n), then the outer loop runs n times and each inner two-pointer sweep is O(n) because `lo` and `hi` only ever move toward each other. O(n log n) + O(n²) = O(n²).

Space: O(1) extra, ignoring the output list and whatever the sort uses internally (O(log n) for Timsort's stack, or O(n) if you must not modify the input and copy it first).

Say this out loud: "Sorting buys me two things: the two-pointer sweep becomes valid, so the inner search drops from O(n) hashing to O(n) scanning with no extra memory, and equal values become adjacent so I can dedup by comparing with the neighbour. Fix one element, two-pointer the rest: O(n²)."

This is the number-one dedup-bug problem in the set. Solve it twice, and each time say "outer dedup" and "inner dedup" aloud as you type each line. `[0, 0, 0, 0]` and `[-2, 0, 0, 2, 2]` are the two test cases that catch a missing skip.

42 Trapping Rain Water ★★

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Two pointers with running maxima (also: monotonic stack) · Optimal: **O(n) time, O(1) space**

1 Clarification communication

1. Each bar has width 1, and the elevation map is non-negative integers?
2. I return the **total** units of water trapped, not the shape or per-column amounts?
3. Can the array be empty or of length 1 or 2? Any of those trap zero water, since water needs a wall on both sides.
4. Are there constraints that would make an O(n) space solution unacceptable? I have an O(1) space version and I will lead with it, but I want to know if that matters to you.
5. Water can only rest between bars, not run off the ends?

2 Understanding of the question

Given an elevation map where the width of each bar is 1, compute how much rainwater is trapped after it rains.

The single observation the entire problem rests on: **the water sitting above index `i` depends only on `i`, not on the shape of the terrain between it and the walls.** Column `i` holds `min(max_to_the_left, max_to_the_right) - height[i]` units, because water rises until it would spill over the shorter of the two enclosing walls. If that quantity is negative, the bar is taller than one of its walls and it holds nothing.

Once that formula is stated, every solution to this problem is just a different strategy for obtaining the two maxima: recompute them (brute force), precompute them into arrays (DP), or derive them on the fly with two converging pointers (optimal).

3 Brute-force approach in English

Apply the formula literally, one index at a time. For each index `i`, scan leftward from `i` to find the tallest bar at or before it, then scan rightward to find the tallest bar at or after it. Take the smaller of the two, subtract `height[i]`, and add the result to the total if it is positive.

This is transparently correct because it is a direct translation of the water-level formula, which makes it an excellent thing to state first: it establishes the formula, and everything after it is an optimisation of *how the maxima are obtained*, not a new idea.

4 Data structures used

- None. Two local integers per index for the left and right maxima, and a running total.
- Space O(1), which is genuinely as good as the optimal solution — only the time is bad.

5 Step-by-step algorithm

1. Set `water = 0`.
2. Loop `i` from 0 to `n - 1`.
3. Scan `j` from 0 to `i` inclusive, tracking the maximum height. Call it `left_max`.
4. Scan `j` from `i` to `n - 1` inclusive, tracking the maximum height. Call it `right_max`.
5. Add `min(left_max, right_max) - height[i]` to `water`. Including `i` itself in both scans means this quantity is never negative, so no clamp is needed.
6. Return `water`.

6 Time & space complexity with explanation

Time: O(n²). Each of the n indices performs two scans that together cover the whole array, so roughly n work per index.

Space: O(1). Only a few integers.

7 Fully commented brute-force code

```
def trap_brute(height):
    """water[i] = min(maxLeft[i], maxRight[i]) - height[i], computed by rescanning."""
    n = len(height)
    water = 0
    for i in range(n):
        left_max = 0
        for j in range(i + 1):            # tallest bar at or left of i
            left_max = max(left_max, height[j])
        right_max = 0
        for j in range(i, n):             # tallest bar at or right of i
            right_max = max(right_max, height[j])
        # both scans include i itself, so this is never negative
        water += min(left_max, right_max) - height[i]
    return water
```

8 Limitation of brute force

The left maximum at index `i` differs from the left maximum at `i - 1` by at most one comparison — it is `max(previous_left_max, height[i])` — yet the brute force recomputes it from index 0 every single time. The same is true on the right. It is recomputing prefix and suffix maxima that could each be built in one pass.

The first fix is the obvious one: precompute two arrays, `left_max` and `right_max`, in two linear sweeps, then combine them in a third. That is O(n) time but costs O(n) space, and it is worth presenting as the intermediate step.

The second fix removes the arrays too. The formula only needs `min(left_max, right_max)`, and if I know that one side's maximum is already smaller than the other's, that side alone determines the water level — I never need the other side's exact value. That observation is what makes two converging pointers work in O(1) space.

9 Optimal approach in English

The two-pointer solution, the one to present. Start `lo` at the left end and `hi` at the right, tracking `left_max` and `right_max` as the tallest bars seen so far from each side. At each step, compare the two maxima and process the **smaller** side.

The reason this is valid: suppose `left_max < right_max`. Then for the position just right of `lo`, whatever the true right-hand maximum turns out to be, it is at least `right_max`, which is already greater than `left_max`. So `min(true_left, true_right)` is determined entirely by the left side, and I can safely add `left_max - height[lo]` without knowing anything more about the right. Symmetrically when the right side is smaller. Each step advances one pointer, so the pointers meet after n steps.

The monotonic-stack alternative, worth mentioning. Keep a stack of indices with decreasing heights. When a bar arrives that is taller than the top of the stack, that bar and the element below the popped index form the two walls of a horizontal basin whose floor is the popped bar. Compute `width × depth` for that layer and add it. The two versions fill water in different directions, and saying that difference out loud is the most memorable thing you can say about this problem: **the stack fills water horizontally, layer by layer; the two-pointer version fills it vertically, column by column.**

10 Optimal data structures

- **Two pointers plus two running maxima** — four integers, no arrays. O(1) space.
- The intermediate DP version instead uses **two arrays of length n**, `left_max` and `right_max`; same time, O(n) space. Present it as the stepping stone.
- The alternative uses a **monotonic decreasing stack of indices**, O(n) space, which is the same structure as problem 239 and the Largest Rectangle in Histogram family.

11 Step-by-step optimal algorithm

1. Return 0 if the array is empty.
2. Set `lo = 0`, `hi = n − 1`, `left_max = height[lo]`, `right_max = height[hi]`, `water = 0`.
3. While `lo < hi`: compare `left_max` and `right_max`.
4. If `left_max < right_max`, the left wall is the binding constraint, so advance `lo` by one, update `left_max` with the new bar, and add `left_max − height[lo]` to `water`. This is non-negative because `left_max` was just updated to include `height[lo]`.
5. Otherwise advance `hi` by one to the left, update `right_max`, and add `right_max − height[hi]`.
6. When the pointers meet, return `water`.
7. For the stack version: iterate over indices; while the stack is non-empty and the current bar is taller than the bar at the stack top, pop that index as the basin floor, and if the stack is now empty break because there is no left wall; otherwise compute `width = i − stack[−1] − 1` and `depth = min(height[stack[−1]], h) − height[floor]`, and add their product. Push the current index.

12 Fully commented optimal code

```
def trap(height):
    """WAY A -- two pointers, O(1) space. Process whichever side has the SMALLER
    running maximum: that side alone determines the water level there."""
    if not height:
        return 0

    lo, hi = 0, len(height) - 1
    left_max, right_max = height[lo], height[hi]
    water = 0

    while lo < hi:
        if left_max < right_max:
            # The right side already guarantees a wall taller than left_max,
            # so the level here is fixed by the left side alone.
            lo += 1
            left_max = max(left_max, height[lo])
            water += left_max - height[lo]      # never negative
        else:
            hi -= 1
            right_max = max(right_max, height[hi])
            water += right_max - height[hi]

    return water


def trap_stack(height):
    """WAY B -- monotonic decreasing stack of indices. Fills water in horizontal
    layers instead of vertical columns."""
    stack = []                              # indices, heights decreasing
    water = 0
    for i, h in enumerate(height):
        while stack and height[stack[-1]] < h:
            bottom = stack.pop()            # the floor of this basin
            if not stack:
                break                       # no left wall -> nothing can be trapped
            left = stack[-1]                # left wall
            width = i - left - 1
            depth = min(height[left], h) - height[bottom]
            water += width * depth          # one horizontal layer
        stack.append(i)
    return water


def trap_dp(height):
    """The intermediate step worth naming: precompute both maxima. O(n) time,
    O(n) space -- the version the two-pointer solution improves on."""
    if not height:
        return 0
    n = len(height)
    left_max = [0] * n
    right_max = [0] * n

    left_max[0] = height[0]
    for i in range(1, n):
        left_max[i] = max(left_max[i - 1], height[i])

    right_max[n - 1] = height[n - 1]
    for i in range(n - 2, -1, -1):
        right_max[i] = max(right_max[i + 1], height[i])

    return sum(min(left_max[i], right_max[i]) - height[i] for i in range(n))
```

+ **Optimal complexity, and what to say out loud**

Time: O(n) for all three optimal variants. The two-pointer version advances exactly one pointer per iteration and the pointers meet after n steps. The stack version pushes and pops each index at most once. The DP version makes three linear passes.

Space: O(1) for two pointers, **O(n)** for the stack, **O(n)** for the DP arrays. Lead with the two-pointer version precisely because of that column.

Say this out loud: "Water above column i is min of the left and right maxima, minus the column height. The brute force recomputes those maxima, which is O(n²). Two pointers derive them on the fly: I always advance the side with the smaller running maximum, because the other side is already guaranteed to be at least that tall, so the level there is fully determined. O(n) time, O(1) space."

Budget 45 minutes for this one in practice, and finish by naming the difference between the two methods — horizontal layers versus vertical columns. That sentence is what interviewers remember.

76 Minimum Window Substring ★★★ (stretch)

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Sliding window with have/need satisfaction counters · Optimal: **O(|s| + |t|) time, O(|s| + |t|) space**

1 Clarification communication

1. Does `t` have **multiplicity** — if `t` is `"aab"`, must the window contain at least two `a`s? That is the detail that decides the data structure.
2. If several minimum windows tie in length, may I return any of them, or specifically the leftmost?
3. If no valid window exists, I return the empty string?
4. May the window contain extra characters not in `t`? Yes, it need only *contain* `t`, not equal it.
5. Are `s` and `t` case-sensitive, and can `t` be longer than `s`, in which case the answer is immediately empty?

2 Understanding of the question

Given strings `s` and `t`, return the shortest substring of `s` that contains every character of `t` including repeats, or the empty string if there is none.

This is the hardest member of the sliding-window family, and the reason is the validity test. Naively, checking whether the window contains all of `t` means comparing two frequency maps, which is O(alphabet) on every single step. The idea that fixes it is to track **satisfaction** rather than raw counts: keep an integer `have` counting *how many distinct characters of `t` currently meet or exceed their required quota*, and a constant `need_count` equal to the number of distinct characters in `t`. The window is valid exactly when `have == need_count`, which is an O(1) test.

The window then follows the standard two-phase rhythm: grow the right edge until valid, then shrink the left edge while it stays valid, recording the best length each time.

3 Brute-force approach in English

Generate every substring of `s` and test each one. For each start index `i`, extend the end index `j`, maintaining a frequency map of the window's characters, and after each extension check whether the window covers `t` — that is, whether for every character in `t` the window's count is at least the required count. The first `j` at which the window becomes valid gives the shortest valid window for that start, so record its length and break to the next start. Track the global minimum.

Breaking at the first valid `j` is a legitimate optimisation: extending further from the same start can only make the window longer, never shorter.

4 Data structures used

- A **frequency map of the required characters** built once from `t`.
- A **frequency map of the current window**, rebuilt for each start index.
- Integers tracking the best length and the best start position.

5 Step-by-step algorithm

1. Build `need`, the frequency map of `t`. If `t` is empty or longer than `s`, return the empty string.
2. Set `best_len` to infinity and `best_start` to 0.
3. Loop `i` from 0 to `n − 1`, resetting an empty `window` map.
4. Loop `j` from `i` to `n − 1`, incrementing `window[s[j]]`.
5. Test coverage: for every character `c` in `need`, check `window.get(c, 0) >= need[c]`. This costs O(distinct characters in t).
6. If covered, and `j − i + 1 < best_len`, record `best_len` and `best_start = i`, then break to the next `i`.
7. Return the empty string if `best_len` is still infinity, otherwise the slice of length `best_len` starting at `best_start`.

6 Time & space complexity with explanation

Time: O(|s|² · |t|). There are O(|s|²) substrings, and each coverage test costs up to the number of distinct characters in `t`. Rebuilding the window map from scratch for each start adds another factor if done carelessly.

Space: O(|s| + |t|) for the two frequency maps.

7 Fully commented brute-force code

```
def minWindow_brute(s, t):
    """Every substring, tested for coverage. O(|s|^2 * |t|)."""
    from collections import Counter
    if not s or not t or len(t) > len(s):
        return ""

    need = Counter(t)
    n = len(s)
    best_len = float('inf')
    best_start = 0

    for i in range(n):
        window = {}                       # rebuilt for every start index
        for j in range(i, n):
            window[s[j]] = window.get(s[j], 0) + 1
            # full coverage check -- O(distinct chars in t) EVERY step
            if all(window.get(c, 0) >= need[c] for c in need):
                if j - i + 1 < best_len:
                    best_len = j - i + 1
                    best_start = i
                break                     # shortest window for THIS start
    return "" if best_len == float('inf') else s[best_start:best_start + best_len]
```

8 Limitation of brute force

Two layers of waste. The window map is discarded and rebuilt for every start index, even though moving the start forward by one only removes a single character. And the coverage test walks all of `t`'s distinct characters on every single extension, re-deriving a fact that changes by at most one character per step.

Both are fixed by tracking a **derivative** instead of recomputing a **state**. Rather than asking "is this window covered?" from scratch, maintain a counter of how many required characters are currently satisfied, and update it by ±1 exactly when a character crosses its required quota. Coverage then becomes a single integer comparison, and the window map survives across the whole pass rather than being rebuilt.

9 Optimal approach in English

Two pointers, one pass. Maintain `need`, the frequency map of `t`; `window`, the frequency map of the current window; `need_count`, the number of distinct characters in `t`; and `have`, how many of those are currently satisfied.

Extend `right`, incrementing `window[ch]`. If `ch` is required and its window count has just become **exactly equal** to its required count, increment `have` — that character is now satisfied. The equality is essential: using `>=` would increment `have` again on every additional copy and permanently corrupt the count.

While `have == need_count` the window is valid, so shrink from the left: record the length if it beats the best, then remove `s[left]` from the window and advance `left`. If the removed character was required and its count has **dropped below** its requirement, decrement `have`, which ends the shrinking phase. The condition here is strictly `<`, not `<=`, for the symmetric reason.

Record `best_start` and `best_len` rather than materialising the substring inside the loop; slicing on every improvement would add an O(|s|) cost inside an O(|s|) loop.

10 Optimal data structures

- A `Counter` **of** `t` giving the required multiplicity of each character.
- A **frequency map of the window**, maintained incrementally for the whole pass.
- Two integers, `have` and `need_count`, which reduce the validity test from O(alphabet) to O(1). This is the core idea of the problem.
- Two integers `best_len` and `best_start`, so the result is sliced exactly once at the end.

11 Step-by-step optimal algorithm

1. Return the empty string if either `s` or `t` is empty.
2. Build `need = Counter(t)` and set `need_count = len(need)`, the number of **distinct** required characters.
3. Initialise `window = {}`, `have = 0`, `left = 0`, `best_len = infinity`, `best_start = 0`.
4. Iterate `right` over `s` with `ch = s[right]`. Increment `window[ch]`.
5. If `ch` is in `need` and `window[ch] == need[ch]` exactly, increment `have`.
6. While `have == need_count`: if the current window is shorter than `best_len`, update `best_len` and `best_start`.
7. Then let `out = s[left]`, decrement `window[out]`, and if `out` is in `need` and `window[out] < need[out]`, decrement `have`, which will exit the shrink loop.
8. Advance `left` by one and repeat the while test.
9. After the main loop, return the empty string if `best_len` is still infinity, otherwise `s[best_start : best_start + best_len]`.

12 Fully commented optimal code

```
from collections import Counter

def minWindow(s, t):
    """Track SATISFACTION (have vs need_count), not raw counts, so the validity
    test is O(1) instead of O(alphabet)."""
    if not t or not s:
        return ""

    need = Counter(t)
    need_count = len(need)               # number of DISTINCT chars still to satisfy
    window = {}
    have = 0
    left = 0
    best_len = float('inf')
    best_start = 0

    for right, ch in enumerate(s):
        window[ch] = window.get(ch, 0) + 1
        # *** == not >= : otherwise every extra copy re-increments `have` ***
        if ch in need and window[ch] == need[ch]:
            have += 1

        while have == need_count:        # VALID -> try to shrink from the left
            if right - left + 1 < best_len:
                best_len = right - left + 1
                best_start = left        # store the index; slice once at the end

            out = s[left]
            window[out] -= 1
            # *** < not <= : we only lose satisfaction when we drop BELOW need ***
            if out in need and window[out] < need[out]:
                have -= 1                # window just became invalid; stop shrinking
            left += 1

    return "" if best_len == float('inf') else s[best_start:best_start + best_len]
```

+ **Optimal complexity, and what to say out loud**

Time: O(|s| + |t|). Building `need` is O(|t|). In the main loop `right` advances |s| times and `left` only moves forward, so at most |s| times overall; every operation inside is O(1) because the validity test is an integer comparison.

Space: O(|s| + |t|) for the two maps, bounded in practice by the alphabet size.

Say this out loud: "Sliding window, but the trick is what I track. Comparing two frequency maps on every step would be O(alphabet); instead I keep a count of how many required characters are currently satisfied, and I only change it when a character crosses its quota. That makes validity O(1), so the whole thing is linear."

The three gotchas, all worth stating before you type them: `== need[ch]` on the way in, not `>=`; `< need[out]` on the way out, not `<=`; and store `best_start` rather than slicing inside the loop.

239 Sliding Window Maximum ★★ (stretch)

Day 2 · Wed 26 Aug · Round Type B (reduce an O(n²) solution) · Pattern: Monotonic decreasing deque of indices · Optimal: **O(n) time, O(k) space**

1 Clarification communication

1. The window has fixed size `k` and slides one position at a time, so the output has `n - k + 1` entries?
2. Is `k` guaranteed to be between 1 and `n`? If `k` is 1 the answer is the array itself, which is a useful sanity check.
3. Can the values be negative or duplicated? Duplicates matter for whether I pop on `<` or `<=`, so I want to be explicit about that.
4. Do you want the maximum values themselves, or their indices?
5. Is a heap-based O(n log n) acceptable, or are you looking for the O(n) solution? I will go straight for O(n) but I will mention the heap as the intermediate step.

2 Understanding of the question

Given an array and a window size `k`, return the maximum of every contiguous window of size `k` as the window slides from left to right.

The insight that makes a linear solution possible is a **dominance** argument. Suppose index `j` is to the right of index `i` and `nums[j] >= nums[i]`. Then `nums[i]` can never again be the maximum of any window: any future window containing `i` must also contain `j`, since `j` is further right and windows only move right, and `j` holds a value at least as large. So `nums[i]` is permanently useless and can be discarded the moment `j` arrives.

Discarding dominated elements leaves behind a sequence of indices whose values are strictly decreasing, with the current window maximum always at the front. That structure is a **monotonic deque**, and it needs both ends: new elements are compared and appended at the back, while indices that slide out of the window are removed at the front.

3 Brute-force approach in English

For each of the `n - k + 1` window positions, scan all `k` elements of that window and take the largest. Append it to the output and slide one position right.

Directly correct and trivial to write, which makes it a good thing to state in one sentence before immediately noting its cost. The intermediate improvement worth naming is a max-heap of `(value, index)` pairs with lazy deletion: push each new element, and before reading the top, pop any entries whose index has fallen out of the window. That gets to O(n log n) but still is not linear.

4 Data structures used

- None beyond the output list. A nested loop and a local maximum.
- The heap variant would add a max-heap holding up to n entries.

5 Step-by-step algorithm

1. Return an empty list if the array is empty or `k` is 0.
2. Loop `i` from 0 to `n - k` inclusive; this is the start of each window.
3. Set `best` to the smallest possible value.
4. Loop `j` from `i` to `i + k - 1`, updating `best` with `nums[j]`.
5. Append `best` to the output.
6. Return the output.

6 Time & space complexity with explanation

Time: O(n · k). There are n − k + 1 windows and each is scanned in full. When `k` is proportional to `n` — for example k = n/2 — this is O(n²).

Space: O(1) beyond the required output.

7 Fully commented brute-force code

```
def maxSlidingWindow_brute(nums, k):
    """Rescan every window from scratch. O(n*k)."""
    n = len(nums)
    if n == 0 or k == 0:
        return []
    out = []
    for i in range(n - k + 1):            # each window start
        best = nums[i]
        for j in range(i, i + k):         # rescan all k elements
            if nums[j] > best:
                best = nums[j]
        out.append(best)
    return out
```

8 Limitation of brute force

Consecutive windows share `k - 1` of their `k` elements, yet every maximum is recomputed from nothing. Sliding by one position removes exactly one element on the left and adds exactly one on the right, so almost all the comparison work is repeated.

A heap fixes the additions cheaply but handles the removals badly: you cannot delete an arbitrary element from a binary heap in better than O(n), so the standard workaround is lazy deletion, which leaves the complexity at O(n log n) and the heap potentially holding all n elements.

The dominance argument is what gets to linear: most elements never need to be stored at all, because a later, larger element makes them permanently irrelevant. Keeping only the non-dominated candidates gives a structure where the maximum is always at the front, with no search and no log factor.

9 Optimal approach in English

Maintain a deque of **indices** whose corresponding values are in decreasing order. For each new index `i`:

- **Pop from the back** while the value at the back index is less than `nums[i]`. Those elements are dominated by `i` — they are older and smaller — so they can never be a maximum again.
- **Append** `i` to the back, preserving the decreasing order.
- **Pop from the front** if the front index is `<= i - k`, meaning it has slid out of the current window. Evicting by **index**, not by value, is essential; a value-based test cannot distinguish two equal values at different positions.
- Once `i >= k - 1`, a full window exists, so append `nums[dq[0]]` to the output — the front index always holds the largest value in the current window.

Storing indices rather than values is what makes the window-eviction test possible at all, and it is the design decision to state explicitly.

10 Optimal data structures

- A **deque of indices**, kept monotonically decreasing by value. Both ends are used, which is exactly why a deque rather than a stack or a queue: dominated elements leave from the back, expired elements leave from the front.
- Indices rather than values, so expiry can be tested positionally.
- The output list.

11 Step-by-step optimal algorithm

1. Create an empty deque and an empty output list.
2. Iterate over the array with index `i` and value `x`.
3. While the deque is non-empty and `nums[deque[-1]] < x`, pop from the back. Those older, smaller elements are permanently dominated by `x`.
4. Append `i` to the back of the deque.
5. If `deque[0] <= i - k`, the front index has fallen out of the window, so pop it from the front. A single `if` suffices, because at most one index expires per step.
6. If `i >= k - 1`, the first full window has been formed, so append `nums[deque[0]]` to the output.
7. Return the output.
8. On duplicates: popping on strict `<` keeps equal values in the deque, which is safe because the older duplicate expires first and the newer one remains to serve later windows.

12 Fully commented optimal code

```
from collections import deque

def maxSlidingWindow(nums, k):
    """Monotonic decreasing deque of INDICES. The front is always the maximum
    of the current window."""
    dq = deque()                          # indices; nums[dq] is decreasing
    out = []

    for i, x in enumerate(nums):
        # 1. Anything smaller than x, and older than x, can never be a max again.
        while dq and nums[dq[-1]] < x:
            dq.pop()

        # 2. x is now a live candidate.
        dq.append(i)

        # 3. Evict the front BY INDEX if it has slid out of the window.
        #    (By index, not by value -- equal values must stay distinguishable.)
        if dq[0] <= i - k:
            dq.popleft()

        # 4. Once a full window exists, the front holds its maximum.
        if i >= k - 1:
            out.append(nums[dq[0]])

    return out
```

+ Optimal complexity, and what to say out loud

Time: O(n). This needs the amortised argument: each index is appended to the deque exactly once and removed at most once, either from the back as dominated or from the front as expired. So although the inner `while` can pop several items in one iteration, the total number of pops across the whole run is bounded by n. Two operations per element, so linear.

Space: O(k). The deque never holds more than one window's worth of indices, since older ones are evicted as they expire.

Say this out loud: "Monotonic decreasing deque of indices. If a later element is at least as large as an earlier one, the earlier one can never be a maximum again, so I discard it. That leaves the front of the deque as the current window's maximum. Each index is pushed and popped at most once, so the total is O(n) even though one step can pop many items."

The gotcha: evict the front by index, not by value. **The related problems:** this same monotonic-stack-or-deque structure solves Next Greater Element, Daily Temperatures, Largest Rectangle in Histogram, and the stack version of Trapping Rain Water.

56 Merge Intervals ★★ Amazon favourite

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Sort by start, then one linear merge pass · Optimal: **O(n log n) time, O(n) space**

1 Clarification communication

1. Are the intervals **given sorted**? If they already arrive sorted by start, the whole thing is a single O(n) pass and I should say so.
2. Do intervals that merely **touch** count as overlapping — does `[1,4]` merge with `[4,5]`? That decides whether my comparison is `<=` or `<`. The standard version says yes, they merge.
3. Can an interval be fully contained inside another, like `[1,10]` and `[2,3]`? This is the case that breaks the naive merge, so I want it confirmed.
4. May I modify the input list, or should I sort a copy?
5. Are the endpoints integers, and is `start <= end` guaranteed for every interval?

2 Understanding of the question

Given a list of intervals, merge all overlapping ones and return the resulting non-overlapping list covering the same total span.

The reason sorting by **start** is the right move: after sorting, if the current interval does not overlap the last merged interval, then no later interval can overlap that merged one either — every later interval starts at or after the current one. That single fact turns a global, all-pairs problem into a purely local one, where each interval need only be compared against the single most recent output entry. It is the interval equivalent of the two-pointer trick: sort so that a local decision becomes globally correct.

3 Brute-force approach in English

Repeatedly look for any overlapping pair anywhere in the list and fuse it, until no overlapping pair remains. Concretely: scan every pair `(i, j)`; if they overlap, replace them with a single interval spanning `min` of the starts to `max` of the ends, restart the scan, and keep going until a full pass finds nothing to merge.

It is correct because merging two overlapping intervals never destroys coverage and never creates new overlap that was not already implied, and the process terminates because each merge reduces the list length by one.

4 Data structures used

- A mutable list of intervals, repeatedly rewritten.
- A `changed` flag to know when the process has stabilised.

5 Step-by-step algorithm

1. Set `changed = True`.
2. While `changed`: set `changed = False`.
3. Scan every pair of distinct indices `(i, j)` in the current list.
4. Two intervals overlap when `a.start <= b.end` **and** `b.start <= a.end`.
5. On finding an overlapping pair, replace interval `i` with `[min(a.start, b.start), max(a.end, b.end)]`, delete interval `j`, set `changed = True`, and break out to restart the scan on the modified list.
6. When a complete scan finds no overlapping pair, return the list.

6 Time & space complexity with explanation

Time: O(n³) in the worst case. Each full pass examines O(n²) pairs, and there can be up to n−1 merges, each of which restarts the scanning. Deleting from a Python list is itself O(n), adding another factor.

Space: O(n) for the working list.

7 Fully commented brute-force code

```
def merge_brute(intervals):
    """Repeatedly fuse any overlapping pair until a full pass finds none."""
    out = [list(iv) for iv in intervals]  # work on a copy
    changed = True
    while changed:
        changed = False
        n = len(out)
        for i in range(n):
            for j in range(i + 1, n):
                a, b = out[i], out[j]
                # overlap test for two unordered intervals
                if a[0] <= b[1] and b[0] <= a[1]:
                    out[i] = [min(a[0], b[0]), max(a[1], b[1])]
                    out.pop(j)            # O(n) deletion
                    changed = True
                    break
            if changed:
                break                     # restart the scan on the modified list
    return out
```

8 Limitation of brute force

Because the intervals are in arbitrary order, an interval could overlap anything anywhere in the list, so every merge forces a fresh all-pairs search. The algorithm never accumulates any knowledge about where overlapping partners might be.

Sorting by start supplies exactly that knowledge. Once sorted, overlap can only ever occur with the immediately preceding merged interval, because every subsequent interval starts later still. The all-pairs search collapses to a single comparison per interval, and the whole algorithm becomes one pass after the sort.

9 Optimal approach in English

Sort the intervals by start. Walk them in order, maintaining an output list whose last element is the interval currently being extended.

For each interval `[start, end]`: if the output is non-empty and `start <= out[-1][1]`, this interval begins before the last merged one has finished, so they overlap and I extend the last one's end. Otherwise there is a genuine gap, so I append `[start, end]` as a new interval.

The critical line is the extension: it must be `out[-1][1] = max(out[-1][1], end)`, **not** `= end`. A fully contained interval such as `[2, 3]` arriving while `[1, 10]` is being extended would otherwise shrink the merged interval from `[1, 10]` to `[1, 3]` and silently lose coverage. That is the one bug that decides this problem, and it is worth naming as you write the line.

Using `<=` rather than `<` means touching intervals like `[1,4]` and `[4,5]` merge, which matches the standard requirement clarified up front.

10 Optimal data structures

- The **sorted list** itself; sorting by start is the enabling step and the only reason a local comparison is sufficient.
- An **output list**, whose last element acts as the running accumulator. No stack, no map, no auxiliary structure — the output doubles as the working state.

11 Step-by-step optimal algorithm

1. Sort `intervals` by start using `key=lambda x: x[0]`.
2. Create an empty output list.
3. For each `[start, end]` in sorted order:
4. If the output is non-empty and `start <= out[-1][1]`, the intervals overlap or touch, so set `out[-1][1] = max(out[-1][1], end)`. The `max` is what protects against a fully contained interval.
5. Otherwise append `[start, end]` to the output as a fresh interval.
6. Return the output.

12 Fully commented optimal code

```
def merge(intervals):
    """Sort by START, then a single linear pass extending the last output entry."""
    if not intervals:
        return []

    intervals.sort(key=lambda x: x[0])    # sorting by start is the whole idea
    out = []

    for start, end in intervals:
        # `<=` so touching intervals ([1,4] and [4,5]) also merge
        if out and start <= out[-1][1]:
            # *** max(), not just `end` ***
            # [1,10] followed by [2,3] must stay [1,10], not shrink to [1,3].
            out[-1][1] = max(out[-1][1], end)
        else:
            out.append([start, end])      # a genuine gap: start a new interval

    return out
```

+ Optimal complexity, and what to say out loud

Time: O(n log n), entirely dominated by the sort. The merging pass itself is O(n), with one comparison and at most one update per interval.

Space: O(n) for the output, plus O(log n) to O(n) for the sort's internal working memory depending on the implementation. If the input arrives pre-sorted, the whole thing is O(n) time and O(1) extra space beyond the output.

Say this out loud: "Sorting by start means an interval can only overlap the most recent merged one, because everything after it starts later. So one comparison per interval suffices. Sort dominates at O(n log n), the merge pass is O(n)."

The gotcha: `max(out[-1][1], end)`, demonstrated with `[[1,10],[2,3]]`. **The family:** Insert Interval is the same pass with a pre-sorted list; Non-overlapping Intervals and Meeting Rooms II are the same shape but sort by **end** instead, because maximising how many intervals fit is a different objective from merging them.

253 Meeting Rooms II ★★ Amazon favourite

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Min-heap of end times, or a sweep line · Optimal: **O(n log n) time, O(n) space**

1 Clarification communication

1. If one meeting ends at 10 and another starts at 10, can they share a room? I will assume yes — end times are exclusive — and that determines whether my comparison is `<=` or `<`.
2. Am I returning just the **number** of rooms, or an actual room assignment for each meeting? The heap version extends naturally to the assignment if you need it.
3. Are the intervals sorted on input? I will assume not and sort them myself.
4. Can meetings have zero duration, where start equals end?
5. How large is n? That decides whether an O(n log n) sort is acceptable, which it almost always is.

2 Understanding of the question

Given a list of meeting intervals, return the minimum number of conference rooms required so that no two meetings in the same room overlap.

The reframing that makes the answer obvious: the number of rooms needed is exactly the **maximum number of meetings that are simultaneously in progress** at any instant. If four meetings overlap at 10:15, you need four rooms at that moment; and conversely, if the peak concurrency is four, four rooms always suffice, because a greedy assignment can always place a meeting into some room whose previous occupant has already finished.

So this is not really a scheduling problem, it is a **peak concurrency** problem. Both standard solutions compute that peak; they just track it differently.

3 Brute-force approach in English

Compute the concurrency at each meeting's start time directly. For every meeting `i`, count how many meetings — including `i` itself — are in progress at the instant `i` begins: that is, every meeting `j` with `start[j] <= start[i] < end[j]`. The answer is the maximum of those counts.

This is correct because the concurrency of a set of intervals can only increase at a start point, so the global maximum is always attained at the start of some meeting. It is therefore sufficient to sample only at the n start times, rather than at every possible instant.

4 Data structures used

- None beyond two loop counters and a running maximum. O(1) extra space.

5 Step-by-step algorithm

1. Return 0 for an empty list.
2. Set `best = 0`.
3. Loop `i` over all meetings; let `t = start[i]`, the instant to sample.
4. Set `count = 0`, then loop `j` over all meetings.
5. If `start[j] <= t < end[j]`, meeting `j` is in progress at time `t`, so increment `count`. The strict `<` on the end is what implements the rule that a meeting ending at `t` frees its room for a meeting starting at `t`.
6. Update `best` with `count`.
7. Return `best`.

6 Time & space complexity with explanation

Time: O(n²). For each of the n sample instants, all n meetings are tested for containment.

Space: O(1). Nothing is stored beyond counters.

Note that this is a genuinely better brute force than sampling every minute of the day, which would be O(n · T) and would break entirely if the times were not small integers.

7 Fully commented brute-force code

```
def minMeetingRooms_brute(intervals):
    """Concurrency can only rise at a start time, so sample at the n starts.
    O(n^2), O(1) space."""
    if not intervals:
        return 0
    best = 0
    for i in range(len(intervals)):
        t = intervals[i][0]              # sample instant: when meeting i begins
        count = 0
        for j in range(len(intervals)):
            # `< end` (not <=): a meeting ending at t frees the room at t
            if intervals[j][0] <= t < intervals[j][1]:
                count += 1
        best = max(best, count)
    return best
```

8 Limitation of brute force

Every sample instant re-examines all n meetings from scratch, even though moving from one start time to the next changes the set of in-progress meetings by only a handful of entries — one meeting begins, and some number end.

Both optimal solutions exploit exactly that. Once the meetings are processed in chronological order, the running concurrency can be updated **incrementally**: add one when a meeting starts, subtract one when a meeting ends. All the redundant recounting disappears, and the only cost left is putting the events in time order, which is the sort.

9 Optimal approach in English

Way A, the min-heap of end times — present this one. Sort the meetings by start time. Keep a min-heap holding the end times of the meetings currently occupying rooms; the heap's size is therefore the number of rooms in use, and its smallest element is the earliest moment any room becomes free. For each meeting in start order: if the heap is non-empty and its minimum end time is at or before this meeting's start, a room has freed up, so pop it. Then push this meeting's end time. The answer is the final heap size, which equals the maximum it ever reached, because a pop is always immediately followed by a push.

Way B, the sweep line — the elegant one. Extract all start times into one sorted array and all end times into another, then decouple them entirely: the identity of which start belongs to which end is irrelevant to counting concurrency. Walk the two arrays together with two pointers. If the next start comes before the next end, a meeting is beginning, so increment the room count and record the new maximum. Otherwise a meeting is ending, so decrement. The maximum reached is the answer.

Both are O(n log n). The heap generalises better if you also need to know *which* room each meeting was assigned to; the sweep line is easier to explain and has a smaller constant factor.

10 Optimal data structures

- **Way A:** a **min-heap** of end times. A min-heap because the only question ever asked is "what is the earliest a room becomes free?", and that is precisely the operation a min-heap answers in O(1) with O(log n) removal.
- **Way B:** two **sorted arrays**, one of starts and one of ends, plus two integer pointers and two counters. No heap at all.
- Both require sorting, which is where the log factor comes from.

11 Step-by-step optimal algorithm

1. **Way A:** return 0 if the list is empty. Sort the meetings by start time.
2. Create an empty min-heap. For each `[start, end]` in order:
3. If the heap is non-empty and `heap[0] <= start`, the earliest-finishing meeting is already over, so pop it: that room is reusable.
4. Push `end` onto the heap, occupying a room for this meeting.
5. After all meetings, return `len(heap)` — the number of rooms in simultaneous use at the peak.
6. **Way B:** build `starts` as the sorted list of all start times and `ends` as the sorted list of all end times.
7. Set `rooms = 0`, `best = 0`, and two pointers `s = 0`, `e = 0`. While `s < len(starts)`: if `starts[s] < ends[e]`, a meeting begins, so increment `rooms`, update `best`, and advance `s`; otherwise a meeting ends, so decrement `rooms` and advance `e`.
8. Return `best`.

12 Fully commented optimal code

```
import heapq

def minMeetingRooms(intervals):
    """WAY A -- min-heap of end times. Heap size == rooms currently in use."""
    if not intervals:
        return 0

    intervals.sort(key=lambda x: x[0])    # process meetings in start order
    heap = []                             # end times of ongoing meetings

    for start, end in intervals:
        # heap[0] is the EARLIEST end time == the soonest a room frees up.
        if heap and heap[0] <= start:
            heapq.heappop(heap)           # that room is free again -- reuse it
        heapq.heappush(heap, end)         # this meeting occupies a room

    return len(heap)                      # peak concurrency == rooms needed


def minMeetingRooms_sweep(intervals):
    """WAY B -- sweep line. Which start pairs with which end is irrelevant to
    counting how many meetings are simultaneously in progress."""
    if not intervals:
        return 0

    starts = sorted(i[0] for i in intervals)
    ends = sorted(i[1] for i in intervals)

    rooms = best = 0
    s = e = 0
    while s < len(starts):
        if starts[s] < ends[e]:           # a meeting begins before the next one ends
            rooms += 1
            best = max(best, rooms)
            s += 1
        else:                             # a meeting ends: free a room
            rooms -= 1
            e += 1
    return best
```

+ Optimal complexity, and what to say out loud

Time: O(n log n) for both. Way A sorts once and performs at most n pushes and n pops, each O(log n). Way B performs two sorts and then a single linear two-pointer sweep.

Space: O(n) — the heap can hold every meeting when they all overlap; the sweep line stores two arrays of n elements.

Say this out loud: "The number of rooms is the maximum number of meetings in progress at once. I sort by start and keep a min-heap of end times, so the heap size is the rooms in use and the heap top is the soonest a room frees up. n pushes and pops at O(log n) each, plus the sort: O(n log n)."

The related sorting rule, worth stating: sort by **start** to merge intervals; sort by **end** to maximise how many non-overlapping intervals fit. Meeting Rooms I — can one person attend all meetings — is the same sort with a single adjacent-pair comparison.

215 Kth Largest Element in an Array ★

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Min-heap of size k (alternative: Quickselect) · Optimal: **O(n log k) time, O(k) space**

1 Clarification communication

1. Is this the kth largest **by value** or the kth **distinct** value? With `[3, 3, 2]` and `k = 2`, by value the answer is 3, distinct it would be 2. The standard problem means by value.
2. Is `k` guaranteed to satisfy `1 <= k <= n`?
3. May I modify or sort the input array? Quickselect partitions in place, so this matters if I choose that route.
4. Is the data a fixed array, or is it **streaming**? If elements arrive over time, the heap is the only viable answer and Quickselect is not applicable — this is the follow-up they usually want.
5. Are the values ordinary 32-bit integers, possibly with duplicates?

2 Understanding of the question

Return the kth largest element of an array, counting duplicates as separate elements.

The observation that drives the efficient solution: **I do not need the array sorted.** I need one specific order statistic. Fully sorting produces n−1 pieces of ordering information when I only need one, and paying O(n log n) for that is over-solving the problem.

The inversion that people find counter-intuitive and that is worth stating plainly: to keep track of the k **largest** elements, use a **min**-heap. The heap holds the k best candidates seen so far, and its root is the *weakest* of them — which is precisely the element to evict when a better one arrives, and, at the end, precisely the kth largest.

3 Brute-force approach in English

Find and remove the maximum, k times. Scan the array for the largest remaining value, mark it as used, and repeat. After k rounds, the last value found is the kth largest.

The simpler alternative is to sort descending and index at `k - 1`, which is one line and O(n log n). Both are worth naming: the repeated-scan version is the true brute force, and the sort is the obvious-but-wasteful improvement that motivates the heap.

4 Data structures used

- A copy of the array, or a `used` set of indices, so that removed maxima are not found again.
- For the sort variant: whatever the sort allocates.

5 Step-by-step algorithm

1. Make a working copy of the array so the input is not destroyed.
2. Repeat `k` times: scan the whole copy tracking the index of the largest value.
3. Record that value, then remove it from the copy, or mark its index as used.
4. After `k` rounds, return the last value recorded.
5. The one-line alternative: `return sorted(nums, reverse=True)[k - 1]`, which is O(n log n) and worth stating as the baseline you would improve on.

6 Time & space complexity with explanation

Time: O(n · k). Each of the k rounds scans up to n elements. When k is close to n — the median, say — this is O(n²). The sorting variant is O(n log n), better for large k but still more than necessary.

Space: O(n) for the working copy.

7 Fully commented brute-force code

```
def findKthLargest_brute(nums, k):
    """Remove the maximum k times. O(n*k)."""
    arr = list(nums)                    # work on a copy
    value = None
    for _ in range(k):
        best_i = 0
        for i in range(1, len(arr)):    # full scan for the current maximum
            if arr[i] > arr[best_i]:
                best_i = i
        value = arr.pop(best_i)         # remove it so it is not found again
    return value


def findKthLargest_sort(nums, k):
    """The obvious baseline: O(n log n), computing far more order information
    than the question actually asks for."""
    return sorted(nums, reverse=True)[k - 1]
```

8 Limitation of brute force

The repeated-scan version re-examines the entire array on every round, having learned nothing from the previous rounds. The sorting version has the opposite problem: it computes the complete ordering of all n elements when only a single order statistic was requested, and most of that work is discarded.

The right frame is that this is a **top-K** question, and top-K questions never need the whole ordering. They need a bounded structure that holds only the k best candidates seen so far and can cheaply identify and evict the weakest of them. That structure is a heap of size k, and bounding the heap at k is what turns log n into log k.

9 Optimal approach in English

Maintain a **min-heap containing at most k elements**, holding the k largest values seen so far. Push each element of the array. Whenever the heap grows past size k, pop the root, which is the smallest element in the heap and therefore the weakest candidate — it cannot be among the k largest overall if k better ones exist. After the whole array is processed, the heap contains exactly the k largest elements, and its root is the smallest of those, which is the kth largest overall.

The push-then-pop ordering keeps the code to three lines and is correct: pushing first temporarily makes the heap size k+1, and popping the minimum then discards whichever element — possibly the one just pushed — is genuinely the weakest.

The alternative to name is **Quickselect**: partition the array around a pivot as in quicksort, but recurse into only the side containing the target position. That gives O(n) average time, with an $O(n^2)$ worst case that randomising the pivot makes vanishingly unlikely. Quickselect is faster on a fixed in-memory array; the heap is the right answer when the data streams, because it never needs to see all n elements at once.

10 Optimal data structures

- A **min-heap bounded at size k**. Min, not max, because the operation performed constantly is "evict the weakest of my current best k", and a min-heap exposes that element at its root in O(1). Bounded at k, because that is what makes each operation O(log k) rather than O(log n) and the space O(k) rather than O(n).
- Python's `heapq` is a min-heap natively, which is exactly what is wanted here — no negation tricks required.

11 Step-by-step optimal algorithm

1. Create an empty heap.
2. For each value `x` in the array: push `x` onto the heap.
3. If the heap size now exceeds `k`, pop the root. The root is the smallest element currently held, so discarding it keeps the k largest seen so far.
4. After processing every element, the heap holds exactly the k largest values.
5. Return `heap[0]`, the smallest element of that set, which is the kth largest overall.
6. The library shortcut to mention: `heapq.nlargest(k, nums)[-1]`, which does the same thing internally.

12 Fully commented optimal code

```
import heapq

def findKthLargest(nums, k):
    """Min-heap of size k. The root is the weakest of the current best k,
    so at the end it IS the kth largest."""
    h = []
    for x in nums:
        heapq.heappush(h, x)
        if len(h) > k:
            heapq.heappop(h)              # evict the SMALLEST -> keeps the k largest
    return h[0]                           # smallest of the k largest


import random

def findKthLargest_quickselect(nums, k):
    """O(n) average. Partition like quicksort, but recurse into ONE side only."""
    arr = list(nums)
    target = len(arr) - k                 # kth largest == index (n-k) when sorted asc
    lo, hi = 0, len(arr) - 1

    while lo <= hi:
        pivot_i = random.randint(lo, hi)  # randomise to dodge the O(n^2) worst case
        arr[pivot_i], arr[hi] = arr[hi], arr[pivot_i]
        pivot = arr[hi]

        store = lo
        for i in range(lo, hi):           # everything < pivot moves to the left
            if arr[i] < pivot:
                arr[i], arr[store] = arr[store], arr[i]
                store += 1
        arr[store], arr[hi] = arr[hi], arr[store]   # pivot lands at its final index

        if store == target:
            return arr[store]
        elif store < target:
            lo = store + 1                # recurse right only
        else:
            hi = store - 1                # recurse left only
```

+ Optimal complexity, and what to say out loud

Time: O(n log k). Each of the n elements causes one push and possibly one pop, and every heap operation on a heap of size k costs O(log k). When k is small this is effectively linear, and it is strictly better than the O(n log n) sort.

Space: O(k) — the heap never exceeds k+1 elements.

Quickselect: O(n) average, O(n²) worst case, O(1) extra space if partitioning in place. The average case comes from the recurrence T(n) = T(n/2) + O(n), which sums to 2n.

Say this out loud: "A min-heap of size k for the k *largest* — the root is the weakest of my current best, so it is the one to evict, and at the end it is the answer. n operations at O(log k) each. Sorting would be O(n log n) and computes far more than I need; Quickselect is O(n) average but needs the whole array in memory, so the heap is the right answer if the data is streaming."

The rule to memorise: K largest ⇒ **min**-heap of size K. K smallest ⇒ **max**-heap of size K.

973 K Closest Points to Origin ★ Amazon favourite

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Max-heap of size k by distance (alternative: Quickselect) · Optimal: **O(n log k) time, O(k) space**

1 Clarification communication

1. Distance is the Euclidean distance from the origin, so `sqrt(x² + y²)`? I will note immediately that I can skip the square root, since it is monotonic and I only ever compare distances.
2. If several points tie at the same distance, may I return any of them?
3. Does the output order matter, or is any order of the k closest acceptable?
4. Is `k` guaranteed to be at most the number of points?
5. Can coordinates be negative or zero? Squaring handles both, so it makes no difference to the method.

2 Understanding of the question

Given a list of points on a plane and an integer `k`, return the `k` points closest to the origin.

This is the same top-K shape as problem 215, with two differences worth naming. First, the ordering key is a **computed** value rather than the element itself, so I choose the comparison function. Second, the direction is inverted: I want the k **smallest** distances, so by the top-K rule I need a **max**-heap of size k, whose root is the *worst* of my current best and therefore the one to evict.

The small optimisation that interviewers listen for: comparing `sqrt(x² + y²)` and comparing `x² + y²` produce identical orderings, because the square root is monotonically increasing on non-negative inputs. So I skip the square root entirely — it saves n floating-point operations and avoids any precision issues in the comparisons.

3 Brute-force approach in English

Compute the squared distance for every point, sort the points by that value, and return the first k. Alternatively, and more literally brute-force: scan the whole list k times, each time finding and removing the point with the smallest remaining distance.

The sort version is one line and gets the right answer; it is the natural baseline. The repeated-scan version is what you would write with no data structures at all, and it is worth having in mind because its cost, O(n·k), is what the heap improves on when k is large.

4 Data structures used

- A list of `(distance, point)` pairs, or a working copy of the points list.
- No heap. The sort does all the work.

5 Step-by-step algorithm

1. For each point `(x, y)`, compute `d = x*x + y*y`, skipping the square root because it does not change the ordering.
2. Sort the points by `d` ascending.
3. Return the first `k` points of the sorted list.
4. The repeated-scan variant: repeat k times — scan all remaining points for the minimum `d`, append that point to the output, and remove it from the working list.

6 Time & space complexity with explanation

Time: O(n log n) for the sort version, dominated entirely by sorting; O(n · k) for the repeated-scan version, which degrades to O(n²) when k is proportional to n.

Space: O(n) to hold the distances or the working copy.

7 Fully commented brute-force code

```
def kClosest_brute(points, k):
    """Sort everything by squared distance, take the first k. O(n log n)."""
    # No sqrt: it is monotonic, so it never changes the ordering.
    return sorted(points, key=lambda p: p[0] * p[0] + p[1] * p[1])[:k]


def kClosest_scan(points, k):
    """The true brute force: find the closest remaining point, k times. O(n*k)."""
    remaining = list(points)
    out = []
    for _ in range(k):
        best_i = 0
        for i in range(1, len(remaining)):
            di = remaining[i][0] ** 2 + remaining[i][1] ** 2
            db = remaining[best_i][0] ** 2 + remaining[best_i][1] ** 2
            if di < db:
                best_i = i
        out.append(remaining.pop(best_i))
    return out
```

8 Limitation of brute force

Sorting orders all n points when only the k smallest are wanted, and the relative order among the n−k discarded points is computed and then thrown away. The repeated-scan version has the opposite failing: it never accumulates information, so every round starts over.

The correct framing is again top-K: I only ever need to hold k candidates and to know which of them is currently the worst, so that an incoming better point can displace it. That is a size-k heap, and bounding it at k reduces both the per-operation cost to O(log k) and the memory to O(k) — which also means the algorithm works when the points arrive as a stream too large to store.

9 Optimal approach in English

Maintain a **max-heap of size k** keyed on squared distance. Python's `heapq` only provides a min-heap, so I obtain max-heap behaviour by pushing the **negated** distance: the largest true distance becomes the smallest stored value and therefore sits at the root.

For each point, compute `d = x*x + y*y` and push `(-d, x, y)`. If the heap then exceeds size k, pop the root, which is the point with the **largest** true distance among the candidates — the worst one, correctly discarded. After every point is processed, the heap holds exactly the k closest, and I strip out the coordinates.

Two details worth narrating. Skipping the square root is safe because it is monotonic. And storing `x` and `y` inside the tuple means comparisons fall through to them when two distances tie, which is harmless for integers — but with unorderable payloads it would raise a TypeError, exactly the trap that problem 23 turns on.

As with 215, **Quickselect** is the O(n)-average alternative: partition on squared distance until the pivot lands at index k, at which point the first k entries are the answer in any order.

10 Optimal data structures

- A **max-heap of size k**, simulated with `heapq` by negating the key. Max, not min, because I am collecting the k *smallest* distances and therefore need instant access to the largest one I am currently holding.
- Tuples of `(-d, x, y)` so the heap orders by distance while carrying the point with it.
- Squared distance as the key, never the square root.

11 Step-by-step optimal algorithm

1. Create an empty heap.
2. For each point `(x, y)`: compute `d = x*x + y*y`.
3. Push the tuple `(-d, x, y)`. Negating turns the min-heap into a max-heap on true distance.
4. If the heap size now exceeds `k`, pop the root, which holds the largest true distance and is therefore the worst current candidate.
5. After all points, the heap holds the k closest. Return `[[x, y] for _, x, y in heap]`.
6. Mention the alternative: `heapq.nsmallest(k, points, key=...)`, and Quickselect for O(n) average time.

12 Fully commented optimal code

```
import heapq

def kClosest(points, k):
    """Max-heap of size k, keyed on NEGATED squared distance."""
    heap = []
    for x, y in points:
        d = x * x + y * y                # no sqrt: monotonic, so ordering is identical
        heapq.heappush(heap, (-d, x, y))  # negate -> heapq's min-heap acts as a max-heap
        if len(heap) > k:
            heapq.heappop(heap)          # evict the FARTHEST of the current candidates
    return [[x, y] for _, x, y in heap]   # the k closest, in arbitrary order
```

+ Optimal complexity, and what to say out loud

Time: O(n log k). One push and at most one pop per point, each O(log k) on a heap bounded at k.

Space: O(k) for the heap, plus the output.

Say this out loud: "Top-K, so a heap of size k rather than a full sort. I want the k smallest distances, so it is a max-heap: its root is the farthest point I am currently keeping, which is exactly what a closer point should displace. And I compare squared distances, because the square root is monotonic and changes nothing about the ordering."

The two things to volunteer: skipping `sqrt`, and Quickselect as the O(n)-average alternative. **The follow-up:** if the points arrive as an unbounded stream, the heap is the only workable answer, since Quickselect requires random access to the entire dataset.

23 Merge k Sorted Lists ★★

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Min-heap of list heads (alternative: divide and conquer) · Optimal: **O(N log k) time, O(k) space**

1 Clarification communication

1. Each of the `k` lists is individually sorted ascending, and the output must be one sorted list?
2. Can the input array contain **empty lists**, or be empty itself? Both need guarding, and a null head pushed into a heap is a crash.
3. Should I **reuse the existing nodes** by relinking them, or allocate new ones? Relinking is O(1) extra per node and is what I will do.
4. Roughly what are `k` and the total number of nodes `N`? That determines whether O(N log k) beats the alternatives.
5. Can the values repeat across lists? Yes, and that is precisely why I need a tiebreaker in the heap tuple.

2 Understanding of the question

Given `k` sorted linked lists, merge them into one sorted linked list.

The structural insight: at every step, the next node of the output must be the smallest among the **current heads** of the k lists. Nothing deeper in any list can be smaller than its own head, since each list is already sorted. So the whole problem reduces to repeatedly answering "which of these k candidates is smallest?", after which that list's head advances by one.

"Repeatedly extract the minimum from a small changing set" is the textbook definition of a min-heap. Bounding the heap at k rather than N is what makes this O(N log k) instead of O(N log N).

3 Brute-force approach in English

Two natural brute forces, and both are worth naming.

The first is **collect and sort**: walk every list, dump all N values into one array, sort it, and build a fresh linked list from the sorted values. Simple and reliable, but it discards the fact that the inputs were already sorted, which is the entire structure the problem hands you.

The second is **repeated linear scan**: keep the k current heads in an array, and on each step scan all k of them to find the smallest, append it, and advance that pointer. This does use the sortedness, but it re-scans all k heads for every one of the N output nodes.

4 Data structures used

- Collect-and-sort: an array of all N values, plus newly allocated nodes.
- Repeated scan: an array of the k current head pointers.

5 Step-by-step algorithm

1. **Collect and sort:** traverse every list, appending each node's value to one array.
2. Sort the array ascending.
3. Build a new linked list from the sorted values using a dummy head, and return `dummy.next`.
4. **Repeated scan:** copy the k head pointers into an array.
5. Repeat until every pointer is null: scan the array for the index holding the smallest non-null value, append that node to the output tail, and advance that entry to `node.next`.

6 Time & space complexity with explanation

Collect and sort: O(N log N) time, O(N) space for the value array and the new nodes. The log N factor comes from sorting data that was already largely ordered.

Repeated scan: O(N · k) time, O(k) space. Each of the N output nodes costs a linear scan over k heads. With k = 10,000 lists this becomes hopeless, and it is the version the heap directly improves.

7 Fully commented brute-force code

```
def mergeKLists_collect(lists):
    """Brute force 1 -- dump every value, sort, rebuild. O(N log N), O(N) space.
    Throws away the fact that the inputs were already sorted."""
    values = []
    for node in lists:
        while node:
            values.append(node.val)
            node = node.next
    values.sort()

    dummy = ListNode()
    tail = dummy
    for v in values:
        tail.next = ListNode(v)           # allocates N new nodes
        tail = tail.next
    return dummy.next


def mergeKLists_scan(lists):
    """Brute force 2 -- scan all k heads for the minimum, N times. O(N*k)."""
    heads = list(lists)
    dummy = ListNode()
    tail = dummy
    while True:
        best_i = -1
        for i, node in enumerate(heads):
            if node and (best_i == -1 or node.val < heads[best_i].val):
                best_i = i
        if best_i == -1:
            break                         # every list exhausted
        tail.next = heads[best_i]         # relink, no allocation
        tail = tail.next
        heads[best_i] = heads[best_i].next
    return dummy.next
```

8 Limitation of brute force

Collect-and-sort ignores the input structure entirely: the lists were already sorted, and re-sorting from scratch pays a log N factor for information that was handed to you for free. It also allocates N new nodes and holds all N values in memory at once, which rules it out if the lists are huge or streaming.

Repeated scan uses the sortedness but re-derives the same comparison over and over: after taking the minimum, k−1 of the k heads are completely unchanged, yet all k are compared again on the next step.

A min-heap fixes exactly that. It remembers the ordering among the k candidates, so replacing one candidate costs O(log k) rather than a fresh O(k) scan — and the heap holds only k entries, so memory stays proportional to the number of lists, not the number of nodes.

9 Optimal approach in English

Push the head of every non-empty list into a min-heap. Then repeatedly pop the smallest node, append it to the output tail, and — if that node has a successor — push the successor. The heap therefore always holds exactly one candidate per still-live list, at most k entries.

The detail that makes or breaks the implementation: the heap entries must be **tuples of `(value, index, node)`**. Python compares tuples element by element, so when two nodes have equal values it moves on to the next component. Without a tiebreaker it would reach the `ListNode` objects and raise `TypeError: '<' not supported between instances of 'ListNode'`. Inserting the list index as a second component guarantees the comparison always resolves on an integer and never touches the node. This is the single most common crash in this problem, so I would state it while typing the tuple.

A **dummy head** node removes every special case around the first append: the tail pointer always has somewhere to write, and the real result is `dummy.next`.

The alternative to mention is **divide and conquer**: merge the lists pairwise, halving the number of lists on each round, for log k rounds each costing O(N). Same O(N log k) time, but O(1) extra space instead of O(k), and no comparison-tiebreaker issue at all.

10 Optimal data structures

- A **min-heap holding at most k entries**, one per live list. Bounded at k, which is why the log factor is log k and not log N.
- **Tuples `(value, list_index, node)`** — the middle element is a pure tiebreaker to keep Python from comparing nodes.
- A **dummy head** node plus a moving `tail` pointer, so no branch is needed for the first element.

11 Step-by-step optimal algorithm

1. Create an empty heap. For each list index `i` with a non-null head, push `(head.val, i, head)`. Skipping null heads is essential; pushing `None` crashes on the first comparison.
2. Create `dummy = ListNode()` and set `tail = dummy`.
3. While the heap is non-empty: pop `(value, i, node)`, the globally smallest current head.
4. Link it in: `tail.next = node`, then `tail = node`. This relinks the existing node rather than allocating a new one.
5. If `node.next` is not null, push `(node.next.val, i, node.next)`, replacing the list's candidate.
6. When the heap empties, return `dummy.next`.
7. Optionally set `tail.next = None` at the end for hygiene; here it is already correct because the final popped node was the tail of its own list.

12 Fully commented optimal code

```
import heapq

def mergeKLists(lists):
    """Min-heap holding one candidate per live list: at most k entries."""
    heap = []
    for i, node in enumerate(lists):
        if node:                          # never push a null head
            # *** i is a TIEBREAKER. Without it, equal values make Python compare
            # ListNode objects -> TypeError: '<' not supported. ***
            heapq.heappush(heap, (node.val, i, node))

    dummy = ListNode()                    # dummy head: no special case for the first node
    tail = dummy

    while heap:
        _, i, node = heapq.heappop(heap)  # globally smallest current head
        tail.next = node                  # relink the existing node (no allocation)
        tail = node
        if node.next:                     # refill this list's candidate slot
            heapq.heappush(heap, (node.next.val, i, node.next))

    return dummy.next


def mergeKLists_divide(lists):
    """Alternative -- pairwise merge, halving the list count each round.
    Same O(N log k) time, but O(1) extra space and no tiebreaker needed."""
    def merge_two(a, b):
        dummy = ListNode()
        tail = dummy
        while a and b:
            if a.val <= b.val:
                tail.next, a = a, a.next
            else:
                tail.next, b = b, b.next
            tail = tail.next
        tail.next = a or b                # attach whatever remains
        return dummy.next

    if not lists:
        return None
    while len(lists) > 1:                 # log k rounds, each O(N) total
        merged = []
        for i in range(0, len(lists), 2):
            second = lists[i + 1] if i + 1 < len(lists) else None
            merged.append(merge_two(lists[i], second))
        lists = merged
    return lists[0]
```

+ Optimal complexity, and what to say out loud

Time: O(N log k) where N is the total number of nodes. Each node is pushed once and popped once, and every heap operation costs O(log k) because the heap never exceeds k entries.

Space: O(k) for the heap. The output reuses the input nodes, so no extra node allocation — unlike collect-and-sort, which allocates N.

Say this out loud: "The next output node is always the smallest of the k current heads, so this is repeated extract-min over a set of size k, which is exactly a min-heap. Each of the N nodes enters and leaves the heap once at O(log k), so O(N log k) time and O(k) space."

The gotcha to volunteer: the index tiebreaker in the tuple, and why — equal values would otherwise force Python to compare `ListNode` objects and raise `TypeError`. **The alternative answer:** divide-and-conquer pairwise merging, same time bound, O(1) extra space.

621 Task Scheduler ★ Amazon favourite

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Greedy counting / closed-form arithmetic · Optimal: **O(n) time, O(26) space**

1 Clarification communication

1. The cooldown `n` is the number of intervals that must separate two runs of the **same** task, so identical tasks must be at least `n + 1` apart?
2. May the CPU sit **idle**, and does each idle slot count as one unit of time? Yes on both, and that is what the answer is measuring.
3. Are tasks always uppercase letters A–Z? That is what bounds the counting space at 26.
4. Do I return only the total time, or the actual schedule? The formula gives the total; producing a schedule needs a different, simulation-based approach, so I want to know before I choose.
5. Can `n` be 0? Then no cooldown applies and the answer is simply the number of tasks — a good check that my formula degrades correctly.

2 Understanding of the question

Given a list of tasks and a cooldown `n`, each unit of time the CPU runs one task or idles, and two identical tasks must be separated by at least `n` units. Return the least total time needed.

The insight that removes all simulation: the answer is dictated entirely by the **most frequent** task. If task A occurs `max_freq` times, those occurrences must be spread out, creating a rigid skeleton of `max_freq − 1` blocks each of width `n + 1` — one slot for A and `n` slots of cooldown — followed by a final A. Every other task simply fills the cooldown slots inside that skeleton.

It is worth drawing this on the whiteboard: a grid with `max_freq` rows and `n + 1` columns, the first column entirely A's, and the remaining cells filled with other tasks or left idle.

3 Brute-force approach in English

Simulate the clock one time unit at a time, greedily. Keep the remaining count of each task and the next time at which each task becomes legal again. At each tick, among the tasks that still have work left and are off cooldown, run the one with the **highest remaining count**; if none is available, idle. Either way advance the clock by one.

The greedy choice of "most remaining first" is correct because the task with the most remaining occurrences is the most constrained — delaying it can only force idle slots later — so scheduling it as early as legally possible is never worse.

4 Data structures used

- A **counter** of remaining occurrences per task.
- A map from task to the next time it is allowed to run again.
- An integer clock. Alternatively a max-heap of remaining counts plus a cooldown queue, which is the more idiomatic simulation and is still slower than the formula.

5 Step-by-step algorithm

1. Build `remaining`, a counter of how many times each task must still run, and `available_at`, a map from task to the earliest legal time, initialised to 0.
2. Set `time = 0` and `left` to the total number of tasks.
3. While `left > 0`: among all tasks with `remaining > 0` and `available_at <= time`, choose the one with the largest `remaining`.
4. If such a task exists, decrement its `remaining`, set its `available_at` to `time + n + 1`, and decrement `left`.
5. If none exists, the CPU idles for this tick.
6. Increment `time` and repeat.
7. Return `time`.

6 Time & space complexity with explanation

Time: O(T · 26) where T is the final answer, the total number of time units. Each tick scans the 26 possible tasks to pick the best available one. Since T can be roughly `n × max_freq`, and `n` may be large while the task list is short, this can be far larger than the input — it is output-sensitive rather than input-sensitive, which is the warning sign.

Space: O(26) = O(1) for the two maps.

7 Fully commented brute-force code

```
from collections import Counter

def leastInterval_brute(tasks, n):
    """Greedy tick-by-tick simulation: always run the task with the most
    remaining occurrences that is currently off cooldown."""
    remaining = Counter(tasks)
    available_at = {t: 0 for t in remaining}
    time = 0
    left = len(tasks)

    while left > 0:
        pick = None
        for t, cnt in remaining.items():
            if cnt > 0 and available_at[t] <= time:
                # most-remaining-first: the most constrained task goes earliest
                if pick is None or cnt > remaining[pick]:
                    pick = t
        if pick is not None:
            remaining[pick] -= 1
            available_at[pick] = time + n + 1   # cooldown before it may run again
            left -= 1
        # else: the CPU idles for this tick
        time += 1

    return time
```

8 Limitation of brute force

The simulation's running time is proportional to the **answer**, not to the input. With `tasks = ['A', 'B']` and `n = 10⁶`, the input has two elements but the loop runs a million times. Any algorithm whose cost scales with the output value rather than the input size is a red flag in an interview.

And the simulation is unnecessary, because the structure of the optimal schedule is fully determined in advance by the frequency counts. The most frequent task fixes the skeleton, and every other task either fits into the gaps or overflows them. Both cases can be evaluated with arithmetic in constant time once the counts are known.

9 Optimal approach in English

Count the task frequencies. Let `max_freq` be the highest, and `num_max` the number of tasks achieving it.

The most frequent task creates `max_freq - 1` gaps, each of which must be `n + 1` units wide — one slot for the task itself and `n` for its cooldown. That accounts for `(max_freq - 1) × (n + 1)` units. Then the final round must contain one slot for each task tied at the maximum frequency, adding `num_max`. So the skeleton has length `(max_freq - 1) * (n + 1) + num_max`.

But that formula assumes the cooldown gaps have empty space to spare. If there are many distinct tasks, they fill every gap and then some, and the CPU never idles at all — in which case the answer is simply the total number of tasks. The two cases are unified by taking the maximum: `max(skeleton, len(tasks))`.

That clamp is the part people miss, and it is where the whole solution is verified or broken. The formula can never *undercount*, because the skeleton is a genuine lower bound; and it can never *overcount*, because whenever extra tasks exist they land in idle slots that were already paid for.

10 Optimal data structures

- A `Counter` over at most 26 tasks. Nothing else — no heap, no queue, no clock.
- Two integers derived from it: `max_freq` and `num_max`.
- The realisation that the answer depends only on those two numbers plus `len(tasks)`, not on the arrangement, is the entire optimisation.

11 Step-by-step optimal algorithm

1. Build `count = Counter(tasks)`.
2. Let `max_freq = max(count.values())`, the highest frequency.
3. Let `num_max` be the number of distinct tasks whose count equals `max_freq`.
4. Compute `slots = (max_freq - 1) * (n + 1) + num_max`. The first term is the `max_freq - 1` full blocks of width `n + 1`; the second is the final partial round holding one slot per maximum-frequency task.
5. Return `max(slots, len(tasks))`. The clamp handles the case where there is enough task variety that every cooldown gap is filled and no idling is ever required.
6. On the whiteboard, draw a grid with `max_freq` rows and `n + 1` columns to make the derivation visible; it is far more convincing than stating the formula.

12 Fully commented optimal code

```
from collections import Counter

def leastInterval(tasks, n):
    """No simulation. The most frequent task fixes a rigid skeleton; every other
    task either fills the gaps or overflows them."""
    count = Counter(tasks)
    max_freq = max(count.values())                       # length of the tallest column
    num_max = sum(1 for v in count.values() if v == max_freq)   # ties at the top

    #   (max_freq - 1) full blocks, each (n + 1) wide  ... the cooldown skeleton
    # + num_max                                        ... the final partial round
    slots = (max_freq - 1) * (n + 1) + num_max

    # *** the clamp: with enough task variety, every gap is filled and the CPU
    # never idles, so the answer is just the number of tasks. ***
    return max(slots, len(tasks))
```

+ Optimal complexity, and what to say out loud

Time: O(n) where n is the number of tasks, spent building the counter. Everything after that is constant-time arithmetic over at most 26 counts.

Space: O(26) = O(1) for the counter.

Say this out loud: "No simulation is needed. The most frequent task forces max_freq − 1 gaps of width n + 1, and the last round holds one slot per task tied at that frequency. That skeleton is a lower bound. If there is enough variety to fill every gap, there is no idling and the answer is just the number of tasks — hence the max. O(n) time, O(26) space."

The clamp `max(slots, len(tasks))` is what people miss. Draw the grid: rows = max_freq, columns = n + 1.

The follow-up: if the interviewer wants the actual schedule rather than its length, the formula does not suffice and you need the max-heap simulation — a max-heap of remaining counts plus a cooldown queue — which is worth naming.

347 Top K Frequent Elements (stretch)

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Hash count + heap (O(n) follow-up: bucket sort) · Optimal: **O(n log k) time, O(n) space**

1 Clarification communication

1. Is `k` guaranteed to be at most the number of **distinct** elements?
2. If two elements tie in frequency at the boundary, may I return either one?
3. Does the output order matter, or is any order of the k most frequent acceptable?
4. Are the values arbitrary integers? Hashing handles that, but it rules out an array-indexed count.
5. Is an O(n log k) solution acceptable, or do you want the strict O(n)? I have the bucket-sort version ready either way — it is the standard follow-up.

2 Understanding of the question

Given an integer array and an integer `k`, return the `k` most frequent elements.

This decomposes cleanly into two independent steps. **Step one:** compute the frequency of each distinct value, which a hash map does in one linear pass. **Step two:** select the k largest by that frequency, which is a top-K problem exactly like 215 and 973.

Keeping the two steps separate is the clean way to present it, because it makes obvious that only the second step has any room for optimisation — and that the second step is where the O(n) follow-up lives.

3 Brute-force approach in English

Count without a hash map, then sort. For each distinct value in the array, scan the whole array counting its occurrences, producing a list of `(value, frequency)` pairs. Sort that list by frequency descending and return the first `k` values.

The counting-by-rescanning is the genuinely brute-force part; the sort is the obvious baseline for the selection step. Naming both separately is useful, because the two get optimised by different techniques — hashing for the first, a heap or bucket sort for the second.

4 Data structures used

- A list of `(value, frequency)` pairs.
- A `seen` set so each distinct value is only counted once.

5 Step-by-step algorithm

1. Create an empty `seen` set and an empty list of pairs.
2. For each value `x` in the array: if `x` is already in `seen`, skip it.
3. Otherwise add `x` to `seen`, scan the entire array counting occurrences of `x`, and append `(x, count)` to the list.
4. Sort the list by frequency descending.
5. Return the first `k` values from it.

6 Time & space complexity with explanation

Time: O(n · d) where d is the number of distinct values, so **O(n²)** when most elements are distinct, plus O(d log d) for the sort.

Space: O(d) for the pair list and the seen set.

7 Fully commented brute-force code

```
def topKFrequent_brute(nums, k):
    """Count each distinct value by rescanning the array, then sort. O(n^2)."""
    seen = set()
    pairs = []
    for x in nums:
        if x in seen:
            continue
        seen.add(x)
        c = 0
        for y in nums:                    # a FULL rescan per distinct value
            if y == x:
                c += 1
        pairs.append((x, c))
    pairs.sort(key=lambda p: p[1], reverse=True)
    return [v for v, _ in pairs[:k]]
```

8 Limitation of brute force

Two separate inefficiencies, worth separating because they have different fixes.

The counting rescans the array once per distinct value, when a single pass with a hash map computes every count at once — the same membership-versus-scan trade as Two Sum.

The selection fully sorts all d distinct values when only the top k are needed, computing and discarding the ordering of the other d−k. A size-k heap reduces that to O(d log k), and because the frequencies are bounded integers, bucket sort removes the log factor entirely.

9 Optimal approach in English

Step one: one pass with a `Counter` gives every frequency in O(n).

Step two, the O(n log k) version: feed the `(value, frequency)` pairs into a size-k heap keyed on frequency, exactly as in problem 215. `heapq.nlargest(k, count.items(), key=...)` does this in one line and is the idiomatic answer.

Step two, the O(n) follow-up — bucket sort. This is what the interviewer is usually fishing for. A frequency can never exceed `n`, so create `n + 1` buckets indexed by frequency, and place each value into the bucket matching its count. Then walk the buckets from the highest index downward, collecting values until `k` have been gathered. There is no comparison sorting at all, so no log factor: the bounded key range is what makes it possible, the same way counting sort beats comparison sort when the key range is small.

I would present the heap version as the main answer for its clarity, then immediately offer bucket sort as the linear-time follow-up.

10 Optimal data structures

- A `Counter` from value to frequency, built in one pass.
- Either a **size-k heap** over the counter's items, giving O(n log k);
- or a **bucket array of length n + 1**, indexed by frequency, each cell holding the list of values with that exact frequency, giving O(n). Indexing by frequency is legal only because frequencies are integers bounded by n — that bound is the whole justification.

11 Step-by-step optimal algorithm

1. Build `count = Counter(nums)` in a single O(n) pass.
2. **Heap version:** return `[value for value, freq in heapq.nlargest(k, count.items(), key=lambda p: p[1])]`.
3. **Bucket version:** create `buckets` as a list of `n + 1` empty lists, indexed by frequency.
4. For each `(value, freq)` in the counter, append `value` to `buckets[freq]`.
5. Walk `freq` downward from `n` to `1`, appending every value in `buckets[freq]` to the output.
6. Stop as soon as the output holds `k` values, and return it.

12 Fully commented optimal code

```
import heapq
from collections import Counter

def topKFrequent(nums, k):
    """O(n log k) -- count in one pass, then a size-k heap over the counts."""
    count = Counter(nums)
    return [x for x, _ in heapq.nlargest(k, count.items(), key=lambda p: p[1])]


def topKFrequent_buckets(nums, k):
    """The O(n) follow-up -- bucket sort by frequency.
    Legal because a frequency is an integer in [1, n], so it can index an array."""
    count = Counter(nums)

    # buckets[f] = every value that occurs exactly f times
    buckets = [[] for _ in range(len(nums) + 1)]
    for value, freq in count.items():
        buckets[freq].append(value)

    out = []
    for freq in range(len(nums), 0, -1):  # walk from the highest frequency down
        for value in buckets[freq]:
            out.append(value)
            if len(out) == k:
                return out
    return out
```

+ Optimal complexity, and what to say out loud

Heap version — time: O(n log k). O(n) to count, then O(d log k) to select over d distinct values, and d ≤ n. **Space: O(n)** for the counter plus O(k) for the heap.

Bucket version — time: O(n). O(n) to count, O(d) to distribute into buckets, and O(n) to walk them. **Space: O(n)** for the counter and the bucket array.

Say this out loud: "Two steps: count with a hash map in O(n), then select the top k. Sorting the counts is O(d log d) and computes more than I need; a size-k heap is O(d log k). But frequencies are integers bounded by n, so I can bucket by frequency and walk the buckets downward — that is O(n) with no comparisons at all."

Offer the bucket-sort follow-up without being asked. The general principle is worth stating too: when sort keys are integers from a small bounded range, counting or bucket sort beats any comparison sort, because the O(n log n) comparison lower bound simply does not apply.

295 Find Median from Data Stream ★★ (stretch)

Day 3 · Thu 27 Aug · Round Type C (intervals, heaps, top-K) · Pattern: Two heaps balanced around the median · Optimal: **addNum O(log n), findMedian O(1)**

1 Clarification communication

1. How often is `findMedian` called relative to `addNum`? If reads are far more frequent than writes, I would consider making writes more expensive to make reads O(1) — and the two-heap design happens to give O(1) reads anyway.
2. Are the numbers integers or floats, and is the range bounded? A bounded small integer range would allow a counting-based solution instead.
3. For an even count, the median is the average of the two middle values, returned as a float?
4. Can `findMedian` be called before any number is added? I will assume it is only called on a non-empty stream, but I would guard it if not.
5. Is the total volume bounded? If it is unbounded and enormous, an approximate quantile sketch such as t-digest is the real production answer — worth naming.

2 Understanding of the question

Design a structure supporting `addNum(x)` to insert a value into a stream, and `findMedian()` to return the median of everything inserted so far.

The median only depends on the **middle** of the sorted order, so maintaining a full sorted sequence is more than the problem requires. What is actually needed is the boundary between the lower half and the upper half, and access to the one or two elements adjacent to that boundary.

That leads directly to the design: split the data into two halves and keep each half in a heap oriented so that the elements nearest the boundary sit at the roots. The lower half goes in a **max**-heap, whose root is the largest small value; the upper half in a **min**-heap, whose root is the smallest large value. Those two roots are exactly the candidates for the median, so reading it is O(1).

3 Brute-force approach in English

Keep every number in a plain list. `addNum` appends in O(1). `findMedian` sorts the list and picks the middle element, or averages the two middle elements when the count is even.

A slightly better variant keeps the list sorted at all times by binary-searching for the insertion position and inserting there — the search is O(log n) but the insertion itself shifts elements and costs O(n). That gives O(n) writes and O(1) reads, which is the mirror image of the sort-on-read version's O(1) writes and O(n log n) reads.

4 Data structures used

- A single **list** of all values seen so far.
- For the sorted variant, `bisect.insort`, which finds the position in O(log n) but still performs an O(n) memory move.

5 Step-by-step algorithm

1. **Sort on read:** `addNum(x)` appends `x` to the list, O(1).
2. `findMedian()` sorts a copy of the list, then returns the middle element if the length is odd, or the average of the two middle elements if it is even.
3. **Keep sorted on write:** `addNum(x)` uses `bisect.insort` to place `x` at its sorted position, O(log n) to locate plus O(n) to shift.
4. `findMedian()` indexes the middle directly in O(1).
5. Both are correct; both make one operation linear or worse.

6 Time & space complexity with explanation

Sort on read: `addNum` O(1), `findMedian` **O(n log n)**. Over m calls that is O(m · n log n), which collapses if the median is queried often.

Keep sorted on write: `addNum` **O(n)** because of the element shifting, `findMedian` O(1).

Space: O(n) for either, which is unavoidable since every value must be retained.

7 Fully commented brute-force code

```
import bisect

class MedianFinder_brute:
    """Sort on every read. addNum O(1), findMedian O(n log n)."""
    def __init__(self):
        self.data = []

    def addNum(self, num):
        self.data.append(num)

    def findMedian(self):
        s = sorted(self.data)             # re-sorts the whole stream every call
        n = len(s)
        mid = n // 2
        return s[mid] if n % 2 else (s[mid - 1] + s[mid]) / 2


class MedianFinder_sorted:
    """Keep the list sorted on write. addNum O(n) (the shift), findMedian O(1)."""
    def __init__(self):
        self.data = []

    def addNum(self, num):
        bisect.insort(self.data, num)     # O(log n) search + O(n) memory move

    def findMedian(self):
        n = len(self.data)
        mid = n // 2
        return self.data[mid] if n % 2 else (self.data[mid - 1] + self.data[mid]) / 2
```

8 Limitation of brute force

Sorting on read redoes the entire ordering for a single number that has not changed since the last query. Keeping the list sorted avoids that but pays an O(n) memory shift on every insert, since arrays cannot insert in the middle cheaply.

Both are over-solving: they maintain the complete ordering of the stream when the query only ever concerns the middle. If I stop trying to order everything and instead just maintain the **partition** into a lower half and an upper half, each side only needs to expose the single element closest to the boundary — and "give me the extreme element of this set, with O(log n) inserts" is exactly a heap.

9 Optimal approach in English

Keep two heaps. `small` is a **max-heap** holding the lower half, simulated in Python by storing negated values so `heapq`'s min-heap surfaces the largest one. `large` is a **min-heap** holding the upper half. Two invariants are maintained at all times: every element of `small` is at most every element of `large`, and `len(small)` equals `len(large)` or exceeds it by exactly one.

The elegant part is how `addNum` enforces both invariants with no comparisons at all. Push the new value onto `small`; immediately pop `small`'s maximum and push it onto `large`. That second step guarantees the ordering invariant unconditionally, because whatever now sits at the top of `small` is by construction no larger than everything in `large`. But it may have left `large` one too big, so finally, if `len(large) > len(small)`, move `large`'s minimum back to `small`. Three heap operations, no branching on values.

`findMedian` is then O(1). If `small` is larger, the total count is odd and the median is `small`'s root. Otherwise the counts are equal, the total is even, and the median is the average of the two roots.

10 Optimal data structures

- A **max-heap** `small` for the lower half, implemented by negating values on push and negating again on read. Max-oriented so its root is the largest of the small values — the left-hand median candidate.
- A **min-heap** `large` for the upper half, whose root is the smallest of the large values — the right-hand median candidate.
- No other state. The two roots are always exactly the elements adjacent to the median boundary, which is what makes reads O(1).

11 Step-by-step optimal algorithm

1. In the constructor, create `small = []` as the negated max-heap and `large = []` as the min-heap.
2. **addNum(num):** push `-num` onto `small`.
3. Pop the root of `small`, negate it back, and push it onto `large`. This unconditionally restores the ordering invariant: the value moved is the largest of the lower half, so after the move nothing in `small` exceeds anything in `large`.
4. If `len(large) > len(small)`, pop `large`'s minimum and push its negation onto `small`, restoring the size invariant. `small` therefore holds the extra element when the count is odd.
5. **findMedian():** if `len(small) > len(large)`, the count is odd, so return `-small[0]`.
6. Otherwise the count is even, so return `(-small[0] + large[0]) / 2`.

12 Fully commented optimal code

```
import heapq

class MedianFinder:
    """Two heaps straddling the median.
       small: MAX-heap (values negated) holding the lower half
       large: MIN-heap holding the upper half
    Invariants: max(small) <= min(large), and len(small) - len(large) is 0 or 1."""

    def __init__(self):
        self.small = []                  # max-heap via negation -- lower half
        self.large = []                  # min-heap             -- upper half

    def addNum(self, num):
        # 1. Always enter through `small`.
        heapq.heappush(self.small, -num)

        # 2. Move small's maximum into large. This restores the ORDERING invariant
        #    unconditionally -- no value comparison is needed anywhere.
        heapq.heappush(self.large, -heapq.heappop(self.small))

        # 3. Step 2 may have made large too big; restore the SIZE invariant.
        if len(self.large) > len(self.small):
            heapq.heappush(self.small, -heapq.heappop(self.large))

    def findMedian(self):
        if len(self.small) > len(self.large):
            return -self.small[0]         # odd count: small holds the extra element
        return (-self.small[0] + self.large[0]) / 2   # even: average the two roots
```

+ **Optimal complexity, and what to say out loud**

addNum: O(log n). A constant number of heap pushes and pops, each O(log n).

findMedian: O(1). Only the two roots are read; nothing is removed or reordered.

Space: O(n), since every value must be retained across the two heaps.

Say this out loud: "I keep the lower half in a max-heap and the upper half in a min-heap, balanced so their sizes differ by at most one. The two roots are exactly the elements next to the median, so reading it is O(1). On insert I always push into small, move small's max into large — which guarantees the ordering with no comparisons — then rebalance if large got too big. Three heap operations, so O(log n)."

Explain the push-to-small-then-move-to-large dance explicitly; it is the part that looks like a trick until you say why it works. **The follow-ups:** if all values are in a small bounded range such as 0 to 100, a counting array with a running scan is simpler and O(1) amortised; and if 99% of values fall in a known range, keep exact counts inside it and heaps only for the outliers.

875 Koko Eating Bananas ★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Binary search on the answer · Optimal: **O(n log(max pile)) time, O(1) space**

1 Clarification communication

1. If Koko's speed is `k` and a pile has fewer than `k` bananas, does she finish that pile and stop for the hour, rather than moving on to the next pile? Yes — that is what makes the cost per pile `ceil(pile / k)` rather than a continuous division.
2. Is `h` guaranteed to be at least the number of piles? If it were smaller the task would be impossible, since each pile takes at least one hour.
3. Are the pile sizes and `h` positive integers?
4. Must the eating speed be an integer? Yes, which is what makes the answer space a discrete range I can binary search.
5. How large can the piles get? That sets the upper bound of my search range and therefore the log factor.

2 Understanding of the question

Koko eats bananas at some integer speed `k` per hour, finishing at most one pile each hour. Given the piles and a deadline of `h` hours, find the minimum `k` that lets her finish in time.

This is the archetype of **binary search on the answer**, and the recognition rule is worth stating out loud whenever it appears: the problem asks for the *minimum* (or maximum) value of something such that a condition holds, and I can write an O(n) function that checks whether a specific candidate value works.

The property that makes binary search valid is **monotonicity**: if speed `k` finishes in time, then every speed greater than `k` also finishes in time, because eating faster can never take more hours. So the feasibility function looks like `False, False, …, False, True, True, …, True` across the range of speeds, and I am searching for the boundary. That is exactly what binary search finds.

3 Brute-force approach in English

Try every candidate speed in increasing order, starting at 1 and going up to the largest pile, and return the first one that finishes within `h` hours.

Two bounds are worth justifying aloud. Speed 1 is the slowest possible, so nothing below it needs testing. And a speed equal to the largest pile means every pile takes exactly one hour, which is the fastest the task can ever be completed — going faster changes nothing, since Koko cannot start a second pile within the same hour. So the answer always lies in `[1, max(piles)]`.

Because feasibility is monotonic, the first speed that works is by definition the minimum, so returning immediately on the first success is correct.

4 Data structures used

- None. A loop counter and a helper that computes the hours needed for a candidate speed.
- The helper is the important artefact: it is exactly the `feasible()` function the optimal solution reuses unchanged.

5 Step-by-step algorithm

1. Write `hours_needed(speed)`: sum `ceil(p / speed)` over all piles. Using `ceil` is what encodes the rule that a partly eaten pile still consumes a whole hour.
2. Loop `speed` from 1 up to `max(piles)`.
3. If `hours_needed(speed) <= h`, return `speed`.
4. The loop is guaranteed to succeed at or before `max(piles)`, provided `h` is at least the number of piles.

6 Time & space complexity with explanation

Time: O(n · max(piles)). Each candidate speed costs an O(n) pass over the piles, and in the worst case every speed from 1 up to the maximum pile is tried. With piles up to 10^9 this is completely infeasible.

Space: O(1).

7 Fully commented brute-force code

```
import math

def minEatingSpeed_brute(piles, h):
    """Try every speed from 1 upward. O(n * max(piles))."""
    def hours_needed(speed):
        # ceil: a partly eaten pile still costs a whole hour
        return sum(math.ceil(p / speed) for p in piles)

    for speed in range(1, max(piles) + 1):
        if hours_needed(speed) <= h:
            return speed              # monotonic, so the first hit is the minimum
    return max(piles)
```

8 Limitation of brute force

The linear scan tests thousands of speeds that are obviously too slow, one at a time, when a single test already reveals which entire half of the range can be discarded. If speed 500 misses the deadline, every speed below 500 misses it too — and the scan learns nothing from that and simply tries 501 next.

That is precisely the structure binary search exploits. Because feasibility is monotonic in the speed, one O(n) evaluation halves the remaining search space, so the number of evaluations drops from `max(piles)` to `log₂(max(piles))` — about 30 evaluations instead of a billion.

9 Optimal approach in English

Binary search over the **answer space** `[1, max(piles)]` rather than over the array.

Maintain `lo` and `hi` as the inclusive bounds of the range still containing the answer. At each step, test the midpoint. If `hours_needed(mid) <= h`, then `mid` is a feasible speed — but possibly not the smallest — so I keep it as a candidate and search the lower half by setting `hi = mid`. If it is infeasible, `mid` and everything below it are ruled out, so `lo = mid + 1`.

I use the `while lo < hi` form with `hi = mid` rather than `hi = mid - 1`. This is the **lower-bound** template, and it is the one to memorise for every "minimum X such that" problem: it converges on the smallest feasible value and terminates with `lo == hi` pointing at the answer, with no post-loop adjustment and no risk of skipping past the boundary. Because `mid` is computed by flooring and `lo < hi`, `mid` is always strictly less than `hi`, so the range shrinks on every iteration and the loop cannot spin forever.

The move that makes this look easy in the interview is to write `hours_needed` on the whiteboard **first**, before any binary search code. That establishes the feasibility predicate as the thing being searched over, and the search itself is then a template.

10 Optimal data structures

- **No data structure at all.** Two integer bounds. The insight is about the *search space*, not about storage — which is exactly why it belongs in a round about justifying the paradigm.
- The `feasible` helper, here `hours_needed(speed) <= h`, is the reusable component. Problems 1011, Split Array Largest Sum, and Minimum Days to Make Bouquets are all the same search with a different predicate.

11 Step-by-step optimal algorithm

1. Define `hours_needed(speed)` as the sum of `ceil(p / speed)` over all piles. This is the O(n) feasibility check.
2. Set `lo = 1`, the slowest possible speed, and `hi = max(piles)`, beyond which nothing improves because each pile already takes exactly one hour.
3. While `lo < hi`: compute `mid = (lo + hi) // 2`.
4. If `hours_needed(mid) <= h`, `mid` is feasible, so it may be the answer or the answer may be smaller: set `hi = mid`, keeping `mid` inside the range.
5. Otherwise `mid` is too slow, and so is everything below it: set `lo = mid + 1`.
6. When the loop ends, `lo == hi` and that value is the smallest feasible speed. Return `lo`.

12 Fully commented optimal code

```
import math

def minEatingSpeed(piles, h):
    """Binary search the ANSWER SPACE, not the array.
    Valid because feasibility is monotonic: faster is never slower."""

    def hours_needed(speed):
        """O(n) feasibility check -- write this on the board FIRST."""
        return sum(math.ceil(p / speed) for p in piles)

    lo, hi = 1, max(piles)   # lo: slowest possible. hi: 1 hour per pile, the fastest useful.

    # Lower-bound template: converges on the SMALLEST value that satisfies the predicate.
    while lo < hi:
        mid = (lo + hi) // 2
        if hours_needed(mid) <= h:
            hi = mid              # feasible -> keep mid as a candidate, search lower
        else:
            lo = mid + 1          # too slow -> mid and everything below is impossible

    return lo                     # lo == hi == the minimum feasible speed
```

+ Optimal complexity, and what to say out loud

Time: O(n · log(max(piles))). The binary search performs O(log(max pile)) iterations — about 30 for values up to 10^9 — and each iteration runs the O(n) feasibility check.

Space: O(1). Two integers and the generator inside the sum.

Say this out loud: "The answer is an integer between 1 and max(piles), and feasibility is monotonic — if a speed works, every faster speed works. So I binary search the answer space, using an O(n) check for whether a given speed finishes in time. About 30 iterations of O(n) work: O(n log max)."

The recognition rule to state explicitly: the problem says "minimum X such that…" or "maximum X such that…", and I can write an O(n) `feasible(x)`. Naming the rule out loud is the fastest way to show you recognised the pattern rather than recalled a template.

33 Search in Rotated Sorted Array ★★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Modified binary search on a pivoted array · Optimal: **O(log n) time, O(1) space**

1 Clarification communication

1. Are the values **distinct**? This matters a great deal: with duplicates the worst case degrades to O(n), because `nums[lo] == nums[mid]` no longer tells you which half is sorted. The standard version guarantees distinct values.
2. Was the array originally sorted ascending and then rotated left by some unknown amount?
3. Could the rotation be zero, leaving the array fully sorted? My solution must handle that as a normal case.
4. I return the **index** of the target, or −1 if it is absent?
5. Is O(log n) required, or would O(n) be acceptable? I will assume O(log n) is the point of the question.

2 Understanding of the question

A sorted ascending array of distinct integers has been rotated at an unknown pivot. Find the index of a target in O(log n).

The key structural fact: **wherever you split a rotated sorted array at its midpoint, at least one of the two halves is itself properly sorted.** The pivot can only fall in one half, so the other half is a clean ascending run. That is what rescues binary search — I cannot compare the target against `nums[mid]` alone to decide direction, but I *can* identify the sorted half and then test, with two simple comparisons, whether the target lies inside its known range. If it does, search there; if it does not, the target must be in the other half.

So each iteration still discards half the array, and the complexity is unchanged from ordinary binary search.

3 Brute-force approach in English

Scan the array from left to right, returning the index as soon as an element equals the target, and −1 if the scan completes without a match.

This is worth one sentence, not more. Its only virtue is that it ignores the ordering entirely and therefore does not care about the rotation — which is precisely why it is the wrong answer here: the rotation is the structure the question is testing whether you can exploit.

4 Data structures used

- None. One index variable.

5 Step-by-step algorithm

1. Loop `i` from 0 to `n - 1`.
2. If `nums[i] == target`, return `i`.
3. If the loop finishes without a match, return −1.

6 Time & space complexity with explanation

Time: O(n). Every element may be examined.

Space: O(1).

Correct but it throws away the sortedness completely, which is the entire point of the problem. Stating it takes ten seconds and establishes the baseline the O(log n) solution improves on.

7 Fully commented brute-force code

```
def searchRotated_brute(nums, target):
    """Linear scan. Correct, but ignores the sortedness entirely."""
    for i, x in enumerate(nums):
        if x == target:
            return i
    return -1
```

8 Limitation of brute force

A linear scan on sorted data is always leaving information unused: after a single comparison against the midpoint of a sorted array, half the elements can be excluded, and the scan exploits none of that.

The rotation makes the standard comparison insufficient but not useless. It only breaks the *direction* decision, and that is repairable, because at any midpoint one side is guaranteed to be a clean sorted run whose bounds are known. Two comparisons against that run's endpoints restore a correct direction decision, and binary search proceeds as normal.

9 Optimal approach in English

Standard binary search with an extra layer of reasoning at each step.

Compute `mid`. If `nums[mid]` is the target, return immediately. Otherwise decide which half is sorted by comparing `nums[lo]` with `nums[mid]`:

- If `nums[lo] <= nums[mid]`, the **left** half is sorted. The target is in it exactly when `nums[lo] <= target < nums[mid]`. If so, set `hi = mid - 1`; otherwise the target must be in the unsorted right half, so `lo = mid + 1`.
- Otherwise the **right** half is sorted. The target is in it exactly when `nums[mid] < target <= nums[hi]`. If so, `lo = mid + 1`; otherwise `hi = mid - 1`.

The comparison must be `nums[lo] <= nums[mid]`, with the `<=`. When the range narrows to two elements, `mid` equals `lo`, so `nums[lo] == nums[mid]`; with a strict `<` the code would wrongly conclude the right half is sorted and could search the wrong side. That single character is the classic bug in this problem.

The range bounds are inclusive and the loop runs `while lo <= hi`, so a final single-element range is still examined before the loop exits with −1.

10 Optimal data structures

- **No data structure.** Three integer indices: `lo`, `hi`, `mid`. Constant extra space.
- The alternative worth naming: first binary search for the pivot — the index of the minimum — then binary search the appropriate sorted segment. Two clean searches instead of one clever one, same O(log n), and easier to reason about if the branching feels risky under pressure.

11 Step-by-step optimal algorithm

1. Set `lo = 0` and `hi = n - 1`, an inclusive range.
2. While `lo <= hi`: compute `mid = (lo + hi) // 2`.
3. If `nums[mid] == target`, return `mid`.
4. If `nums[lo] <= nums[mid]`, the left half is sorted. Note the `<=` — it is required for the two-element case where `mid == lo`.
5. Within that branch: if `nums[lo] <= target < nums[mid]`, the target lies in the sorted left half, so set `hi = mid - 1`; otherwise set `lo = mid + 1`.
6. Otherwise the right half is sorted. If `nums[mid] < target <= nums[hi]`, set `lo = mid + 1`; otherwise set `hi = mid - 1`.
7. If the loop exits, the target is absent, so return −1.

12 Fully commented optimal code

```
def searchRotated(nums, target):
    """At any midpoint, at least ONE half is properly sorted -- the pivot can
    only fall in one of them. Identify that half, then test whether the target
    lies inside its known range."""
    lo, hi = 0, len(nums) - 1

    while lo <= hi:                       # inclusive range
        mid = (lo + hi) // 2
        if nums[mid] == target:
            return mid

        # *** `<=` not `<`: when the range is two elements, mid == lo ***
        if nums[lo] <= nums[mid]:                 # LEFT half is sorted
            if nums[lo] <= target < nums[mid]:    # target inside that sorted range
                hi = mid - 1
            else:
                lo = mid + 1                      # must be in the messy right half
        else:                                     # RIGHT half is sorted
            if nums[mid] < target <= nums[hi]:
                lo = mid + 1
            else:
                hi = mid - 1

    return -1
```

+ Optimal complexity, and what to say out loud

Time: O(log n). Each iteration performs O(1) comparisons and discards half the remaining range, exactly as in ordinary binary search — identifying the sorted half costs nothing asymptotically.

Space: O(1).

Say this out loud: "Wherever I split it, one half is always properly sorted, because the pivot can only be in one of them. So I identify the sorted half by comparing nums[lo] with nums[mid], then check whether the target lies within that half's known range. Half the array is eliminated each step, so O(log n)."

The gotcha: `nums[lo] <= nums[mid]` with the equals sign, for the two-element case. **The follow-up to have ready:** with duplicates — problem 81 — `nums[lo]` can equal `nums[mid]` while telling you nothing, and the standard fix is to increment `lo` by one and retry, which pushes the worst case to O(n). Saying that unprompted is a strong signal.

322 Coin Change ★★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Bottom-up unbounded-knapsack DP · Optimal: **O(amount · coins) time, O(amount) space**

1 Clarification communication

1. Is the supply of each coin **unlimited**? That is what makes it unbounded knapsack rather than 0/1, and it changes the loop structure.
2. Should I return the **fewest coins**, or the number of ways to make the amount? Those are different problems — problem 518 is the counting version.
3. What do I return when the amount cannot be formed at all — −1?
4. Can the amount be 0? Then the answer is 0 coins, which is my DP base case.
5. Are all coin denominations positive? And are the denominations arbitrary, or a canonical currency system? This matters because **greedy is wrong for arbitrary denominations**, and I would want to make that explicit before choosing an approach.

2 Understanding of the question

Given coin denominations and a target amount, return the fewest coins that sum to the amount, or −1 if it is impossible. Each denomination may be used any number of times.

The first thing to say, unprompted, is that **greedy does not work here**, and to have the counterexample ready: with `coins = [1, 3, 4]` and `amount = 6`, greedy takes 4, then 1, then 1, for three coins, while the optimum is 3 + 3, two coins. Greedy fails because taking the largest coin now can leave a remainder that is awkward to fill.

What makes DP correct instead is **optimal substructure**: if the best solution for amount `a` uses a coin `c`, then the rest of that solution must itself be an optimal solution for `a − c` — otherwise I could substitute a better one and improve the whole. So `dp[a] = 1 + min(dp[a − c])` over all usable coins `c`, and the subproblems overlap heavily, which is exactly when DP beats recursion.

3 Brute-force approach in English

Recursive exhaustive search. To make amount `a`, try each coin `c` that does not exceed `a`, recursively solve for `a − c`, and take the smallest result plus one. The base case is `a == 0`, which needs zero coins; if no coin leads to a solution, report infinity.

This explores every distinct multiset of coins summing to the amount. It is obviously correct, and it is the version worth writing on the board first because the recurrence it exposes, `dp[a] = 1 + min(dp[a − c])`, is literally the DP transition. The optimisation is then just "stop recomputing the same subproblems", which is an easy story to tell.

4 Data structures used

- The **call stack**, up to `amount` frames deep when the smallest coin is 1.
- No memoisation at all — that absence is the entire problem.

5 Step-by-step algorithm

1. Define `solve(rem)`, the fewest coins needed to make `rem`.
2. If `rem == 0`, return 0.
3. If `rem < 0`, this branch is invalid, so return infinity.
4. Otherwise take the minimum of `1 + solve(rem − c)` over every coin `c`.
5. Call `solve(amount)`; if the result is infinity, return −1, otherwise the result.

6 Time & space complexity with explanation

Time: O(c^a) where c is the number of coins and a the amount — exponential. The recursion tree branches c ways at every level and can be up to `amount` levels deep when the smallest coin is 1. For `amount = 100` this is astronomically large.

Space: O(amount) for the recursion depth.

7 Fully commented brute-force code

```
def coinChange_brute(coins, amount):
    """Exhaustive recursion. Exposes the recurrence dp[a] = 1 + min(dp[a - c]),
    which is exactly what the DP fills in -- but recomputes it exponentially often."""
    INF = float('inf')

    def solve(rem):
        if rem == 0:
            return 0                    # base case: nothing left to pay
        if rem < 0:
            return INF                  # overshot: invalid branch
        best = INF
        for c in coins:
            best = min(best, 1 + solve(rem - c))
        return best

    result = solve(amount)
    return -1 if result == INF else result
```

8 Limitation of brute force

The recursion recomputes identical subproblems an enormous number of times. With `coins = [1, 2, 5]`, the value of `solve(7)` is reached through 5+2, through 2+5, through 1+1+... and dozens of other prefixes, and each arrival recomputes the entire subtree beneath it from scratch.

But `solve(7)` has one fixed answer regardless of how it was reached. There are only `amount + 1` distinct subproblems in total, so the exponential blow-up is pure repetition. Storing each result the first time it is computed reduces the work to one evaluation per subproblem — that is memoisation, and turning the recursion inside out into a bottom-up table achieves the same thing without any recursion depth risk.

9 Optimal approach in English

Build a table bottom-up. Let `dp[a]` be the fewest coins needed to make amount `a`. Initialise `dp[0] = 0`, since zero coins make zero, and every other entry to infinity, meaning "not yet known to be reachable".

Then for each amount `a` from 1 to the target, and each coin `c` that does not exceed `a`, consider using `c` as the last coin: that costs `dp[a - c] + 1`. Take the minimum over all coins. Because the loop runs amounts in increasing order, `dp[a - c]` is always already final when it is read.

Iterating **amounts on the outside and coins on the inside** is what makes this the unbounded version: each amount considers every coin freely, so a denomination can be reused any number of times. Reversing the loops and iterating amounts downward would give 0/1 knapsack, where each item is used at most once — a distinction worth naming, because it is the difference between this problem and the coin-counting variant.

At the end, `dp[amount]` is either the answer or still infinity, in which case the amount cannot be formed and I return −1.

10 Optimal data structures

- A **1-D DP array of length** `amount + 1`, where index `a` holds the fewest coins for amount `a`. One dimension suffices because the coin supply is unlimited, so there is no need to track which coins have been consumed.
- `float('inf')` as the sentinel for unreachable, which makes the `min` comparison work without special cases.
- Optionally a parallel `choice` array recording which coin was used at each amount, if the follow-up asks for the actual coins rather than the count.

11 Step-by-step optimal algorithm

1. Create `dp` of length `amount + 1`, with `dp[0] = 0` and every other entry set to infinity.
2. Loop `a` from 1 to `amount` inclusive.
3. For each coin `c`: if `c <= a` and `dp[a - c] + 1 < dp[a]`, set `dp[a] = dp[a - c] + 1`.
4. The guard `c <= a` prevents a negative index; the comparison against `dp[a]` keeps the running minimum.
5. Because amounts are processed in increasing order, `dp[a - c]` is already final whenever it is read.
6. After the loops, return −1 if `dp[amount]` is still infinity, otherwise `dp[amount]`.

12 Fully commented optimal code

```
def coinChange(coins, amount):
    """dp[a] = fewest coins to make amount a.
    Unbounded supply => iterate AMOUNTS outward, coins inside."""
    INF = float('inf')
    dp = [0] + [INF] * amount             # dp[0] = 0: zero coins make zero

    for a in range(1, amount + 1):
        for c in coins:
            # `c <= a` guards the index; use c as the LAST coin and take the best
            if c <= a and dp[a - c] + 1 < dp[a]:
                dp[a] = dp[a - c] + 1

    return -1 if dp[amount] == INF else dp[amount]

# Greedy is WRONG for arbitrary denominations:
#   coins = [1, 3, 4], amount = 6
#   greedy: 4 + 1 + 1 = 3 coins      DP: 3 + 3 = 2 coins
# Have this counterexample ready before they ask for it.
```

+ Optimal complexity, and what to say out loud

Time: O(amount · number_of_coins). Two nested loops with O(1) work inside. Each of the `amount` subproblems is solved exactly once, examining every coin.

Space: O(amount) for the DP array. There is no recursion, so no stack-depth risk on large amounts — a real advantage of the bottom-up form over memoised recursion.

Say this out loud: "Greedy is wrong here, and the counterexample is coins [1,3,4] with amount 6. The problem has optimal substructure — the best way to make a, using coin c, contains the best way to make a−c — and the subproblems overlap heavily, so it is DP. dp[a] = 1 + min over coins of dp[a−c], filled bottom-up. O(amount × coins) time, O(amount) space."

Lead with the greedy counterexample before you are asked. The variants: problem 518, Coin Change II, counts the *number of ways* and requires the loops swapped — coins outside, amounts inside — so that combinations are not double-counted as permutations. Knowing why the loop order differs is a strong signal.

139 Word Break ★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: DP over prefixes · Optimal: **O(n² · k) time, O(n) space**

1 Clarification communication

1. May each dictionary word be reused **any number of times**?
2. Do I return only a boolean, or the actual segmentation? Word Break II asks for all segmentations and needs backtracking with memoisation, which is a different shape.
3. Are matches case-sensitive, and are there any characters outside the alphabet I should expect?
4. Can the dictionary be large? That decides whether I convert it to a set, which I would do regardless.
5. Can `s` be empty? By convention the empty string is segmentable, which is exactly my base case.

2 Understanding of the question

Given a string `s` and a dictionary of words, decide whether `s` can be segmented into a sequence of one or more dictionary words, with reuse allowed.

The reason greedy fails, which is worth stating: taking the longest matching prefix at each step can dead-end. With `s = "aaaab"` and a dictionary of `["aaaa", "aaa", "ab"]`, greedily taking `"aaaa"` leaves `"b"`, which is unmatchable, while taking `"aaa"` first leaves `"ab"`, which succeeds. A local choice cannot be validated locally.

The DP formulation: let `dp[i]` mean "the first `i` characters of `s` can be segmented". Then `dp[i]` is true if there exists some split point `j < i` where `dp[j]` is true **and** the substring `s[j:i]` is a dictionary word. `dp[0]` is true because the empty prefix is trivially segmentable, and that base case is what seeds everything else.

3 Brute-force approach in English

Recursive exploration of every split. Starting at position `start`, try every end position `end` greater than `start`; if `s[start:end]` is a dictionary word, recurse from `end`. If any branch reaches the end of the string, the segmentation succeeds.

This enumerates every possible way of cutting the string. It is clearly correct, and it exposes exactly the same recurrence the DP fills in — the only difference being that the recursion re-solves the same suffixes over and over.

4 Data structures used

- A **set** of dictionary words for O(1) membership.
- The call stack, up to n frames deep.
- No memoisation, which is the flaw.

5 Step-by-step algorithm

1. Convert the dictionary to a set.
2. Define `can(start)`: true if `s[start:]` can be segmented.
3. If `start == len(s)`, the whole string has been consumed, so return True.
4. For each `end` from `start + 1` to `len(s)`: if `s[start:end]` is in the set and `can(end)` is true, return True.
5. If no split works, return False.
6. Call `can(0)`.

6 Time & space complexity with explanation

Time: O(2ⁿ) in the worst case. Every position is either a cut point or not, giving 2^(n−1) possible segmentations, and pathological inputs such as `s = "aaaaaaaaab"` with dictionary `["a", "aa", "aaa"]` actually reach that bound, because almost every prefix matches and almost every branch fails only at the very end.

Space: O(n) for the recursion depth, plus O(dictionary) for the set.

7 Fully commented brute-force code

```
def wordBreak_brute(s, wordDict):
    """Try every cut point recursively. Exponential -- the same suffix is
    re-solved once per distinct path that reaches it."""
    words = set(wordDict)

    def can(start):
        if start == len(s):
            return True                  # consumed the whole string
        for end in range(start + 1, len(s) + 1):
            if s[start:end] in words and can(end):
                return True
        return False

    return can(0)
```

8 Limitation of brute force

The recursion asks "can `s[7:]` be segmented?" once for every distinct way of segmenting the first seven characters — and there can be exponentially many of those. But the answer for position 7 does not depend on how the search arrived there at all.

There are only `n + 1` distinct subproblems, one per starting position. Computing each exactly once collapses the exponential to a polynomial. Memoising the recursion achieves this directly; the bottom-up table is the same computation with the dependency order made explicit and no recursion depth to worry about.

9 Optimal approach in English

Build a boolean array `dp` of length `n + 1`, where `dp[i]` means the first `i` characters are segmentable. Set `dp[0] = True` for the empty prefix.

For each end position `i` from 1 to n, look back at every possible split point `j` from 0 to `i - 1`. If `dp[j]` is true and the substring `s[j:i]` is in the dictionary, then `s[:i]` is segmentable, so set `dp[i] = True` and break out of the inner loop — one witness is enough, and continuing would only find more of them.

The order matters and is what makes it correct: all values `dp[j]` for `j < i` are already final when `dp[i]` is computed, because the outer loop runs left to right. The answer is `dp[n]`.

Two optimisations worth naming. Converting the dictionary to a **set** is essential — the inner membership test would otherwise be a linear scan. And the inner loop can be bounded by the **length of the longest dictionary word**, since a substring longer than that can never match, which in practice turns the $O(n^2)$ inner work into $O(n \cdot L)$.

10 Optimal data structures

- A **boolean DP array of length n + 1** over prefix lengths. `dp[0] = True` is the base case that everything else is built from, and it is the analogue of `count[0] = 1` in problem 560 — the empty-prefix seed.
- A **set** of dictionary words, making each substring test O(L) hashing rather than an O(dictionary) scan.
- Optionally the maximum word length, used to bound the inner loop.

11 Step-by-step optimal algorithm

1. Convert `wordDict` to a set.
2. Create `dp` of length `len(s) + 1`, all False, then set `dp[0] = True` for the empty prefix.
3. Loop `i` from 1 to `len(s)` inclusive — the end of the prefix under consideration.
4. Loop `j` from 0 to `i - 1` — the candidate split point.
5. If `dp[j]` is True and `s[j:i]` is in the set, set `dp[i] = True` and break: one valid witness settles it.
6. Return `dp[len(s)]`.
7. Optional speed-up: start `j` at `max(0, i - max_word_len)` instead of 0, since any longer substring cannot be a dictionary word.

12 Fully commented optimal code

```
def wordBreak(s, wordDict):
    """dp[i] = 'the first i characters of s can be segmented'."""
    words = set(wordDict)                # O(1) membership instead of a scan
    n = len(s)

    dp = [False] * (n + 1)
    dp[0] = True                         # the empty prefix is segmentable -- the seed

    for i in range(1, n + 1):            # end of the prefix
        for j in range(i):               # candidate split point, j < i
            # dp[j] is already final because i runs left to right
            if dp[j] and s[j:i] in words:
                dp[i] = True
                break                    # one witness is enough

    return dp[n]

# Why not greedy? Taking the longest match can dead-end:
#   s = "aaaab", dict = ["aaaa", "aaa", "ab"]
#   greedy "aaaa" -> leaves "b" -> fails;  "aaa" -> leaves "ab" -> succeeds.
```

+ Optimal complexity, and what to say out loud

Time: O(n² · k) where k is the average word length. The two nested loops give O(n²) substring positions, and each `s[j:i]` slice plus hash costs O(k). Bounding the inner loop by the longest dictionary word improves this to O(n · L · k) in practice.

Space: O(n) for the DP array, plus O(total dictionary size) for the set.

Say this out loud: "Greedy fails because a longer match can dead-end, so I need to consider every split. dp[i] means the first i characters are segmentable, and it is true if some earlier true position j has s[j:i] in the dictionary. dp[0] is true for the empty prefix. Two nested loops with an O(k) substring test: O(n²k) time, O(n) space."

The follow-up: Word Break II asks for all valid segmentations, which needs backtracking with memoisation from position to the list of sentences — and the output itself can be exponentially large, so no polynomial bound is possible there. Saying that shows you understand why the two problems have different complexity classes.

146 LRU Cache ★★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Hash map + doubly linked list (design) · Optimal: **get and put both O(1), O(capacity) space**

1 Clarification communication

1. Does a `get` on an existing key count as a **use**, moving it to most-recently-used? Yes, in the standard definition — and it is the detail people forget.
2. Does `put` on an **existing** key also count as a use, updating the value and refreshing its recency?
3. What does `get` return for a missing key — −1?
4. Is the capacity guaranteed to be at least 1?
5. Must both operations be **O(1)**, or is O(log n) acceptable? O(1) is what forces the hash map plus linked list design, so I want that confirmed — it is the whole point of the question.

2 Understanding of the question

Design a fixed-capacity cache supporting `get(key)` and `put(key, value)` in O(1), evicting the least recently used entry when the capacity is exceeded.

The design tension is that two different requirements pull toward two different structures. **O(1) lookup by key** demands a hash map. **O(1) maintenance of a recency ordering**, with the ability to move an arbitrary item to the front and to drop the item at the back, demands a doubly linked list — an array cannot reorder in O(1), and a singly linked list cannot unlink a node without walking to its predecessor.

The resolution is to use **both**, wired together: the hash map stores key → **node reference**, and the nodes live in the linked list. That combination gives O(1) lookup *and* O(1) reordering, because the map hands you the node directly and the doubly-linked structure lets you unlink it without any traversal. Recognising that no single structure can do the job is the insight the question is testing.

3 Brute-force approach in English

Store the entries in a plain list ordered from most to least recently used, and search it linearly. `get` scans the list for the key; if found, it removes the entry from its position and reinserts it at the front, then returns the value. `put` scans for the key, removes it if present, inserts the new entry at the front, and pops from the end if the capacity is exceeded.

This is correct and easy to reason about — the list order *is* the recency order — but every operation involves a linear search and a list shift.

4 Data structures used

- A single **list of `(key, value)` pairs** kept in recency order, most recent first.
- No index structure of any kind, which is what forces the scan.

5 Step-by-step algorithm

1. Store `self.data` as a list of `(key, value)` pairs, most recently used first, and `self.cap`.
2. **get(key):** scan the list for a matching key. If absent, return −1.
3. If found at index `i`, pop it and reinsert it at index 0, then return its value.
4. **put(key, value):** scan for the key; if present, pop it.
5. Insert `(key, value)` at index 0.
6. If the list is now longer than the capacity, pop the last element, which is the least recently used.

6 Time & space complexity with explanation

Time: O(n) per operation, where n is the current number of entries. The linear scan dominates, and `pop(i)` and `insert(0, …)` each shift O(n) elements in a Python list.

Space: O(capacity).

At a realistic cache size of 100,000 entries, this is roughly 100,000 times slower per operation than the target — and a cache is precisely the component that must not be slow.

7 Fully commented brute-force code

```
class LRUCache_brute:
    """List in recency order, searched linearly. O(n) per operation."""

    def __init__(self, capacity):
        self.cap = capacity
        self.data = []                    # [(key, value)], most recent first

    def get(self, key):
        for i, (k, v) in enumerate(self.data):    # O(n) scan
            if k == key:
                self.data.pop(i)          # O(n) shift
                self.data.insert(0, (k, v))       # O(n) shift -- now most recent
                return v
        return -1

    def put(self, key, value):
        for i, (k, _) in enumerate(self.data):    # O(n) scan
            if k == key:
                self.data.pop(i)
                break
        self.data.insert(0, (key, value))
        if len(self.data) > self.cap:
            self.data.pop()               # drop the least recently used
```

8 Limitation of brute force

Two independent O(n) costs. Finding the key requires a scan, because the list is ordered by recency and therefore carries no information about where a given key sits. And reordering requires shifting elements, because arrays store their order positionally.

Each has a standard fix, and the design insight is that they are **different** fixes and both are needed. A hash map removes the search but has no notion of order. A linked list removes the shifting but has no notion of lookup. Combining them — a map whose *values are the list nodes themselves* — makes both operations O(1) simultaneously.

9 Optimal approach in English

A doubly linked list holds the entries in recency order, with the most recently used next to the head and the least recently used next to the tail. A hash map maps each key directly to its **node** in that list.

Use **sentinel head and tail nodes** that never hold data. This is the detail that makes the code short: every real node always has both a predecessor and a successor, so unlinking is four pointer assignments with no null checks and no special cases for the first or last element.

`get(key)`: if the key is absent, return −1. Otherwise fetch the node from the map in O(1), unlink it from its current position, relink it just after the head to mark it most recent, and return its value.

`put(key, value)`: if the key exists, unlink its old node. Create a new node, store it in the map, and insert it just after the head. If the map now exceeds the capacity, take the node just before the tail — the least recently used — unlink it, and delete its key from the map.

The node must store its own key. Eviction begins from the list, giving you a node, and you then need to remove the corresponding entry from the map — which requires knowing the key. Without it you would have to scan the map, which reintroduces the O(n) you just eliminated. This is the single most important line in the whole design and is worth saying aloud.

10 Optimal data structures

- A **hash map from key to node reference**. Not to the value — to the node — because the node is what needs to be repositioned.
- A **doubly linked list** in recency order. Doubly, not singly, because unlinking a node requires its predecessor, and a singly linked list would need an O(n) walk to find it.
- **Sentinel head and tail nodes**, eliminating every null check.
- Each node stores `key`, `value`, `prev`, `next`. The `key` field exists solely so that eviction can clean up the map.

11 Step-by-step optimal algorithm

1. Define a `Node` class with `key`, `val`, `prev`, `next`.
2. In the constructor, create sentinel `head` and `tail` nodes and link them to each other, so the list is never empty from the pointer logic's point of view. Store the capacity and an empty map.
3. Write `_remove(node)`: set `node.prev.next = node.next` and `node.next.prev = node.prev`. Sentinels guarantee both neighbours exist.
4. Write `_add_front(node)`: splice `node` between `head` and `head.next`, updating all four pointers.
5. **get(key):** return −1 if the key is absent. Otherwise fetch the node, `_remove` it, `_add_front` it, and return `node.val`.
6. **put(key, value):** if the key is present, `_remove` its existing node first.
7. Create a new node, set `map[key] = node`, and `_add_front(node)`.
8. If `len(map) > cap`: take `lru = tail.prev`, `_remove(lru)`, and `del map[lru.key]` — which is only possible because the node carries its key.

12 Fully commented optimal code

```
class Node:
    """Doubly linked node. It stores its own KEY so that eviction can also
    delete the corresponding map entry -- without this you would have to scan."""
    def __init__(self, key=0, val=0):
        self.key, self.val = key, val
        self.prev = self.next = None


class LRUCache:
    def __init__(self, capacity):
        self.cap = capacity
        self.map = {}                     # key -> Node (the NODE, not the value)
        # Sentinels: every real node always has a prev and a next,
        # so there is not a single null check anywhere below.
        self.head = Node()                # most-recent side
        self.tail = Node()                # least-recent side
        self.head.next = self.tail
        self.tail.prev = self.head

    def _remove(self, node):
        """Unlink a node in O(1). Needs `prev`, hence a DOUBLY linked list."""
        node.prev.next = node.next
        node.next.prev = node.prev

    def _add_front(self, node):
        """Splice a node in just after head == mark it most recently used."""
        node.next = self.head.next
        node.prev = self.head
        self.head.next.prev = node
        self.head.next = node

    def get(self, key):
        if key not in self.map:
            return -1
        node = self.map[key]
        self._remove(node)                # a read counts as a use...
        self._add_front(node)             # ...so refresh its recency
        return node.val

    def put(self, key, value):
        if key in self.map:
            self._remove(self.map[key])   # drop the stale node; a new one replaces it
        node = Node(key, value)
        self.map[key] = node
        self._add_front(node)

        if len(self.map) > self.cap:
            lru = self.tail.prev          # the node just before the tail sentinel
            self._remove(lru)
            del self.map[lru.key]         # *** only possible because Node stores key ***
```

+ **Optimal complexity, and what to say out loud**

Time: O(1) for both `get` and `put`. The map lookup is O(1) average, and every list operation is a fixed number of pointer assignments — no traversal anywhere.

Space: O(capacity) for the map and the nodes.

Say this out loud: "Two requirements, two structures. O(1) lookup by key needs a hash map; O(1) reordering and eviction needs a doubly linked list. I combine them by mapping keys to nodes, so a lookup hands me the node directly and unlinking it is four pointer writes. Sentinel head and tail remove all the null checks, and the node stores its own key so eviction can clean up the map."

The shortcut and why to avoid it: Python's `OrderedDict` with `move_to_end` and `popitem(last=False)` solves this in about eight lines. Mention that you know it, then write the manual version — interviewers ask this question specifically to see the pointer work. **The follow-up:** LFU Cache, problem 460, which needs a frequency map plus one linked list per frequency and a running minimum-frequency pointer.

138 Copy List with Random Pointer ★★

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Hash map old→new, then the O(1)-space interleave · Optimal: **O(n) time, O(n) space — or O(1) space with the weave**

1 Clarification communication

1. Can a `random` pointer be **null**, point **forward** to a node not yet created, point **backward**, or point at the node itself? All four occur, and the forward case is what makes single-pass copying awkward.
2. Must this be a genuine **deep copy**, where no pointer in the new list refers to any node of the original?
3. May I **modify the original list** during the process, as long as I restore it before returning? That is the precondition for the O(1)-space solution, so I want it confirmed.
4. Can the list be empty?
5. Is there a space constraint? If O(n) auxiliary space is acceptable I would lead with the hash map; the O(1) version is the standard follow-up either way.

2 Understanding of the question

Each node has a `next` pointer and a `random` pointer that may point anywhere in the list or be null. I must produce a deep copy in which every pointer refers only to copied nodes.

The difficulty is entirely the `random` pointer. Copying `next` alone is a trivial single pass, but a `random` pointer may point **forward** to a node whose clone does not exist yet. So I cannot both create and fully wire a node in one step.

The resolution in either solution is the same: **separate creation from wiring.** Create every clone first, so that by the time any pointer is assigned, its target already exists. The two solutions differ only in how they record the correspondence between an original node and its clone — an explicit hash map, or the structure of the list itself.

3 Brute-force approach in English

Copy the `next` chain first, producing a plain clone with all `random` pointers left null. Then, for each original node, determine the **position** of its `random` target by walking from the original head and counting steps until reaching that node. Then walk the same number of steps into the copied list to find the corresponding clone, and assign it.

Position is the correspondence mechanism here: the kth node of the copy corresponds to the kth node of the original. It works, but establishing the position requires a linear walk, and using it requires a second one.

4 Data structures used

- The newly created nodes.
- **No map** — position is used as the correspondence, and computing a position is what costs the linear walk.

5 Step-by-step algorithm

1. First pass: walk the original list creating a clone of each node, linked only by `next`. Remember the head of the copy.
2. Second pass: for each original node `cur`, if `cur.random` is null, leave the clone's `random` as null and continue.
3. Otherwise walk from the original head, counting steps, until reaching `cur.random`; record that count as `idx`.
4. Walk `idx` steps from the copy's head to reach the corresponding clone.
5. Assign that clone to the current clone's `random`.
6. Return the head of the copy.

6 Time & space complexity with explanation

Time: O(n²). Each of the n nodes requires up to two full linear walks — one to find the index of its random target, one to reach that index in the copy.

Space: O(n) for the copied nodes, which is required output rather than overhead.

7 Fully commented brute-force code

```
def copyRandomList_brute(head):
    """Position is the correspondence: the kth clone matches the kth original.
    Finding a position costs a linear walk, so this is O(n^2)."""
    if not head:
        return None

    # Pass 1: clone the `next` chain only.
    new_head = Node(head.val)
    cur, new_cur = head, new_head
    while cur.next:
        new_cur.next = Node(cur.next.val)
        cur, new_cur = cur.next, new_cur.next

    # Pass 2: resolve each random pointer by index.
    cur, new_cur = head, new_head
    while cur:
        if cur.random:
            idx = 0
            probe = head
            while probe is not cur.random:    # O(n): find the target's position
                probe = probe.next
                idx += 1
            target = new_head
            for _ in range(idx):              # O(n): walk to that position in the copy
                target = target.next
            new_cur.random = target
        cur, new_cur = cur.next, new_cur.next

    return new_head
```

8 Limitation of brute force

The algorithm keeps re-answering the same question — "which node is this, and where is its clone?" — by walking the list, when the answer is fixed and could simply be recorded. Using position as the correspondence is the root of the problem: a position is expensive to compute and expensive to use, while the correspondence itself is just a mapping from one node to another.

Storing that mapping directly, in a hash map from original node to clone, makes both directions O(1) and reduces the whole algorithm to two linear passes. And if even the map is too much memory, the correspondence can be encoded **into the list's own structure** by placing each clone immediately after its original — which removes the auxiliary space entirely.

9 Optimal approach in English

Optimal A — the hash map. Lead with this. Pass one: walk the original list and create a clone of each node, recording `old_to_new[original] = clone`. Wire nothing yet. Pass two: now that every clone exists, walk again and set both pointers by lookup — `old_to_new[cur].next = old_to_new.get(cur.next)` and `old_to_new[cur].random = old_to_new.get(cur.random)`.

Use `.get(...)` rather than `[...]`: `cur.next` is `None` at the tail and `cur.random` may be `None` anywhere, and `.get(None)` returns `None` instead of raising `KeyError`. That is a genuinely elegant use of `.get` — the null case is handled by the same expression as the normal case.

Optimal B — the O(1)-space weave. This is the follow-up they want. Instead of a map, store the correspondence *inside the list*: insert each clone immediately after its original, so the list temporarily reads `A → A' → B → B' → C → C'`. Now the clone of any node `X` is simply `X.next`, with no lookup at all — which means the clone of `X.random` is `X.random.next`. So the random-wiring pass is the single line `cur.next.random = cur.random.next`, read aloud as *"my clone's random is the clone of my random."*

Then unweave: restore each original's `next` and link the clones to each other, guarding the final node where `new.next` is null. Three passes, no auxiliary structure.

10 Optimal data structures

- **Optimal A:** a **hash map from original node to cloned node**. Keying on the node object itself, not on its value, since values may repeat.
- **Optimal B: nothing.** The interleaved list structure *is* the map — the relationship "clone of X" is encoded as "the node right after X". That substitution of structure for storage is the idea worth naming.

11 Step-by-step optimal algorithm

1. **A, pass 1:** walk the list; for each `cur`, set `old_to_new[cur] = Node(cur.val)`. Creating every clone first is what makes forward random pointers resolvable in pass 2.
2. **A, pass 2:** walk again; set `old_to_new[cur].next = old_to_new.get(cur.next)` and `old_to_new[cur].random = old_to_new.get(cur.random)`. Return `old_to_new[head]`.
3. **B, pass 1 (weave):** for each original `cur`, set `cur.next = Node(cur.val, cur.next)`, then jump `cur = cur.next.next` to skip the clone just inserted.
4. **B, pass 2 (randoms):** for each original `cur`, if `cur.random` is not null, set `cur.next.random = cur.random.next`. Advance by two.
5. **B, pass 3 (unweave):** save `new_head = head.next`. Then for each pair, restore `old.next = old.next.next` and set `new.next = new.next.next if new.next else None`, guarding the tail.
6. Return `new_head`. Restoring the original list matters: the caller may still hold a reference to it.

12 Fully commented optimal code

```
def copyRandomList(head):
    """OPTIMAL A -- hash map. O(n) time, O(n) space. Lead with this."""
    if not head:
        return None

    old_to_new = {}

    cur = head
    while cur:                              # PASS 1: create EVERY clone first,
        old_to_new[cur] = Node(cur.val)     # so forward randoms already exist in pass 2
        cur = cur.next

    cur = head
    while cur:                              # PASS 2: wire pointers between clones
        # .get(), not [] -- cur.next is None at the tail and random may be None;
        # .get(None) returns None instead of raising KeyError.
        old_to_new[cur].next   = old_to_new.get(cur.next)
        old_to_new[cur].random = old_to_new.get(cur.random)
        cur = cur.next

    return old_to_new[head]


def copyRandomList_O1(head):
    """OPTIMAL B -- the O(1)-space follow-up. The interleaved list IS the map:
    the clone of X is simply X.next."""
    if not head:
        return None

    # 1. WEAVE:  A -> A' -> B -> B' -> C -> C'
    cur = head
    while cur:
        cur.next = Node(cur.val, cur.next)   # clone sits directly after its original
        cur = cur.next.next                  # skip over the clone we just inserted

    # 2. RANDOMS: "my clone's random is the clone of my random"
    cur = head
    while cur:
        if cur.random:
            cur.next.random = cur.random.next
        cur = cur.next.next

    # 3. UNWEAVE back into two independent lists.
    old, new = head, head.next
    new_head = head.next
    while old:
        old.next = old.next.next                        # restore the original chain
        new.next = new.next.next if new.next else None  # guard the tail
        old, new = old.next, new.next

    return new_head
```

+ **Optimal complexity, and what to say out loud**

Optimal A — time O(n), space O(n). Two passes with O(1) hash operations; the map holds n entries.

Optimal B — time O(n), space O(1) excluding the output. Three passes, no auxiliary structure at all.

Say this out loud: "The obstacle is that a random pointer can point forward to a node whose clone does not exist yet, so I separate creation from wiring: create all clones in pass one, wire everything in pass two. The map version is O(n) space. The follow-up removes it by putting each clone directly after its original, so the clone of X is just X.next — the list structure replaces the map. Then unweave to restore the original."

Present A, then offer B unprompted. The detail to state in A: `.get` rather than `[]` for the null tail. **The detail to state in B:** you must restore the original list before returning, since the caller still holds a reference to it.

1011 Capacity To Ship Packages Within D Days ★ (stretch)

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Binary search on the answer · Optimal: **O(n log(sum weights)) time, O(1) space**

1 Clarification communication

1. Must packages be shipped in the **given order**, without reordering? Yes — that is what makes the feasibility check a simple greedy pass rather than a bin-packing problem, which would be NP-hard.
2. Can a day be left with no packages shipped? It never helps, but confirming it means my greedy fill is optimal.
3. Are the weights positive integers, and is `days` at least 1 and at most the number of packages?
4. I return the **minimum** capacity such that everything ships within `days` days?
5. How large can the total weight be? That sets the top of my search range and therefore the log factor.

2 Understanding of the question

Packages must be shipped in their given order within `days` days. Each day the ship carries a prefix of the remaining packages whose total weight does not exceed its capacity. Find the minimum capacity that makes this possible.

This is the **same problem as Koko Eating Bananas** with a different feasibility function, and recognising that instantly is the point. Both ask for the minimum value of a parameter such that a condition holds; both have a feasibility predicate that is monotonic — a larger capacity never needs *more* days — and computable in O(n). So both are binary search on the answer.

The bounds require a moment's thought and are where this problem differs. The lower bound is `max(weights)`, not 1: a capacity below the heaviest single package could never ship that package on any day, no matter how long you wait. The upper bound is `sum(weights)`, which ships everything in one day.

3 Brute-force approach in English

Try every capacity in increasing order, starting from `max(weights)`, and return the first one that ships everything within `days` days.

The feasibility check itself is a greedy simulation: walk the packages in order, adding each to the current day's load; when the next package would overflow the capacity, start a new day. Greedy is optimal here precisely because the order is fixed — loading as much as possible each day can never require more days than any alternative, since any package left off today must still be carried later and cannot be swapped with a different one.

4 Data structures used

- None. A day counter and a running load inside the check.
- The `days_needed` helper is the reusable artefact, exactly as `hours_needed` was in problem 875.

5 Step-by-step algorithm

1. Write `days_needed(cap)`: set `d = 1` and `cur = 0`, then for each weight `w`, if `cur + w > cap` start a new day by incrementing `d` and resetting `cur` to 0; then add `w` to `cur`. Return `d`.
2. Loop `cap` from `max(weights)` up to `sum(weights)`.
3. If `days_needed(cap) <= days`, return `cap`.
4. Monotonicity means the first feasible capacity is the minimum.

6 Time & space complexity with explanation

Time: O(n · sum(weights)). Each candidate capacity costs an O(n) simulation, and the range of candidates spans from the maximum weight to the total weight, which can be enormous.

Space: O(1).

7 Fully commented brute-force code

```
def shipWithinDays_brute(weights, days):
    """Try every capacity upward. O(n * sum(weights))."""
    def days_needed(cap):
        d, cur = 1, 0
        for w in weights:
            if cur + w > cap:             # would overflow -> start a new day
                d += 1
                cur = 0
            cur += w
        return d

    for cap in range(max(weights), sum(weights) + 1):
        if days_needed(cap) <= days:
            return cap
    return sum(weights)
```

8 Limitation of brute force

Identical to problem 875: the linear scan tests every capacity individually, when a single test already eliminates an entire half of the range. If capacity 500 needs too many days, every capacity below 500 needs at least as many, and the scan makes no use of that.

Since feasibility is monotonic in the capacity, binary search reduces the number of O(n) evaluations from `sum(weights)` to `log₂(sum(weights))` — roughly 25 evaluations instead of tens of millions.

9 Optimal approach in English

Binary search the capacity range `[max(weights), sum(weights)]`, using the same greedy `days_needed` check unchanged.

Take the midpoint. If `days_needed(mid) <= days`, the capacity is sufficient and might not be the smallest such, so keep it in the range with `hi = mid`. Otherwise it is too small, along with everything below it, so `lo = mid + 1`. The `while lo < hi` lower-bound template converges on the smallest feasible capacity with no post-loop adjustment.

The lower bound deserves a sentence of its own in the interview: starting at 1 would be wrong-headed rather than merely slow, since no capacity below the heaviest package can ever work. Stating why `lo = max(weights)` demonstrates you reasoned about the search space rather than pasting a template.

This is the same function as Split Array Largest Sum, problem 410, almost line for line — there, the array is split into `m` subarrays and you minimise the largest subarray sum, which is exactly this greedy check with the roles renamed.

10 Optimal data structures

- **No data structure.** Two integer bounds plus the O(n) greedy feasibility check.
- The transferable component is `days_needed`, which is the same shape as `hours_needed` in problem 875 and the partition check in problem 410.

11 Step-by-step optimal algorithm

1. Define `days_needed(cap)` as the greedy simulation: start on day 1 with an empty load; for each weight, open a new day if adding it would exceed `cap`, then add it. Return the day count.
2. Set `lo = max(weights)` — the heaviest package must fit on some single day — and `hi = sum(weights)`, which ships everything at once.
3. While `lo < hi`: compute `mid = (lo + hi) // 2`.
4. If `days_needed(mid) <= days`, this capacity works, so set `hi = mid`, keeping `mid` as a candidate.
5. Otherwise set `lo = mid + 1`.
6. Return `lo`, which now equals `hi` and is the minimum feasible capacity.

12 Fully commented optimal code

```
def shipWithinDays(weights, days):
    """Same shape as Koko (875): binary search a monotonic answer space."""

    def days_needed(cap):
        """Greedy is optimal because the package ORDER is fixed."""
        d, cur = 1, 0
        for w in weights:
            if cur + w > cap:             # cannot fit today -> open a new day
                d += 1
                cur = 0
            cur += w
        return d

    # *** lo = max(weights), NOT 1: a capacity below the heaviest package
    # could never ship that package on any day. ***
    lo, hi = max(weights), sum(weights)

    while lo < hi:                        # lower-bound template
        mid = (lo + hi) // 2
        if days_needed(mid) <= days:
            hi = mid                      # feasible -> try smaller
        else:
            lo = mid + 1                  # too small -> and so is everything below

    return lo
```

+ Optimal complexity, and what to say out loud

Time: O(n · log(sum(weights) − max(weights))). Roughly 25 to 30 binary search iterations for realistic inputs, each running the O(n) greedy check.

Space: O(1).

Say this out loud: "This is the same pattern as Koko: minimum X such that a condition holds, with an O(n) feasibility check and monotonic feasibility — more capacity never needs more days. So binary search the capacity range. The lower bound is max(weights), because nothing smaller can carry the heaviest package, and the upper bound is the total weight."

The gotcha: `lo = max(weights)`, not 1, and be able to say why. **The family:** Koko Eating Bananas (875), Split Array Largest Sum (410), Minimum Number of Days to Make m Bouquets (1482) — all the same search with a different `feasible()`.

380 Insert Delete GetRandom O(1) ★ (stretch)

Day 4 · Fri 28 Aug · Round Type D (justify the paradigm) · Pattern: Dynamic array + index map, swap-with-last on delete · Optimal: **insert / remove / getRandom all O(1), O(n) space**

1 Clarification communication

1. Must `getRandom` return each element with **equal probability**? That is the requirement that forces a contiguous array, since uniform sampling needs O(1) indexing by position.
2. Are duplicate values allowed? The standard version says no — it is a set. Problem 381 allows duplicates and needs the map to hold a set of indices per value.
3. Should `insert` return False when the value is already present, and `remove` return False when it is absent?
4. Is `getRandom` only ever called on a non-empty structure?
5. Does **average** O(1) suffice, given that hash operations are amortised, or is worst-case O(1) required?

2 Understanding of the question

Design a set supporting `insert`, `remove` and `getRandom`, all in average O(1), where `getRandom` returns a uniformly random current element.

This is another two-structures-in-tension design. **Uniform random selection in O(1)** requires a contiguous array, so a random index maps directly to an element — a hash set cannot do this, because its internal layout is sparse and unordered, and picking a random element from one is O(n). **O(1) lookup and deletion by value** requires a hash map, because finding a value in an array is O(n).

So again: array plus map, wired together. The array holds the values; the map holds value → **its index in that array**. The remaining problem is that deleting from the middle of an array is O(n) because of the shift — and the trick that resolves it is that the *order* of the array is irrelevant here. Nothing about the specification cares where an element sits, so a deleted element can be overwritten by the last element and the array simply truncated.

3 Brute-force approach in English

Pick one structure and accept that one operation will be slow.

With a **list only**: `insert` appends in O(1), but must first scan for duplicates in O(n); `remove` scans for the value and then shifts, O(n); `getRandom` is O(1).

With a **hash set only**: `insert` and `remove` are O(1), but `getRandom` must materialise the set into a list or iterate to a random offset, which is O(n).

Naming both and identifying exactly which operation each one fails is the right way to open this problem — it frames the solution as combining them rather than inventing something new.

4 Data structures used

- Either a single **list**, or a single **hash set**. The point is that neither alone can satisfy all three operations.

5 Step-by-step algorithm

1. **List-only insert:** scan the list for the value; if present return False, otherwise append and return True. O(n).
2. **List-only remove:** scan for the value; if absent return False, otherwise remove it, shifting the tail. O(n).
3. **List-only getRandom:** return `random.choice(self.vals)`. O(1).
4. **Set-only insert / remove:** direct O(1) hash operations.
5. **Set-only getRandom:** `random.choice(list(self.data))` — building the list is O(n) on every single call.

6 Time & space complexity with explanation

List only: insert O(n), remove O(n), getRandom O(1).

Set only: insert O(1), remove O(1), getRandom O(n).

Space: O(n) either way. Each design nails two of the three requirements and fails the third, which is exactly the tension the real solution has to resolve.

7 Fully commented brute-force code

```
import random

class RandomizedSet_list:
    """List only: O(1) random, but O(n) insert and remove."""
    def __init__(self):
        self.vals = []

    def insert(self, val):
        if val in self.vals:              # O(n) scan
            return False
        self.vals.append(val)
        return True

    def remove(self, val):
        if val not in self.vals:          # O(n) scan
            return False
        self.vals.remove(val)             # O(n) shift
        return True

    def getRandom(self):
        return random.choice(self.vals)   # O(1)


class RandomizedSet_set:
    """Set only: O(1) insert and remove, but O(n) getRandom."""
    def __init__(self):
        self.data = set()

    def insert(self, val):
        if val in self.data:
            return False
        self.data.add(val)
        return True

    def remove(self, val):
        if val not in self.data:
            return False
        self.data.remove(val)
        return True

    def getRandom(self):
        return random.choice(list(self.data))   # O(n): rebuilds a list every call
```

8 Limitation of brute force

The list fails because locating a value requires a scan and removing from the middle requires a shift. The set fails because hash tables have no positional index, so uniform sampling cannot be done in constant time.

Adding a map on top of the list removes the scan immediately. What remains is the O(n) shift on deletion — and that is where the real insight lands: **the array's order carries no meaning here.** No operation depends on where an element sits. So instead of shifting everything after the hole, move the *last* element into the hole and shrink the array by one. That is O(1), and the only bookkeeping needed is updating the moved element's index in the map.

9 Optimal approach in English

Keep a list `vals` of the elements and a map `idx` from value to its index in that list.

`insert(val)`: return False if the value is already a key. Otherwise record `idx[val] = len(vals)` and append the value. Both O(1).

`remove(val)`: return False if the value is absent. Otherwise let `i = idx[val]` and take `last = vals[-1]`. Write `last` into position `i`, update `idx[last] = i`, pop the final element, and delete `val` from the map. The array shrinks by one with no shifting.

`getRandom()`: return `random.choice(vals)`, which is a single uniform index into a contiguous array.

The edge case to walk through out loud is **removing the last element**, where `val` and `last` are the same value. The code writes `idx[last] = i`, which re-adds the key being removed — but the subsequent `del self.idx[val]` deletes that same key, since `last == val`, so the map ends up correct. The order of those two statements is what makes it safe, and interviewers do probe this case.

10 Optimal data structures

- A **dynamic array** `vals` holding the elements, giving O(1) uniform random access by index. This is what a hash set cannot provide.
- A **hash map** `idx` from value to its position in `vals`, giving O(1) lookup and O(1) location for deletion. This is what an array cannot provide.
- The **swap-with-last** technique, which exploits the fact that the array's order is semantically irrelevant. That is the observation the whole design turns on.

11 Step-by-step optimal algorithm

1. Constructor: create an empty list `vals` and an empty dict `idx`.
2. **insert(val):** if `val` is in `idx`, return False. Otherwise set `idx[val] = len(vals)`, append `val` to `vals`, and return True.
3. **remove(val):** if `val` is not in `idx`, return False.
4. Let `i = idx[val]` and `last = vals[-1]`.
5. Write `vals[i] = last`, filling the hole with the final element instead of shifting everything after it.
6. Set `idx[last] = i` so the moved element's recorded position stays correct.
7. Pop the final element of `vals`, then `del idx[val]`. This ordering is what makes the val-is-last case correct.
8. **getRandom():** return `random.choice(vals)` — uniform because every element occupies exactly one array slot.

12 Fully commented optimal code

```
import random

class RandomizedSet:
    """Array for O(1) uniform random access; map for O(1) lookup.
    Deletion is O(1) because the array's ORDER is irrelevant here."""

    def __init__(self):
        self.vals = []                    # the elements, in arbitrary order
        self.idx = {}                     # value -> its index inside self.vals

    def insert(self, val):
        if val in self.idx:
            return False
        self.idx[val] = len(self.vals)    # its index is the current end
        self.vals.append(val)
        return True

    def remove(self, val):
        if val not in self.idx:
            return False
        i = self.idx[val]
        last = self.vals[-1]

        self.vals[i] = last               # move the LAST element into the hole
        self.idx[last] = i                # ...and fix its recorded index
        self.vals.pop()                   # shrink by one -- no shifting at all
        del self.idx[val]

        # Edge case, worth walking through aloud: if val IS the last element,
        # then last == val, so `self.idx[last] = i` re-adds the key -- and the
        # `del self.idx[val]` on the next line removes it again. Correct because
        # the delete comes second.
        return True

    def getRandom(self):
        return random.choice(self.vals)   # uniform: one array slot per element
```

+ Optimal complexity, and what to say out loud

Time: O(1) average for all three operations. Insert is a hash write plus an amortised O(1) append. Remove is a constant number of hash and array operations with no shifting. GetRandom is one index into a contiguous array. Hash operations are average-case O(1); worst-case collision behaviour is O(n), which is worth acknowledging if asked for strict worst-case bounds.

Space: O(n) for the array and the map.

Say this out loud: "Uniform random selection in O(1) needs a contiguous array; lookup and delete by value in O(1) needs a hash map. So I keep both, with the map storing each value's index in the array. Deleting from the middle would normally be O(n), but the array's order does not matter here, so I overwrite the hole with the last element, fix that element's index in the map, and pop. All three operations are O(1)."

Walk the val-is-last case out loud — it is the thing interviewers check. **The follow-up:** problem 381 allows duplicates, which changes the map to value → *set of indices*, and the same swap-with-last still works, with a little more bookkeeping when the moved element's index set is updated.